

Week 1 - Practice Assignment Solution _____	2
Week 1 - Graded Assignment Solution _____	6
Week 2 - Practice Assignment Solution _____	12
Week 2 - Graded Assignment Solution _____	15
Week 3 - Practice Assignment Solution _____	22
Week 3 - Graded Assignment Solution _____	33
Week 4 - Practice Assignment Solution _____	54
Qualifier-mock-Solution _____	70
Week 5 - Practice Assignment Solution _____	83
Week 5 - Graded Assignment Solution _____	93
Week 6 - Practice Assignment Solution _____	117
Week 6 - Graded Assignment Solution _____	128
Week 7 - Practice Assignment Solution _____	151
Q2 Mock Test Solution _____	165

Hello computational thinking students. In this tutorial we will see how to solve computational thinking week 1 practice assignment.

Let us start with the instructions given, different font styles or punctuation symbols are used as follows: Italic plus underline represents steps of the procedure, Bold is used for variables, Italic for a field from dataset and Double quotes for a value from dataset or dataset name. These different font styles will help us identify various elements of a procedure.

Question 1: What will the values of **A**, **B** and **C** represent at the end of execution of the given procedure on the “Scores” dataset?

In this question we have to find exact values of three variable A, B and C. So, let us start with the procedure.

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain three variables **A**, **B**, **C** and initialize them to 0

Here in step 2 we have declared three variables A, B, C and also initialized them to 0.

Step 3. If Pile 1 is empty then stop the execution

Step 4. Read the top card in Pile 1

Step 5. If *Date of Birth* < “1 May” then increment **A**

In step 5, variable A is getting incremented if date of birth is less than 1st May which means we are counting the number of students born on or before 30th April.

Step 6. If *Date of Birth* > “30 April” and *Date of Birth* < “1 September” then increment **B**

In this step variable B is getting incremented if date of birth is greater than 30th April and less than 1st September. Which indicated that variable B is counting the number of students born between 1st may and 31st August inclusive.

Step 7. If *Date of Birth* > “31 August” then increment **C**

In step 7 we are checking for date of birth greater than 31st August which means variable C is counting the number of students born after 31st August as in starting from 1st September onwards.

Step 8. Move the current card to another pile called Pile 2 and repeat from step 3

It is a repeat step which indicates that this procedure is getting executed on all the cards available in the dataset. Therefore, if we compile all the date of births from the dataset then we will get categorization like this,

Card no.	DoB	Card no.	DoB	Card no.	DoB
2	4 Jan	1	3 Jun	0	7 Nov
5	8 Feb	3	5 May	4	17 Nov
6	23 Mar	13	6 May	9	6 Dec
7	15 Mar	14	23 Jul	15	22 Sep
8	28 Feb	19	30 Aug	16	30 Dec
10	12 Jan	22	17 Jul	17	14 Dec
11	30 Apr	23	13 May	18	9 Oct
12	14 Jan	25	16 May	20	17 Sep
21	15 Mar	26	22 Jul	24	26 Dec
27	4 Mar	Total (B) = 9		28	10 Sep
Total (A) = 10				29	13 Oct
				Total (C) = 11	

Hence, the answer for this question is 10, 9 and 11 respectively.

Next question 2: The following procedure counts the number of words from the “Paragraph Words” dataset where *part of speech* is “Noun” and *Letter count* is greater than or equal to 4. But the programmer may have made mistakes in one or more steps. Identify all such steps (if any). It is a Multiple Select Question (MSQ).

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain a variable **count** and initialize it to 1

As we all know that the counting always start from 1 therefore initial value of variable count should be 0. Hence, step 2 is incorrect.

We have to check remaining steps as well for mistakes because it is a MSQ type question.

Step 3. If Pile 1 is empty then stop the execution

Step 4. Read the top card in Pile 1

Step 5. If *part of speech* is “Verb” and *Letter count* ≥ 4 then increment **count**

Now, as per the given statement we are looking for nouns, not verbs therefore step 5 is also incorrect.

Step 6. Move the current card to another pile called Pile 2 and repeat from step 3

Hence, this question has two correct answers and we have to mark both those options step 2 and step5.

Moving on to question 3: The given procedure finds the average *total bill amount* of shopping bills from the “Shopping Bills” dataset. Fill in the blanks from the given choices and complete the procedure.

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain two variables **count**, **sum** and initialize them to 0

Step 3. If Pile 1 is empty then stop the execution

Step 4. Read the top card in Pile 1

Now, next two steps 5 and 6 are missing. But we know the purpose of this procedure is to find average total bill amount. And for that we require two values, total number of bills and total bill amount of all those bills. In order to count total number of bills we should have a step like,

Increment **count**

And to calculate total bill amount of all the bills we should have a procedure step like,

Add *Total bill amount* to variable **sum**

There is only one option where both these steps are given. Hence, the correct answer is, **Step 5. Increment **count** and Step 6. Add *Total bill amount* to variable **sum**.**

Question 4: What will the given procedure compute and store in variable **X** if executed on the “Shopping Bills” dataset?

In this question we are maintaining 3 variables A, B, C and initializing them to 0 in step 2. In step 5 variable A’s value will be set to 1 if “Neeraja” has a shopping bill from “SV stores”. Similarly, in step 6 variable B’s value will be set to 1 if she has a bill from “Big Bazaar” and variable C’s value for “Sun General” in step 7.

Step 9 maintains a new variable X and initializes it to 0. Steps 10 to 12 increments that variable X if the value of variables A, B and C are equal to 1 respectively. Values of variables A, B and C will be equal to 1 only if “Neeraja” has visited the corresponding shop. Therefore, variable X will get incremented for all those shops which she visited. Hence, the correct answer is **Number of shops which Neeraja visited.**

Next question 5: The following information represents a card from the “Paragraph Words” dataset. During data entry, the clerk (or operator) might have made a mistake in one or more

lines. Identify such lines with respect to the sanity of data. It is a Multiple Select Question (MSQ).

In this question few lines of information are given which represents a card from the “Paragraph Words” dataset and we have to find all the incorrect lines with respect to sanity of data. Let us read these lines one by one.

Line 1. Card number: 15

Line 2. Word: unplesant

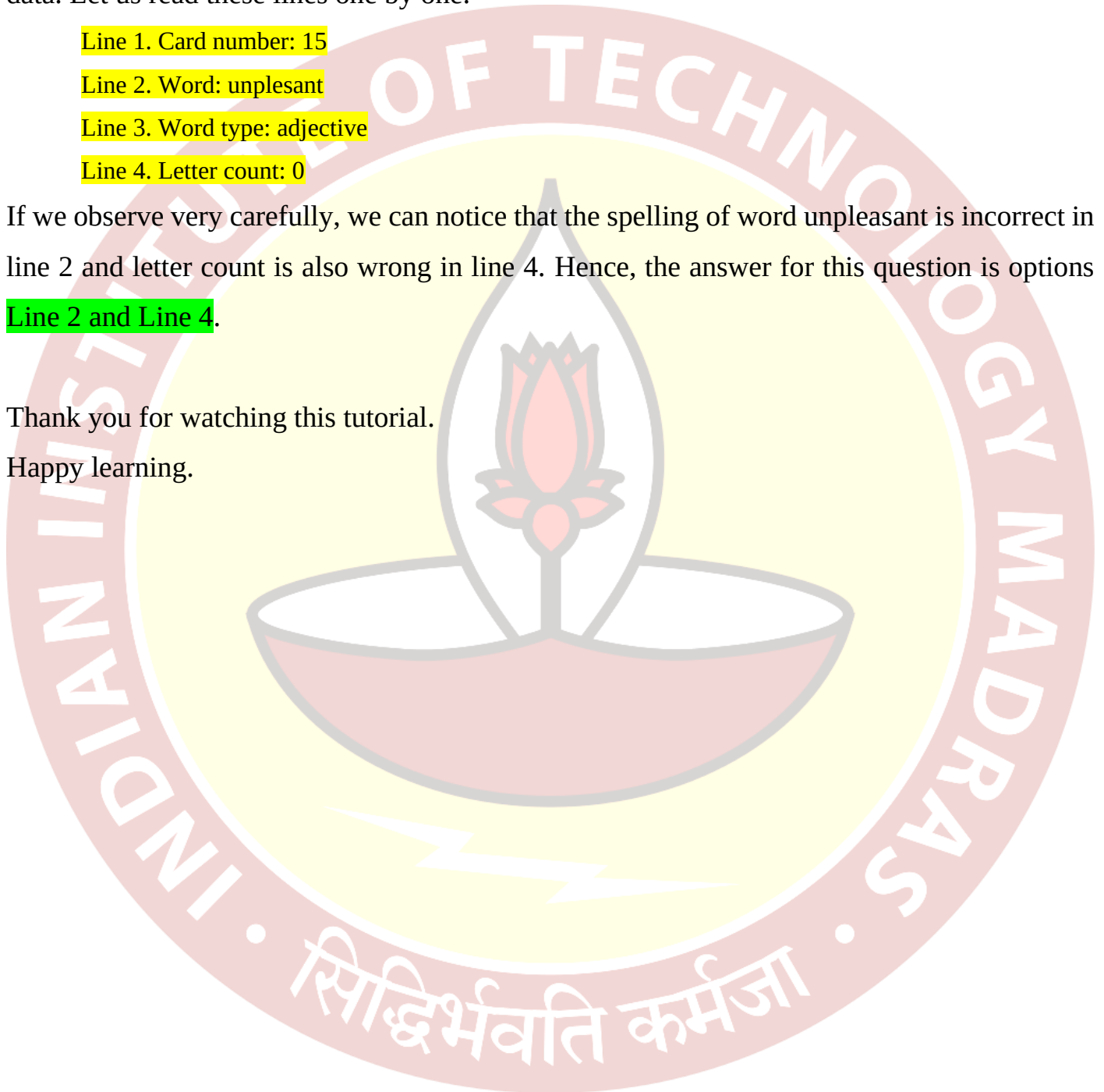
Line 3. Word type: adjective

Line 4. Letter count: 0

If we observe very carefully, we can notice that the spelling of word unplesant is incorrect in line 2 and letter count is also wrong in line 4. Hence, the answer for this question is options Line 2 and Line 4.

Thank you for watching this tutorial.

Happy learning.



Hello computational thinking students. In this tutorial we will see how to solve computational thinking week 1 graded assignment.

Let us start with the instructions given, different font styles or punctuation symbols are used as follows: Italic plus underline represents steps of the procedure, Bold is used for variables, Italic for a field from dataset and Double quotes for a value from dataset or dataset name. These different font styles will help us identify various elements of a procedure.

Question 1: What will be the values of **A**, **B**, **C** and **D** after execution of the following procedure using the “Scores” dataset?

This is a numerical input type question where we have to find the exact values of variables A, B, C and D. Let us read the procedure step by step.

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain five variables **A**, **B**, **C**, **D**, **percentage** and initialize them to 0

Step 3. If Pile 1 is empty then stop the execution

Step 4. Read the top card in Pile 1

Step 5. Calculate percentage: divide *TOTAL marks* by 3 and store the result in **percentage**

In step 5 we are calculating the percentage of a student by dividing his/her total marks by 3 and then the computed result is getting stored in variable percentage.

Step 6. If **percentage** ≥ 85 then increment **A**

Step 7. If **percentage** < 85 and **percentage** ≥ 70 then increment **B**

Step 8. If **percentage** < 70 and **percentage** ≥ 60 then increment **C**

Step 9. If **percentage** < 60 then increment **D**

Step 10. Move the current card to another pile called Pile 2 and repeat from step 3

Here in steps 6 to 9 we are incrementing variables A, B, C and D respectively if the calculated percentage value lies between the specified range. This same process will be carried out for all score cards as per the step 10. Based on this observation we can conclude that the given procedure calculates the student's percentage and allots him/her a grade as per the performance in the exams. Therefore, we can find the values of variables A, B, C and D based on how many students got that particular grade which is represented over here,

Grade A		Grade B		Grade C		Grade D	
Card no.	Percentage	Card no.	Percentage	Card no.	Percentage	Card no.	Percentage
11	93.67	0	70.00	1	66.00	3	57.67
18	87.00	4	80.00	2	62.67	24	58.00
23	92.00	5	82.33	8	63.00	Total (D) = 2	
Total (A) = 3		6	83.33	10	68.00		
		7	84.00	12	62.33		
		9	74.67	19	65.33		
		13	75.67	22	60.33		
		14	81.33	27	69.67		
		15	74.67	Total (C) = 8			
		16	72.00				
		17	70.00				
		20	84.00				
		21	73.67				
		25	84.67				
		26	75.67				
		28	80.00				
		29	73.00				
		Total (B) = 17					

Hence, the answer is 3, 17, 8 and 2 for variables A, B, C and D respectively.

Next question: What will be the value of X after execution of the following procedure using the “Scores” dataset?

In this question we have to find the value of X which is initialized to “None” in step 2. This initialization to none indicates that the variable X is used to store values of string datatype. In step 5, variable X’s value gets updated with city of student named “CK”. But we do not have any student named CK in the “Scores” dataset therefore the value of variable X will not change. Hence, the value of variable X will be None and that is the answer of this question.

Question: What should be the correct sequence of statements to find the total number of nouns, verbs, adjectives and pronouns in the “Paragraph Words” dataset? It is a Multiple Select Question (MSQ).

In this question we have to find the correct sequence of steps from the given set of options. Statements B, H, E, C at the beginning and statement G at the end are in the same order in all the options. Therefore, these statements will generate a template for the procedure.

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain four variables Ncount, Vcount, Acount, Pcount and initialize them to 0

Step 3. If Pile 1 is empty then stop the execution

Step 4. Read the top card in Pile 1

Step 5.

Step 6.

Step 7.

Step 8.

Step 9. Move the current card to another pile called Pile 2 and repeat from step 3

In this procedure template, steps 5 to 8 are missing and statement options remaining are A, D, F and I.

A. If *part of speech* is “Noun” then increment **Ncount**

D. If *part of speech* is “Verb” then increment **Vcount**

F. If *part of speech* is “Adjective” then increment **Account**

I. If *part of speech* is “Pronoun” then increment **Pcount**

Each one of these statements counts the number for each category. Therefore, there is no constrain on the execution sequence of these statements. So, they can be executed in any order given in the options without affecting the purpose of the procedure. Hence, all the given options are correct.

Moving on to next question: What will variable **A** represent after execution of the following procedure on the “Paragraph Words” dataset?

Let us analyse the given procedure, in step 2 two variables A and B are initialized to 0. In step 5 variable B is getting incremented if part of speech is “Verb” which means variable B is counting number of verbs. Conditions in steps 6 and 7 are exactly opposite to each other. As per the combination of steps 6 and 9 we are counting verbs with variable B until we reach to a full stop. In other words, we are counting number of verbs in each sentence. As per step 7 if a sentence ends and verb count in variable B is 1 then we are incrementing variable A. Step 8 resets the value of variable B so that the same variable can be used to count number of verbs in the next sentence. Based on this analysis we can conclude that the variable A gets incremented only if variable B’s value is 1. Hence, the answer of this question is Total number of sentences with only one verb in it.

Question: What will be the value of **X** after execution of the following procedure using the “Shopping Bills” dataset?

In this question we are calculating the total bill amount of “Srivatsan” across all shops. This is the list of all his bills and the total is 1537. Hence the answer of this question is **1537**.

Bill number	Total bill amount
1	567
6	354
26	276
27	340
Total (X) = 1537	

Question: The given procedure finds the category which generated the higher revenue between “electronics” and “stationery” when executed on the “Shopping Bills” dataset. Fill in the blanks from the given choices and complete the procedure.

Now, this is a fill in the blanks type of a question where we have to choose the correct combination of steps from the given options to achieve the procedure’s goal. If we observe the given options then we can easily notice that the missing steps 2 and 8 corresponds to the initialization of variables. Variables **Ecost** and **Scost** are used to calculate the total cost of items sold under categories electronics and stationery respectively. Therefore, the initial value of these variables should be 0 and the correct step 2 should be,

Step 2. Maintain two variables **Ecost**, **Scost** and initialize them to 0

In steps 9 and 10 we are referring to a variable called “higher” which is supposed to store the category which generated the higher revenue between electronics and stationery. As we all know electronics and stationery are of type string therefore, the variable higher should be initialized to “None”.

Step 8. Maintain a variable **higher** and initialize it to “None”

Hence the correct answer for this question is:

Step 2. Maintain two variables **Ecost**, **Scost** and initialize them to 0

Step 8. Maintain a variable **higher** and initialize it to “None”

Next question: The following flowchart is designed to advise a person on what will be the best transportation option to travel from city X to city Y. Fill in the blanks using the given statements and complete the flowchart.

This is also a fill in the blanks type of a question but here instead of a procedure a partially filled flowchart is given. We have to understand the meaning of each component with respect to the context of the flowchart. In order to solve this question, we have to match the given statements with their accurate component labels. Let us see the general sequence of activities while travelling from city X to city Y. We will start from city X, based on the distance we will choose between options like bus and train. If it is a long route then we prefer train and bus for short route. After this, depending on the available budget we will decide between AC and non-AC train and similarly between deluxe and economy bus. Irrespective of the mode and category of transportation we will reach to city Y. Hence, the answers of these flowchart-based questions are **A: 2, B: 6, C: 1, E: 3, G: 4, I: 5**

Moving on to next question: **The following information represents a score card of a new student added in the class. During data entry, the clerk (or operator) might have made a mistake in one or more lines. Identify such lines with respect to the sanity of data. It is a Multiple Select Question (MSQ).**

This is a different type of a question compared to what we have seen so far. Here, a score card of a new student is given to us containing the same fields which we have in “Scores” dataset. We have to analyze these given values with respect to the concept of data sanity.

Line 1. Card number: 30
Line 2. Name: Omkar
Line 3. Gender: M
Line 4. Date of Birth: Pune
Line 5. Town/City: 22 July
Line 6. Mathematics: 89
Line 7. Physics: 72
Line 8. Chemistry: 61
Line 9. TOTAL: 322

Hence, the correct answer for this question is **Line 4, 5 and 9.**

Question: This is also a similar question to previous one where we have to inspect the given data for its sanity. Now, we have a shopping bill which we will go through line by line.

Line 1. Store name: Big Bazaar
Line 2. Customer name: Julia

	Item	Category	Qty	Price	Cost
Line 3.	Cereal	Packed/Food	1	220	220

Line 4.	Milk	Fruits/Food	1	24	24
Line 5.	Cupcakes	Packed/Food	1	25	25
Line 6.	Chocolates	Packed/Food	1	10	100
Line 7.	Shirts	Women/Apparel	1.5	1350	2025

Line 8. Gross total: 2394

Hence, the correct answer for this question is Line 4, 6 and 7.

Moving on to the next question: If we wish to add more data fields to the “Scores”, “Paragraph Words” and “Shopping Bills” datasets, then what is the most appropriate choice of datatype for such data fields?

In this question we are assuming that a new data field is getting added to the existing dataset and we have to find the most appropriate datatype for that field.

Subject marks of English, History and Geography in the “Scores” dataset

As we are adding more than one subject mark hence, the datatype should be “List”

Email address of a student in the “Scores” dataset

Email address contains alphabets, numbers and special characters therefore the datatype should be “String”.

Position of a word in a sentence in the “Paragraph Words” dataset

Here, the position of a word is always a number hence the required datatype is “Integer”.

Serial number of each item bought in the “Shopping Bills” dataset

Serial number is always of type “Integer”.

Thank you for watching this tutorial.

Happy learning.

Hello computational thinking students. In this tutorial we will see how to solve computational thinking week 2 practice assignment solution.

What will be the value of **X** after execution of the following procedure using the “Scores” dataset?

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain a variable **X** and initialize it to 0

Step 3. If Pile 1 is empty then stop the iteration

Step 4. Read the top card in Pile 1

Step 5. If Town/City is “Chennai” and *TOTAL marks* > **X** then store *TOTAL marks* in **X**

Step 6. Move the current card to another pile called Pile 2 and repeat from step 3

From step 5, we can understand that the first filtering condition is about city being Chennai and second filter is to compare total marks with variable X. If the total marks are higher than variable X then we are updating the variable. Based on the combination of these two filtering conditions we can conclude that the variable X will store the highest total marks of a student from Chennai. This is the list of all the students from Chennai:

Card No.	City/Town	TOTAL Marks	Value of X in the current iteration
2	Chennai	188	188
3	Chennai	173	188
5	Chennai	247	247
10	Chennai	204	247
12	Chennai	187	247
17	Chennai	210	247
25	Chennai	254	254
26	Chennai	227	254

Max (X) = 254

Hence, the answer of this question is **254**.

What will the values of **A** and **B** represent at the end of execution of the given procedure on the “Scores” dataset?

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain two variables **A**, **B** and initialize them to 0

Step 3. If Pile 1 is empty then stop the iteration

Step 4. Read the top card in Pile 1

Step 5. If $A < \text{Chemistry marks}$ then store *Chemistry marks* in **A**

Step 6. If *Mathematics marks* $< B$ then store *Mathematics marks* in **B**

Step 7. Move the current card to another pile called Pile 2 and repeat from step 3

In the given procedure's step 5, variable A gets updated only if Chemistry marks are higher than the current value of variable A. Which indicates that the variable A is used to store highest marks in Chemistry. Similarly, we can see that the variable B is getting updated only when Mathematics marks are lower than the variable B. Using step 6, one can assume that the variable B is computing the lowest marks in Mathematics. But that assumption is incorrect because the variable B is initialized to 0 in step 2. And as we have studied that the initial value of variable min should be set to the highest value in that range while finding lowest marks. Therefore, the variable B's value will never change and we will end-up with 0 which does not represent anything specific. Hence, the answer for this question is **None of the above**.

The following procedure finds the Date of Birth of the Physics topper student from the "Scores" dataset. But the programmer may have made mistakes in one or more steps. Identify all such steps (if any). It is a Multiple Select Question (MSQ).

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain a variable **max** and initialize it to 0

Step 3. Maintain a variable **DOB** and initialize it to "None"

Step 4. If Pile 1 is empty then stop the iteration

Step 5. Read the top card in Pile 1

Step 6. If *Physics marks* $> \text{max}$ then store *Physics marks* in **max** and *Date of Birth* in **DOB**

Step 7. Move the current card to another pile called Pile 2 and repeat from step 4

As we can see there is no mistake in the given procedure. Hence, the answer of this question is the last option, **No mistake**.

Answer the following sub-questions based on execution of the following pseudocode using the "Paragraph Words" dataset?

Let us analyse the given pseudocode, here we have initialized variables A and B to 0. The while loop is iterating over all the cards available in Pile 1. First if statement checks the part of speech is Noun or not. If it is Noun then we will execute inner if statement which checks the letter count of the word. If this letter count is greater than 6 then we will increment variable A otherwise we will execute else block and increment variable B.

What will the value of A represent at the end of execution of the above pseudocode?

By combining given two if statements we can conclude that the variable A counts the number of nouns with letter count greater than 6. Hence, the correct answer for this question is **Total number of nouns having letter count greater than 6.**

What will be the value of B after execution of the above pseudocode?

As we have analysed that the variable B is storing the number of nouns with letter count lesser than or equal to 6. This is the list of all such words:

Card No.	Word	Part of Speech	Letter Count
2	Monday	Noun	6
10	eyes.	Noun	4
13	Monday	Noun	6
26	Sunday,	Noun	6
34	Monday	Noun	6
35	mood	Noun	4
37	work	Noun	4
47	school:	Noun	6
56	class	Noun	5
64	cane...	Noun	4

Total (B) = 10

Hence, the answer of this question is **10.**

Thank you for watching this tutorial.

Happy learning.

Hello computational thinking students. In this tutorial we will see how to solve computational thinking week 2 graded assignment.

The following procedure finds the highest marks in Mathematics scored by a “male” student and a “female” student from the “Scores” dataset. But the programmer may have made mistakes in one or more steps. Identify all such steps (if any). It is a Multiple Select Question (MSQ).

Let us read the given procedure step by step.

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain two variables **Mmax**, **Fmax** and initialize them to 0

Step 3. If Pile 1 is empty then stop the iteration

Step 4. Read the top card in Pile 1

Step 5. If Gender is “M” and **Mmax** > Mathematics marks then store Mathematics marks in **Mmax**

Step 5 does not look right, there is some mistake in it but let us continue because step 6 is also similar to this one.

Step 6. If Gender is “F” and **Fmax** > Mathematics marks then store Mathematics marks in **Fmax**

Step 7. Move the current card to another pile called Pile 2 and repeat from step 3

As I said earlier, steps 5 and 6 are similar and there is an identical mistake in both those steps. Here, we are trying to find gender wise toppers of Mathematics and as we know, while finding max the conditions should be, **Mmax** < Mathematics marks and **Fmax** < Mathematics marks in steps 5 and 6 respectively. Hence, the correct answer of this question is **step 5 and 6**.

What will variables **A** and **B** represent after execution of the following procedure on the “Paragraph Words” dataset?

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain two variables **A**, **B** and initialize them to 0

Step 3. If Pile 1 is empty then stop the iteration

Step 4. Read the top card in Pile 1

Step 5. If Letter count > **A** then store **A** in **B** and Letter count in **A**

Step 6. If Letter count < **A** and Letter count > **B** then store Letter count in **B**

Step 7. Move the current card to another pile called Pile 2 and repeat from step 3

In this question, steps 5 and 6 are responsible for local operations in the given iteration. Step 5 checks the letter count of the current card and if letter count is greater than A then variable

B's value will get replaced by A. At the same time, step 5 will also store the letter count in variable A which will be the max letter count in the dataset. This implies that the value of variable A is copied to variable B before updating the value of variable A. In step 6, letter count is getting stored in variable B if the current letter count is smaller than variable A but higher than variable B. Based on these steps, we can conclude that the variable B is storing the second max letter count in the dataset. Hence, the correct answer of this question is **A = Largest word length and B = Second largest word length**.

What will the value of A represent at the end of execution of the given procedure on the "Paragraph Words" dataset?

In this question's step 5, we are adding letter count to variable B without checking any condition. Then, the step 6 checks whether the word ends with a full stop or not which means it checks for the end of the sentence. If that is not the case then we are executing step 9 and repeating the process of accumulation of letter count in variable B. On the other hand, in step 7 if the word ends with a full stop and that is the end of the sentence then we are comparing variables B and A. If variable B is greater than A then we are storing variable B's value in variable A and resetting variable B's value to 0 for the next iteration. Based on all this analysis we can conclude that the purpose of this procedure is to find longest sentence using letter count. Hence, the correct answer of this question is **length of the longest sentence based on the number of characters**.

Let us define the most popular shop to be the one that generates the maximum total revenue. The given procedure finds the revenue of the most popular shop in the "Shopping Bills" dataset. Fill in the blanks from the given choices and complete the procedure.

In this question we are finding the max among total revenues generated by each shop from shopping bills dataset. Missing steps 5, 6 and 7 represent the calculation of the same. By observing the given options, we can identify that the step 5 is accumulating the total bill amounts of SV Stores. Also, it is comparing the computed total revenue with variable max and if it is greater than max then storing the same in variable max. Similar computation and comparison is happening for Big Bazaar and Sun General in steps 6 and 7 respectively. Hence the correct answer of this question is,

Step 5. If the *shop name* is “SV Stores” then add *total bill amount* to **SVtotal**. And if **SVtotal** > **max** then store **SVtotal** in **max**

Step 6. If the *shop name* is “Big Bazaar” then add *total bill amount* to **BBtotal**. And if **BBtotal** > **max** then store **BBtotal** in **max**

Step 7. If the *shop name* is “Sun General” then add *total bill amount* to **SGtotal**. And if **SGtotal** > **max** then store **SGtotal** in **max**

The following procedure finds the highest amount of bill generated at “Sun General” from the “Shopping Bills” dataset. But the programmer may have made mistakes in one or more steps. Identify all such steps (if any). It is a Multiple Select Question (MSQ).

Step 1. Arrange all cards in a single pile called Pile 1

Step 2. Maintain a variable **max** and initialize it to 1

Variable initialization in this step may look different as compared to the previous initializations of variable **max** but it is correct. Because we have learned that the variable **max** should be initialized to the lowest value in the range. Here, we are computing the **max** of total bill amount at Sun General and the lowest possible value for any shopping bill is 1 not 0.

Step 3. If Pile 1 is empty then stop the iteration

Step 4. Read the top card in Pile 1

Step 5. If the *shop name* is “Sun General” and **max** > *total bill amount* then store *total bill amount* in **max**

This step is wrong because of the incorrect comparison operation between variable **max** and total bill amount. It should have been, Step 5. If the *shop name* is “Sun General” and **max** > *total bill amount* then store *total bill amount* in **max**.

Step 6. Move the current card to another pile called Pile 2 and repeat from step 3

Hence, the correct answer of this question is only **step 5**.

What will be the value of **X** after execution of the following procedure using the “Shopping Bills” dataset?

Here in step 2, variables **A** and **B** are initialized to 0 and 1000 respectively. As per the steps 5 and 6, we are doing some calculations using the current card only if the item Bananas

is there on it. In steps 7 and 8, we are finding the highest and lowest unit price of the item Bananas and storing these values in variables A and B respectively. This is the list of all such shopping bills.

Card No.	Item	Price
1	Bananas	8
3	Bananas	8
11	Bananas	8
12	Bananas	5
20	Bananas	8
22	Bananas	5
23	Bananas	8
26	Bananas	8
29	Bananas	8

Max (A) = 8 Min (B) = 5

In step 10, we are adding these two variables A, B and storing the result in variable X.

Hence, the correct answer of this question is 13.

What will be the value of X after execution of the following procedure using the “Shopping Bills” dataset?

Here in step 3, variable X is initialized to None which indicates that this variable is used to store value of string datatype. If we observe steps 6 to 9 then we can notice that the computation in these steps is identical except it is for different customers. Variable Y stores the max of amount spent by customers Akshaya, Advait, Abhinav and Ahmed across all shops. These steps also store the name of respective customer in the variable X. Here is the full list of their spending.

Akshaya		Advait		Abhinav		Ahmed	
Card No.	Amount	Card No.	Amount	Card No.	Amount	Card No.	Amount
3	1341	4	123	11	893	18	603
5	4174	8	3132	13	3060	20	622
21	128	16	187	15	229	25	1364
						29	106

Total (A) = 5643

Total (B) = 3442

Total (C) = 4182

Total (D) = 2695

Hence, the correct answer of this question is **Akshaya**.

The following flowchart represents the sequence of steps through which a patient goes after entering a clinic. Fill in the blanks using the given statements and complete the flowchart.

We have to understand the meaning of each component with respect to the context of the flowchart. In order to solve this question, we have to match the given statements with their accurate component labels. Let us see the general sequence of activities a patient goes through after entering a clinic. First patient will arrive at the clinic then the clinic's admin department will check if the patient is registered patient or not. If it is a new patient then he/she will have to do the registration process then only a nurse will be allotted. After that the allotted nurse will record the health condition and assigned doctor will check the patient. If the doctor thinks that some medication is required then the prescription will be given to the patient. Another decision point comes with respect to the follow up. If follow up is required then the clinic will schedule an appointment and then, the patient will leave the clinic. Hence, the answers of these flowchart-based questions are **A: 3, B: 7, C: 4, G: 1, H: 5, J: 2, K: 6**.

What will be the values of **A**, **B** and **C** after execution of the following pseudocode using the "Scores" dataset?

Let us analyse the given pseudocode, here we have initialized variables A, B and C to 0. The while loop is iterating over all the cards available in Pile 1. First if statement compares Mathematics marks with Physics marks and if they are higher then variable A is getting incremented. Then in second if we are checking if Mathematics marks are lesser than Physics marks or not and in third if we are comparing Mathematics marks with Physics marks for equality. But as we can see that the second and third if are inside the first if which means they will be executed only if outer if statement is true. Therefore, second and third if will never compute to be true and variables B and C will not be incremented. This is the list of all such students who scored higher marks in Mathematics than Physics.

Card No.	Mathematics	Physics	Card No.	Mathematics	Physics
0	68	64	16	87	86
1	62	45	20	93	68
2	57	54	21	78	69
4	87	64	23	97	91

8	74	64	25	87	75
11	97	92	26	74	71
14	89	62	27	81	76
15	76	58	29	72	66

Hence, the answers of these pseudocode-based questions are **A = 16, B = 0 and C = 0.**

What will be the values of A, B and C after execution of the following pseudocode using the “Scores” dataset?

Let us analyse the given pseudocode, here we have initialized variables A, B and C to 0. The while loop is iterating over all the cards available in Pile 1. First if statement checks the city is Chennai or not. If it is Chennai then the second if statement will be executed but if not then we will directly jump to the else block and increment variable C. Because that else is associated with this first if statement and following is the list of all such students.

Card No.	City	Card No.	City	Card No.	City
0	Erode	13	Madurai	22	Nagercoil
1	Salem	14	Trichy	23	Bengaluru
4	Madurai	15	Theni	24	Madurai
6	Ambur	16	Trichy	27	Madurai
7	Vellore	18	Erode	28	Madurai
8	Bengaluru	19	Trichy	29	Vellore
9	Bengaluru	20	Vellore		
11	Bengaluru	21	Bengaluru	Total (C) = 22	

If the student is from Chennai then in second if we are looking for all the students born before 1st May and if that is the case then we are comparing total marks with variable A. If the total marks are higher than the current value of variable A then we will update variable A with those total marks. Therefore, variable A will store the highest total marks of the student from Chennai born before 1st May. The following is the list of all such students.

Card No.	Date of Birth	City	Total marks
2	4 Jan	Chennai	188
5	8 Feb	Chennai	247
10	12 Jan	Chennai	204
12	14 Jan	Chennai	187

Max (A) = 247

Next if is identical to the previous if with variable A except this one is outside the condition check for date of birth. Therefore, variable B will store the highest total marks of the student from Chennai. The following is the list of all such students.

Card No.	Date of Birth	City	Total marks
2	4 Jan	Chennai	188
3	5 May	Chennai	173
5	8 Feb	Chennai	247
10	12 Jan	Chennai	204
12	14 Jan	Chennai	187
17	14 Dec	Chennai	210
25	16 May	Chennai	254
26	22 Jul	Chennai	227

Max (B) = 254

Hence, the answers of these pseudocode-based questions are **A = 247, B = 254 and C = 22.**

Week - 3

Practice Assignment Solution

1. The following pseudocode is executed using the “Shopping bills” dataset. At the end of the execution, the variable **CountBBBA** captures the number of bills from Big Bazaar with total bill amount less than the average total bill amount. Choose the correct code block to complete the pseudocode.

```
SumT = 0
Count = 0
while (Pile 1 has more cards) {
    Read the top card X in Pile 1
    SumT = SumT + X.TotalBillAmount
    Count = Count + 1
    Move X to Pile 2
}
AvgT = SumT / Count
CountBBBA = 0
while (Pile 2 has more cards) {
    Read the top card X in Pile 2
    *****
    *      Fill the code      *
    *****
    Move X to Pile 1
}
```

- a. if (**X.TotalBillAmount** < **AvgT**) {
 CountBBBA = **CountBBBA** + 1
 }
- b. if (**X.ShopName** == “Big Bazaar” or **X.TotalBillAmount** < **AvgT**) {
 CountBBBA = **CountBBBA** + 1
 }
- c. if (**X.ShopName** == “Big Bazaar” and **X.TotalBillAmount** < **AvgT**) {
 Count = **Count** + 1
 }
- d. if (**X.ShopName** == “Big Bazaar” and **X.TotalBillAmount** < **AvgT**) {
 CountBBBA = **CountBBBA** + 1
 }
-

Solution: The sum of the total bill amounts and the count of the bills are computed in the variables **SumT** and **Count** respectively. Then, the average total bill amount is computed and stored in **AvgT**. Our aim is to count bills from Big Bazaar with total bill amount less than average total bill amount. Also, we want to store the count in **CountBBBA**. First, we filter the bills from Big Bazaar using the following filter:

```
if (X.ShopName == "Big Bazaar") {  
    CountBBBA = CountBBBA + 1  
}
```

Second, we filter the bills with total bill amount less than average total bill amount using the following filter:

```
if (X.TotalBillAmount < AvgT) {  
    CountBBBA = CountBBBA + 1  
}
```

Since we have to count the bills which satisfy both filters, we need to apply both filters using **and** operator. Therefore, the following code-block can complete the pseudocode.

```
if (X.ShopName == "Big Bazaar" and X.TotalBillAmount < AvgT) {  
    CountBBBA = CountBBBA + 1  
}
```

2. The following pseudocode is executed using the “Scores” table to compute the number of students in each grade where grades are assigned based on Physics marks. Choose the correct code block to complete the pseudocode.

```

MinP = 100, MaxP = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    if (X.Physics > MaxP) {
        MaxP = X.Physics
    }
    if (X.Physics < MinP) {
        MinP = X.Physics
    }
    Move X to Table 2
}
Interval = (MaxP - MinP) / 3
Mid2 = MinP + Interval
Mid1 = Mid2 + Interval
CountA = 0, CountB = 0, CountC = 0
while (Table 2 has more rows) {
    Read the first row X in Table 2
    if (X.Physics ≥ Mid1) {
        CountA = CountA + 1
    }
    *****
    *      Fill the code      *
    *****
    if (X.Physics < Mid2) {
        CountC = CountC + 1
    }
}

```

- a. if (**X.Physics** < **Mid1** and **X.Physics** ≥ **Mid2**) {
 CountB = **CountB** + 1
 }
- b. if (**X.Physics** ≥ **Mid2**) {
 CountB = **CountB** + 1
 }
- c. if (**X.Physics** < **Mid1** and **X.Physics** ≥ **Mid2**) {
 CountB = 1
 }
- d. if (**X.Physics** ≤ **Mid1** and **X.Physics** ≥ **Mid2**) {
 CountB = **CountB** + 1
 }

Solution: The values captured by the variables used in this Pseudocode are listed as follows:

- MinP** - The minimum Physics marks
- MaxP** - The maximum Physics marks
- Interval** - One third of the difference between the minimum and maximum Physics marks
- Mid2** - The offset for the Grade-3
- Mid1** - The offset for the Grade-2

The variables **CountA**, **CountB** and **CountC** capture the number of students in Grade-1, Grade-2 and Grade-3 respectively. The filter for the Grade-1 is $X.Physics \geq Mid1$ and the filter for the Grade-3 is $X.Physics < Mid2$. Therefore, the Grade-2 is awarded to a student if his/her Physics marks is at least **Mid2** and less than **Mid1**. Note that, the variable **CountB** captures the number of students in the Grade-2. Then, the following code-block completes the above pseudocode.

```
if ( $X.Physics < Mid1$  and  $X.Physics \geq Mid2$ ) {  
    CountB = CountB + 1  
}
```

3. The following pseudocode is executed using the “Scores” dataset. What will the values of **A** and **B** represent at the end of the execution?

```

AM = DoSomething(“M”)
AF = DoSomething(“F”)
A = 0, B = 0
while (Pile 2 has more cards) {
    Read the top card X in Pile 2
    if (X.Gender == “M” and X.Total < AF) {
        A = A + 1
    }
    if (X.Gender == “F” and X.Total < AM) {
        B = B + 1
    }
    Move X to Pile 1
}

Procedure DoSomething (Gen)
    Arrange all cards on Pile 1
    Sum = 0, Count = 0
    while (Pile 1 has more cards) {
        Read the top card X from Pile 1
        if (X.Gender == Gen) {
            Sum = Sum + X.Total
            Count = Count + 1
        }
        Move X to Pile 2
    }
    C = Sum / Count
    return (C)
End DoSomething

```

- A** = Number of male students with total marks less than the average total marks
B = Number of female students with total marks less than the average total marks
- A** = Number of male students with total marks less than the average total marks of male students
B = Number of female students with total marks less than the average total marks of female students
- A** = Number of female students with total marks less than the average total marks
B = Number of male students with total marks less than the average total marks
- A** = Number of male students with total marks less than the average total marks of female students
B = Number of female students with total marks less than the average total marks of male students

Solution: There are two portions in this pseudocode. The first one is main part of the pseudocode and the second one is the procedure **DoSomething**. First we look at the procedure **DoSomething**. It is called with a parameter named **Gen**. In the main pseudocode, the procedure **DoSomething** is called with the values “M” and “F”. Therefore, the parameter passed to the procedure is a value that represents Gender of the students. The condition **X.Gender == Gen** in the procedure filters the students based on the value passed to the variable **Gen**. The sum of the total marks and the count of the filtered students are computed and stored in **Sum** and **Count** respectively. Then, the average total marks is computed and stored in **C**. Therefore, the procedure **DoSomething** returns the average total marks of the students filtered by the gender which is given as an input parameter to the procedure.

In the main pseudocode, the variables **AM** and **AF** hold the average total marks of the male students and the female students. First, we consider the variable **A**. It is getting updated when the condition **X.Gender == “M” and X.Total < AF** is **True**. The above condition has two parts. The first part checks whether the gender is male and the second part checks whether total marks less than the average total marks of female students. Therefore,

A = Number of male students with total marks less than the average total marks of female students

Now we consider the variable **B**. It is getting updated when the condition **X.Gender == “F” and X.Total < AM** is **True**. The above condition has two parts. The first part checks whether the gender is female and the second part checks whether total marks less than the average total marks of male students. Therefore,

B = Number of female students with total marks less than the average total marks of male students

4. The following pseudocode is executed using the “Shopping bills” dataset. What will be the values of **Test1** and **Test2** at the end of the execution?

```
Let AvgT be the average total bill amount across all shops
CountSV = 0, CountBB = 0, CountSG = 0
while (Pile 1 has more cards) {
    Read the top card X from Pile 1
    if (X.TotalBillAmount < AvgT) {
        if (X.ShopName == “SV Stores”) {
            CountSV = CountSV + 1
        }
        if (X.ShopName == “Big Bazaar”) {
            CountBB = CountBB + 1
        }
        if (X.ShopName == “Sun Generals”) {
            CountSG = CountSG + 1
        }
    }
    Move X to Pile 2
}
Test1 = 0, Test2 = None
if (CountSV == 0 or CountBB == 0 or CountSG == 0) {
    Test1 = Test1 + 1
}
if (CountSV < CountSG) {
    if (CountBB < CountSV) {
        Test2 = “Big Bazaar”
    }
    else {
        Test2 = “SV Stores”
    }
}
else {
    if (CountBB < CountSG) {
        Test2 = “Big Bazaar”
    }
    else {
        Test2 = “Sun Generals”
    }
}
```

Answer: **Test1** = 0, **Test2** = “Big Bazaar” (Numerical and text input)

Solution: We execute the above pseudocode using the “Shopping bills” dataset. First we have to compute the average total bill amount. In the dataset, the average total bill amount is **813.77**. Tabulate the contents of the data set and execute it on the pseudocode.

#	TBA	Shop	C1	C2	C3	C4	CountSV	CountBB	CountSG
1	567	SV	True	True			1		
2	1525	BB			True				
3	1341	SV		True					
4	123	SV	True	True			2		
5	4174	BB			True				
6	354	SG	True			True			1
7	96	SV	True	True			3		
8	3132	BB			True				
9	595	BB	True		True			1	
10	378	SG	True			True			2
11	893	SV		True					
12	186	SG	True			True			3
13	3060	BB			True				
14	656	SG	True			True			4
15	229	SG	True			True			5
16	187	SG	True			True			6
17	279	SV	True	True			4		
18	603	SV	True	True			5		
19	592	SV	True	True			6		
20	622	SV	True	True			7		
21	128	SV	True	True			8		
22	315	SG	True			True			7
23	888	SV		True					
24	92	SV	True	True			9		
25	1364	SG				True			
26	276	SV	True	True			10		
27	340	SG	True			True			8
28	514	SV	True	True			11		
29	106	SV	True	True			12		
30	798	BB	True		True			2	

In the above table, we refer the column names as follows:

- # - Sequence number
- TBA - Total bill amount
- Shop - Shop name
- C1 - $X.TotalBillAmount < AvgT$
- C2 - $X.ShopName == \text{“SV Stores”}$
- C3 - $X.ShopName == \text{“Big Bazaar”}$
- C4 - $X.ShopName == \text{“Sun Generals”}$

The empty cells in the table refer either **False** in condition statement or no update to the variables. At the end of the iteration, the values of **CountSV**, **CountBB** and **CountSG** are 12, 2 and 8 respectively. Since none of this variables are having value 0, **Test1** is not updated

by the if condition next to the iteration. Therefore, **Test1** = 0.

Next we move to the final if condition. Since **CountSV** < **CountSG** ($12 < 8$) is **False**, we execute the else code-block. Since **CountBB** < **Count** ($2 < 8$) is **True**, we execute the if code-block. That is, **Test2** = "Big Bazaar". Therefore, **Test2** = "Big Bazaar".

5. The following pseudocode is executed using the “Words” table. Let **A** and **B** be the variables that hold the values of letter counts of shortest and longest word, respectively. What will the value of **Count** represent at the end of the execution?

```
Count = 0
while (Table 1 has more cards) {
    Read the first row X from Table 1
    Move X to T2
    if (X.Word ends with a full stop) {
        Count = Count + DoSomething (T2, A, B)
        Clear all rows in T2
    }
}

Procedure DoSomething (Table 2, X, Y)
    C = 0, D = 0
    while (Table 2 has more rows) {
        Read the first row Z from Table 2
        if (Z.LetterCount ≤ X) {
            C = 1
        }
        if (Z.LetterCount ≥ Y) {
            D = 1
        }
        Move X to Table 3
    }
    if (C + D == 2) {
        return (1)
    }
    else {
        return (0)
    }
End DoSomething
```

- a. Number of sentences with two shortest word or two longest word
- b. Number of sentences with two shortest word and two longest word
- c. Number of sentences with at least one shortest word or at least one longest word
- d. Number of sentences with at least one shortest word and at least one longest word
- e. None of the above

Solution: There are two portions in this pseudocode. The first one is main part of the pseudocode and the second one is the procedure **DoSomething**. First we look at the procedure **DoSomething**. It is called with three parameter named Table 2, **X** and **Y**. In the main pseudocode, the procedure **DoSomething** is called with the values T2, **A** and **B** where T2 is a table of words and, **A** and **B** are integers that hold the values of letter count of shortest word and longest word.

The condition **Z.LetterCount** $\leq$ **X** in the procedure checks whether the word is shortest word. If **True**, then **C** is set to 1. Then, the condition **Z.LetterCount** $\geq$ **Y** in the procedure checks whether the word is longest word. If **True**, then **D** is set to 1. Finally, the if-condition **C + D == 2** is **True** if and only if both **C** and **D** were set to 1. Therefore, the procedure **DoSomething** returns 1 when the input table contains both longest and shortest words. Otherwise, it returns 0.

In the main pseudocode, we iterate the Table 1 and moving the rows to Table T2. The if-condition **X.Word ends with a full stop** is **True**, then we call the procedure **DoSomething** with parameters T2, **A** and **B**. That is, the above condition checks that the word is the end of a sentence or not. If **True**, then the procedure **DoSomething** returns 1 if the sentence has both longest and shortest words, and returns 0 otherwise. When we find the end of a sentence, we are updating the variable **Count** based on the value returned by the procedure **DoSomething**. Therefore, the variable **Count** captures the number of sentences with at least one shortest word and at least one longest word.

COMPUTATIONAL THINKING
Week - 3

Graded Assignment Solution

1. In the “Paragraph words” dataset, the procedure **CountLongNouns** counts the number of nouns that are longer than the average word length. Assume that the variable **Avg** holds the value of average word length. Choose the correct choice to complete the procedure. [3 Marks]

```
Procedure CountLongNouns()
  Count = 0
  while (Pile 1 has more rows) {
    Read the top card X from Pile 1
    *****
    *   Fill the code   *
    *****
  }
  Move X to Pile 2
}
end CountLongNouns
```

- a. if (**X.LetterCount** > **Avg**) {
 Count = **Count** + 1
}
- b. if (**X.PartOfSpeech** == “Noun”) {
 Count = **Count** + 1
}
- c. if (**X.LetterCount** > **Avg** or **X.PartOfSpeech** == “Noun”) {
 Count = **Count** + 1
}
- d. if (**X.LetterCount** > **Avg** and **X.PartOfSpeech** == “Noun”) {
 Count = **Count** + 1
}
- e. None of the above

Solution: The purpose of the procedure **CountLongNouns** is as follows: To count nouns with letter count more than average letter count. The average letter count of the data set is stored in the variable **Avg**. We are iterating the cards in Pile 1 using the iterator **X**. In each step of the iteration, we have to check whether the word is a noun as well as the letter count is more than the average letter count. The variable **Count** is incremented when the both conditions are satisfied. So, we have to use **and** operator to merge the conditions. Therefore, the following code-block complete the procedure **CountLongNouns**.

```
if (X.LetterCount > Avg and X.PartOfSpeech == "Noun") {  
    Count = Count + 1  
}
```



2. The following pseudocode is executed using the “Scores” dataset. What will be the value of the variable **Z** at the end of the execution? [Note: Consider only two decimal places in all division operations.] [4 Marks]

```
SumT = 0, SumM = 0, SumP = 0, SumC = 0
Count = 0
while (Pile 1 has more cards) {
    Read the top card X from Pile 1
    SumT = SumT + X.Total
    SumM = SumM + X.Mathematics
    SumP = SumP + X.Physics
    SumC = SumC + X.Chemistry
    Count = Count + 1
    Move X to Pile 2
}
A = SumT / Count
B = SumM / Count
C = SumP / Count
D = SumC / Count
Z = 0
if ((B + C + D) - A ≤ -1) {
    Z = -1
}
if ((B + C + D) - A ≥ 1) {
    Z = 1
}
```

- a. -1
- b. 0
- c. +1
- d. None of the above

Solution: The values captured by the variables in this pseudocode are listed as follows:

- SumT** - The sum of the total marks (**SumT** = 6674)
- SumM** - The sum of the Mathematics marks (**SumM** = 2171)
- SumP** - The sum of the Physics marks (**SumP** = 2189)
- SumC** - The sum of the Chemistry marks (**SumC** = 2314)
- Count** - Number of cards in the data set (**Count** = 30)

The above values are computed during the iteration. After the iteration, the variables **A**, **B**, **C** and **D** are computed as follows.

- **A** = **SumT** / **Count** = 6674 / 30 $\simeq$ 222.47
- **B** = **SumM** / **Count** = 2171 / 30 $\simeq$ 72.37
- **C** = **SumP** / **Count** = 2189 / 30 $\simeq$ 72.97
- **D** = **SumC** / **Count** = 2314 / 30 $\simeq$ 73.13

The value of the expression **(B + C + D) - A** is $72.37 + 72.97 + 73.13 = 222.47$. Therefore, both if-conditions will be failed and the value of **Z** remains 0. Note that, we consider only two decimal places in all division operation. For nay rounding value, the difference between the values to be rounded is at most 0.01. Therefore, a difference of at most 0.04 is possible on overall evaluation of the expression and it is less than the margin value given in the range. Thus, the answer is **Z = 0**.

3. The following pseudocode is executed using the “Scores” table. At the end of the execution, the variable **CountBA** captures the number of students with total marks less than average total marks and below average in exactly one subject. Assume that the variable **AvgT** holds the value of average total marks. Similarly, the variables **AvgP**, **AvgC** and **AvgM** hold the value of average marks of Physics, Chemistry and Mathematics respectively. Choose the correct choice to complete the pseudocode. [3 Marks]

```

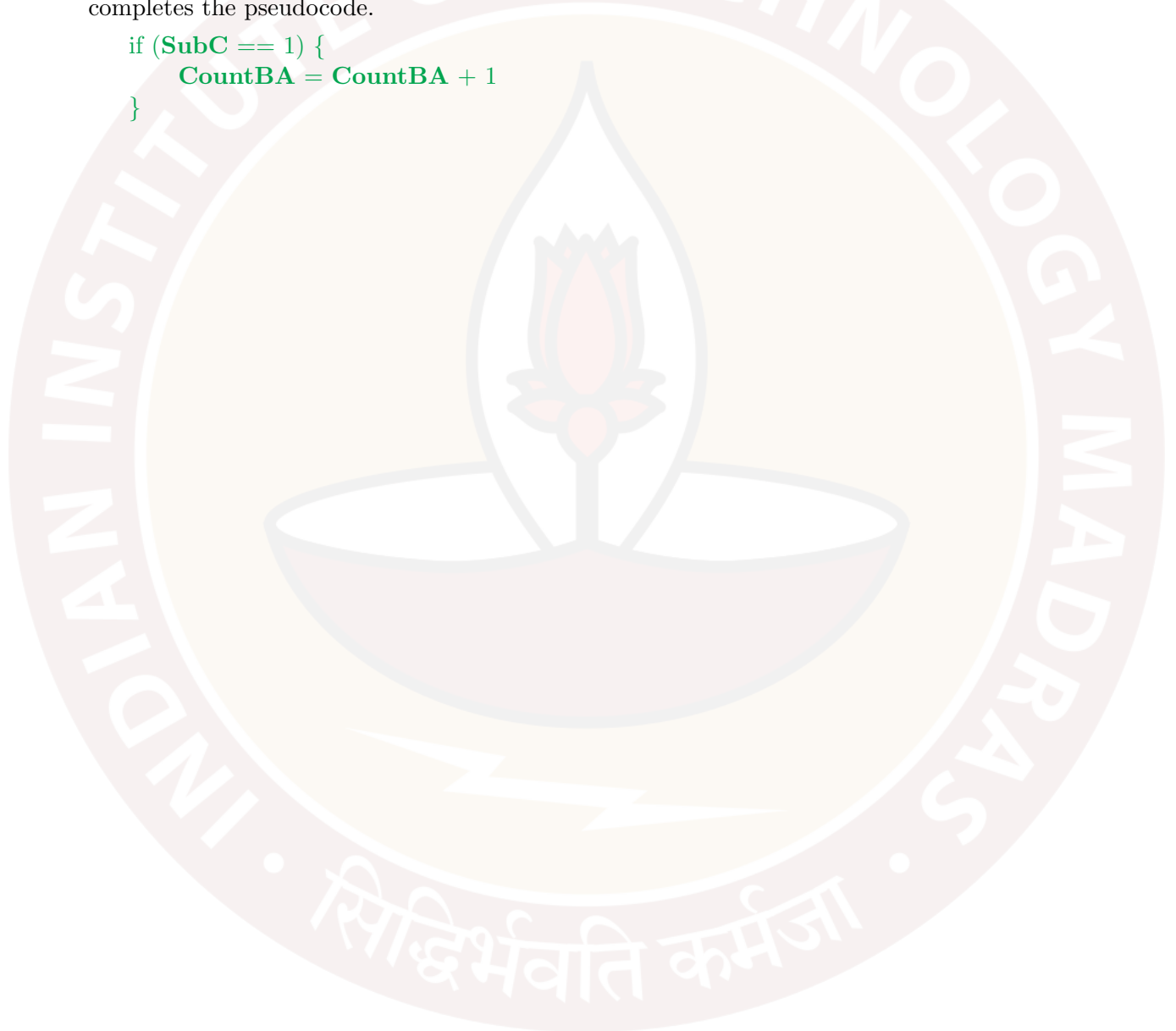
CountBA = 0
while (Table 1 has more rows) {
    Read the first row X from Table 1
    SubC = 0
    if (X.Total < AvgT) {
        if (X.Mathematics < AvgM) {
            SubC = SubC + 1
        }
        if (X.Physics < AvgP) {
            SubC = SubC + 1
        }
        if (X.Chemistry < AvgC) {
            SubC = SubC + 1
        }
        *****
        *      Fill the code      *
        *****
    }
    Move X to Table 2
}

```

- a. if (**SubC** ≤ 1) {
 CountBA = **CountBA** + 1
 }
- b. if (**SubC** ≤ 1) {
 CountBA = 1
 }
- c. if (**SubC** == 1) {
 CountBA = 1
 }
- d. if (**SubC** == 1) {
 CountBA = **CountBA** + 1
 }

Solution: Assume that the variables **AvgT**, **AvgM**, **AvgP** and **AvgC** hold the values of the average total marks, the average marks of Mathematics, Physics and Chemistry, respectively. The cards in the Pile 1 are iterated using the iterator **X**. Our aim is to count the students with total marks less than average total marks and below average in exactly one subject. The variable **SubC** captures the number of subject in which the student is scored below average. The first filter is to check the total marks is less than the average total marks. If the above filter is applied, then we count the number of subjects in which the marks are below average. After computation of the variable **SubC**, we have to check that the **SubC** is either one or not. If the value is one, then increment the variable **CountBA**. Therefore, the following code-block completes the pseudocode.

```
if (SubC == 1) {  
    CountBA = CountBA + 1  
}
```



4. The following pseudocode is executed using the “Shopping bills” dataset. What will the values of the variables **A** and **B** represent at the end of the execution? [4 Marks]

```

SumSV = 0, SumBB = 0
CountSV = 0, CountBB = 0
while (Pile 1 has more cards) {
    Read the top card X from Pile 1
    if (X.ShopName == “SV Stores”) {
        SumSV = SumSV + X.TotalBillAmount
        CountSV = CountSV + 1
    }
    if (X.ShopName == “Big Bazaar”) {
        SumBB = SumBB + X.TotalBillAmount
        CountBB = CountBB + 1
    }
    Move X to Pile 2
}
MSV = SumSV / CountSV
MBB = SumBB / CountBB
A = 0, B = 0
while (Pile 2 has more cards) {
    Read the top card X from Pile 2
    if (X.ShopName == “SV Stores” and X.TotalBillAmount < MSV) {
        A = A + 1
    }
    if (X.ShopName == “Big Bazaar” and X.TotalBillAmount > MBB) {
        B = B + 1
    }
    Move X to Pile 1
}

```

- A** = Number of bills with total bill amount less than the average total bill amount
B = Number of bills with total bill amount greater than the average total bill amount
- A** = Number of bills from “Big Bazaar” with total bill amount greater than the average total bill amount of “Big Bazaar”
B = Number of bills from “SV Stores” with total bill amount less than the average total bill amount of “SV Stores”
- A** = Number of bills from “SV Stores” with total bill amount less than the average total bill amount of “SV Stores”
B = Number of bills from “Big Bazaar” with total bill amount greater than the average total bill amount of “Big Bazaar”
- A** = Number of bills with total bill amount greater than the average total bill amount
B = Number of bills with total bill amount less than the average total bill amount

Solution: The pseudocode consists of two non-nested iterations. There are two if-conditions in the first iteration. The first condition checks whether the bill belongs to “SV Stores”. If the condition is **True**, then the variable **SumSV** is added with the total bill amount of the bill and the variable **CountSV** is incremented by one. Similarly, the second condition checks whether the bill belongs to “Big Bazaar”, and updates **SumBB** and **CountBB**.

At the end of the first iteration, the values captured by the variables are listed below:

- SumSV** - The sum of the total bill amount of the bills from “SV Stores”
- CountSV** - Number of bills from “SV Stores”
- SumBB** - The sum of the total bill amount of the bills from “Big Bazaar”
- CountBB** - Number of bills from “Big Bazaar”
- MSV** - The average total bill amount of the bills from “SV Stores”
- MBB** - The average total bill amount of the bills from “Big Bazaar”

There are two if-conditions in the first iteration. The first condition checks whether the bill belongs to “SV Stores” and the total bill amount is less than the average total bill amount of the bills from “SV Stores”. If the condition is **True**, then the variable **A** is incremented by one. Similarly, the second condition checks whether the bill belongs to “Big Bazaar” and the total bill amount is more than the average total bill amount of the bills from “SV Stores”, and updates **B**. Therefore the variables **A** and **B** capture the following at the end of the execution.

A = Number of bills from “SV Stores” with total bill amount less than the average total bill amount of “SV Stores”

B = Number of bills from “Big Bazaar” with total bill amount greater than the average total bill amount of “Big Bazaar”

5. The following pseudocode is executed using the “Scores” dataset to grade the students by total marks. Also, the pseudocode counts number of male students in each grade. But the pseudocode may have mistakes in one or more lines. Identify all such lines (if any). It is a Multiple Select Question (MSQ). [4 Marks]

```

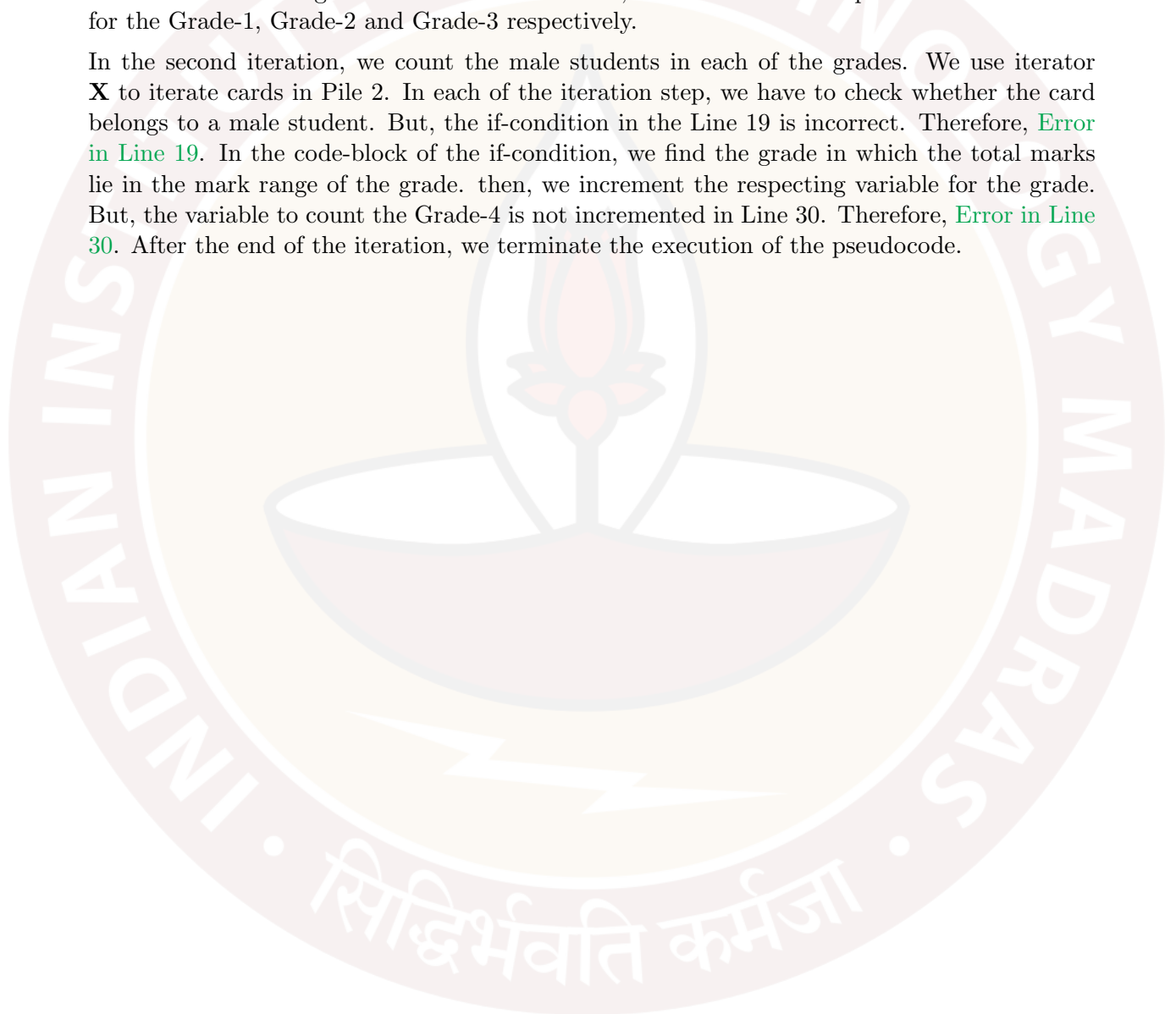
1  MinT = -1, MaxT = 301
2  while (Pile 1 has more cards) {
3      Read the top card X from Pile 1
4      if (MinT > X.Total) {
5          MinT = X.Total
6      }
7      if (MaxT < X.Total) {
8          MaxT = X.Total
9      }
10     Move X to Pile 2
11 }
12 Interval = (MaxT - MinT) / 4
13 Mid3 = MinT + Interval
14 Mid2 = Mid3 + Interval
15 Mid1 = Mid2 + Interval
16 CountA = 0, CountB = 0, CountC = 0, CountD = 0
17 while (Pile 2 has more cards) {
18     Read the top card X from Pile 2
19     if (X.Gender == “F” ) {
20         if (X.Total ≥ Mid1) {
21             CountA = CountA + 1
22         }
23         if (X.Total < Mid1 and X.Total ≥ Mid2) {
24             CountB = CountB + 1
25         }
26         if (X.Total < Mid2 and X.Total ≥ Mid3) {
27             CountC = CountC + 1
28         }
29         if (X.Total < Mid3) {
30             CountD = 1
31         }
32     }
33     Move X to Pile 1
34 }

```

- Error in Line 1
- Error in Line 19
- Error in Line 20
- Error in Line 23
- Error in Line 26
- Error in Line 30
- No error

Solution: The objective of the pseudocode is to grade the students based on total marks and count number of male students in each of the grades. To grade the students, the first step is to fix the range of marks for each of the grades. To find the range, we have to compute the minimum and maximum total marks. The variable **MinT** and **MaxT** are used to find the minimum and maximum total marks. To find the minimum total mark, the variable should be initialized with a large value. Similarly, the variable to compute maximum should be initialized with a small value. In Line 1, the statement **MinT = -1, MaxT = 301** is initialized with wrong values. Therefore, **Error in Lien 1**. In the first iteration, we compute the minimum and maximum total marks. After the iteration, the variable **Interval** captures the uniform range of the marks for the grades. The variables **Mid1**, **Mid2** and **Mid3** capture the lower bound for the Grade-1, Grade-2 and Grade-3 respectively.

In the second iteration, we count the male students in each of the grades. We use iterator **X** to iterate cards in Pile 2. In each of the iteration step, we have to check whether the card belongs to a male student. But, the if-condition in the Line 19 is incorrect. Therefore, **Error in Line 19**. In the code-block of the if-condition, we find the grade in which the total marks lie in the mark range of the grade. then, we increment the respecting variable for the grade. But, the variable to count the Grade-4 is not incremented in Line 30. Therefore, **Error in Line 30**. After the end of the iteration, we terminate the execution of the pseudocode.



6. The following pseudocode is executed using the “Shopping bills” dataset. Let **MinT** be a variable that holds the value of the minimum total bill amount. What will be the values of **A**, **B**, **C** and **D** at the end of the execution? [4 Marks]

```
Interval = 1021
Mid3 = MinT + Interval
Mid2 = Mid3 + Interval
Mid1 = Mid2 + Interval
A = 0, B = 0, C = 0, D = 0
while (Pile 1 has more cards) {
    Read the top card X from Pile 1
    if (X.TotalBillAmount > Mid1) {
        A = A + 1
    }
    if (X.TotalBillAmount > Mid2) {
        B = B + 1
    }
    if (X.TotalBillAmount > Mid3) {
        C = C + 1
    }
    else {
        D = D + 1
    }
    Move X to Pile 2
}
```

Answer: **A** = 1, **B** = 3, **C** = 6 and **D** = 24 (Numerical input)

Solution: In the “Shopping bills” dataset, the minimum total bill amount is 92. Therefore, the variable **MinT** hold the value 92. The values of the variables **Mid1**, **Mid2** and **Mid3** are 1113, 2134 and 3155. Tabulate the contents of the dataset and execute it on the pseudocode. In the below table, we refer the column names as follows:

- # - Sequence number
- TBA - Total Bill Amount
- C1 - $X.TotalBillAmount > Mid1$
- C2 - $X.TotalBillAmount > Mid2$
- C3 - $X.TotalBillAmount > Mid3$
- C3-Else - Else part of C3

#	TBA	C1	C2	C3	C3-Else	A	B	C	D
1	567				True				1
2	1525			True				1	
3	1341			True				2	
4	123				True				2
5	4174	True	True	True		1	1	3	
6	354				True				3
7	96				True				4
8	3132		True	True			2	4	
9	595				True				5
10	378				True				6
11	893				True				7
12	186				True				8
13	3060		True	True			3	5	
14	656				True				9
15	229				True				10
16	187				True				11
17	279				True				12
18	603				True				13
19	592				True				14
20	622				True				15
21	128				True				16
22	315				True				17
23	888				True				18
24	92				True				19
25	1364			True				6	
26	276				True				20
27	340				True				21
28	514				True				22
29	106				True				23
30	798				True				24

Note that the else in the end of the iteration is the else-part of the immediate if-condition, that is the third if-condition. At the end of the execution, **A = 1**, **B = 3**, **C = 6** and **D = 24**

7. The following pseudocode is executed using the “Shopping bills” dataset to find the number of eligible bills for top three total bill amounts. Choose the correct code block to complete the pseudocode. [3 Marks]

```

FirstT = 0, SecondT = 0, ThirdT = 0
while (Pile 1 has more cards) {
    Read the top card X from Pile 1
    if (X.TotalBillAmount ≥ FirstT) {
        ThirdT = SecondT
        SecondT = FirstT
        FirstT = X.TotalBillAmount
    }
    if (X.TotalBillAmount < FirstT and X.TotalBillAmount ≥ SecondT) {
        ThirdT = SecondT
        SecondT = X.TotalBillAmount
    }
    if (X.TotalBillAmount < SecondT and X.TotalBillAmount > ThirdT) {
        ThirdT = X.TotalBillAmount
    }
    Move X to Pile 2
}
Count = 0
while (Pile 2 has more cards) {
    Read the top card X from Pile 2
    *****
    *      Fill the code      *
    *****
    Move X to Pile 1
}

```

- a. if (**X.TotalBillAmount** < **ThirdT**) {
 Count = **Count** + 1
}
- b. if (**X.TotalBillAmount** ≥ **FirstT**) {
 Count = **Count** + 1
}
- c. if (**X.TotalBillAmount** > **ThirdT**) {
 Count = **Count** + 1
}
- d. if (**X.TotalBillAmount** ≥ **ThirdT**) {
 Count = **Count** + 1
}
- e. None of the above

Solution: The pseudocode consists of two non-nested iterations. In the first iteration, we find the top three distinct total bill amounts from the dataset. In the second iteration, we are counting the number of bills eligible for the top three prizes. A bill is said to be eligible for a prize if the total bill amount is at least the third top total bill amount. We increment the count when we encounter such a bill. Therefore, the following code-block completes the pseudocode.

```
if ( $\mathbf{X.TotalBillAmount} \geq \mathbf{ThirdT}$ ) {  
     $\mathbf{Count = Count + 1}$   
}
```



8. The following pseudocode is executed using the “Shopping bills” dataset. What will the values of **A**, **B** and **C** represent at the end of the execution?

[3 Marks]

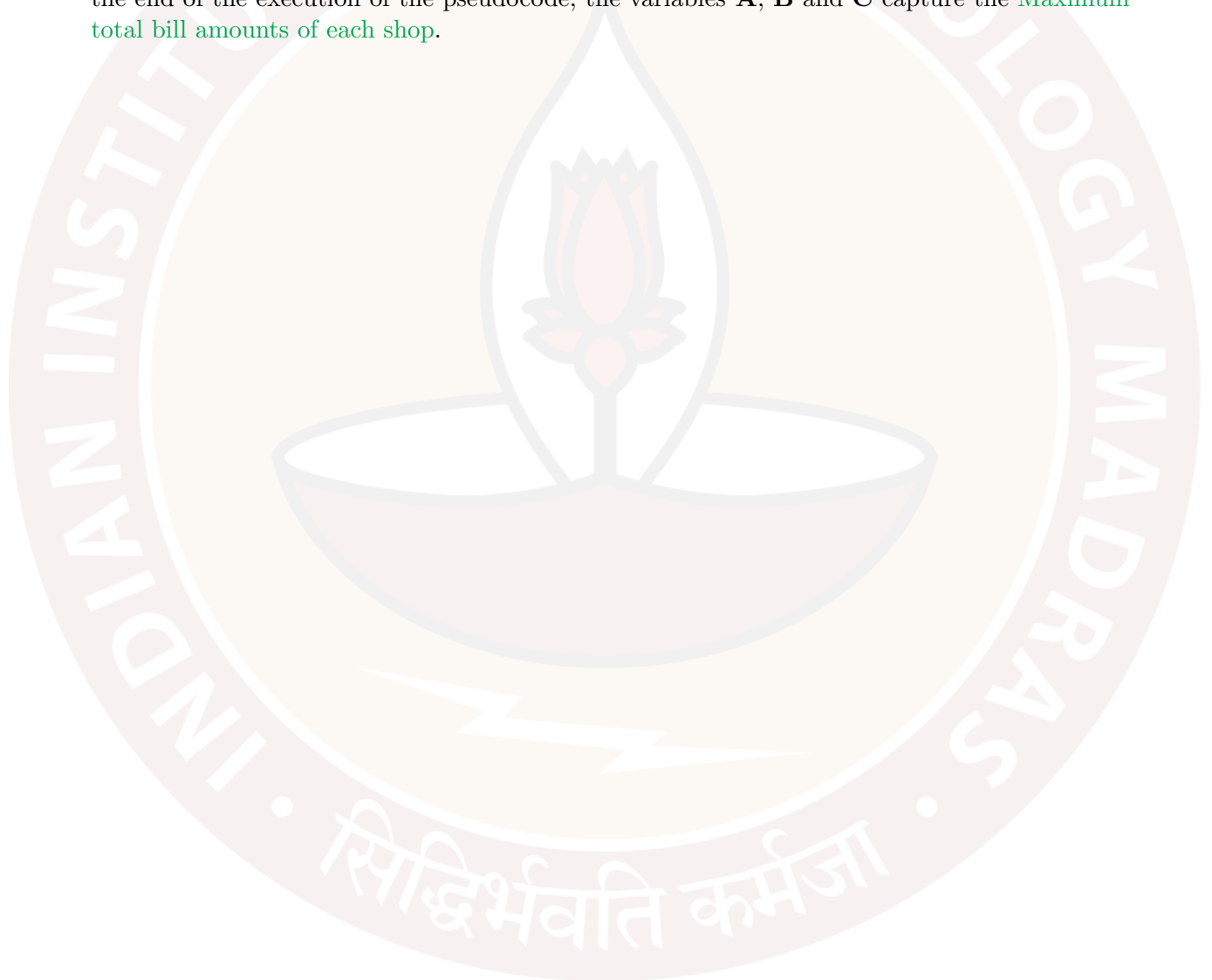
```
A = 0, B = 0, C = 0
while (Pile 1 has more cards) {
    Read the top card X from Pile 1
    A, B, C = DoSomething(X, A, B, C)
    Move X to Pile 2
}

Procedure DoSomething(Y, A, B, C) {
    if (Y.ShopName == “SV Stores” and Y.TotalBillAmount > A) {
        A = Y.TotalBillAmount
    }
    if (Y.ShopName == “Big Bazaar” and Y.TotalBillAmount > B) {
        B = Y.TotalBillAmount
    }
    if (Y.ShopName == “Sun General” and Y.TotalBillAmount > C) {
        C = Y.TotalBillAmount
    }
    return ([A, B, C])
End DoSomething
```

- Maximum total bill amount of each customer
- Maximum total bill amount of each shop
- Shop name of the bill with maximum total bill amount
- Maximum total bill amount

Solution: The pseudocode consists of two parts. The first part is the main pseudocode and the second part is the procedure **DoSomething**. First we consider the procedure **DoSomething**. The procedure is called with four parameters. The first parameter **Y** is denoting a card from the data set, and remaining three parameters are integer values. There are three if-conditions in the procedure. For each shop in the shopping bills dataset, the if-condition check whether the bill belong to the shop and the total bill amount is more the integer value given as parameter. In the **True** case, the integer value is updated with the total bill amount of the shop. at the end, it returns a list of updated integers. The procedure **DoSomething** captures updates the variable value to the maximum bill total bill amount of the specific shops.

In the main pseudocode, thee cards in the Pile 1 are iterated using the iterator **X**. The variable **A**, **B** and **C** are used to find the maximum total bill amount of each shops respectively. At the end of the execution of the pseudocode, the variables **A**, **B** and **C** capture the **Maximum total bill amounts of each shop**.



9. The following pseudocode is executed using the “Paragraph words” dataset. At the end of the execution, **CountB** captures the number of verbs with letter count less than the average letter count of verbs. But the pseudocode may have mistakes in one or more lines. Identify all such lines (if any). It is a Multiple Select Question (MSQ). [6 Marks]

```
1  Sum = 0, Count = 0
2  while (Pile 1 has more cards) {
3      Read the top card X in Pile 1
4      Sum, Count = AddIfVerb(X)
5      Move X to Pile 2
6  }
7  AvgT = Sum / Count
8  CountB = 0
9  while (Pile 2 has more cards) {
10     Read the top card X in Pile 2
11     if (X.LetterCount < AvgT) {
12         CountB = CountB + 1
13     }
14     Move X to Pile 1
15 }

16 Procedure AddIfVerb (Y, SumT, CountT)
17     if (Y.PartOfSpeech == "Verb") {
18         SumT = Y.LetterCount
19         CountT = CountT + 1
20     }
21     return ([SumT, CountT])
22 End AddIfVerb
```

- a. Error in Line 4
- b. Error in Line 8
- c. Error in Line 11
- d. Error at Line 12
- e. Error in Line 17
- f. Error at Line 18
- g. Error in Line 21

Solution: The objective of the pseudocode is to find the number of verbs with letter count less than the average letter count of the verbs. We have to do the following steps:

1. To find the average letter count of verbs
2. Filter verbs in the data set and count if the letter count is less than the average letter count of verbs

The procedure **AddIfVerb** is used to update the variables **SumT** and **CountT**, if the current word (passed to the procedure) is a verb. The if-condition checks whether the word is a verb. If the condition is evaluated to **True**, then we have to add the letter count of the word to the variable **SumT**, and increment **CountT** by one. But, the Line 18 is incorrect since it is not adding the letter count. Therefore, **Error in Line 18**. At the end, it returns the updated **SumT** and **CountT**.

In the main pseudocode, we compute the sum of letter count and the number of words using the variables **Sum** and **Count**. In the the first iteration, the procedure **AddIfVerb** is called only with the card **X**. But, the procedure **AddIfVerb** should be called with **X**, **Sum** and **Count**. Therefore, **Error in Line 4**.

After the first iteration, we store the average letter count of verbs in the variable **AvgT**. In the second iteration, we have to filter words with part of speech is verb and the letter count is less than the average letter count. But, the if-condition in the iteration checks only the letter count less than the average letter count of verb. It is not checking whether the part of speech is verb. Therefore, **Error in Line 11**.

10. The following pseudocode is executed using the “Words” table. What will be the value of **A** at the end of the execution? [6 Marks]

```
Sum = 0
Count = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    Sum, Count = AddWord(X, Sum, Count)
    Move X to Table 2
}
B = Sum / Count
Sum = 0, Count = 0, A = 0
Move all rows from Table 2 to Table 1
while (Table 1 has more rows) {
    Read the first row X in Table 1
    Sum, Count = AddWord(X, Sum, Count)
    if (X.Word ends with a full stop) {
        C = Sum / Count
        A = DoSomething(C, B, A)
        Sum = 0, Count = 0
    }
    Move X to Table 2
}

Procedure AddWord (Y, SumT, CountT)
    SumT = SumT + Y.LetterCount
    CountT = CountT + 1
    return ([SumT, CountT])
End AddWord

Procedure DoSomething (P, Q, R)
    if (P < Q) {
        return (R + 1)
    }
    else {
        return (R)
    }
End DoSomething
```

A = ____ (Numerical input)

Solution: The pseudocode consists of three parts, namely – the main pseudocode and two procedures **AddWord** and **DoSomething**. First, we consider the procedure **Addword**. It is called with three parameters **Y**, **SumT** and **CountT**. the procedure adds the letter count of the word **Y** to **SumT** and increment **CountT** by one. Then, it returns the updated **SumT** and **CountT**. Therefore, the procedure **AddWord** sums the letter count of the first parameter to the second parameter and increment the third parameter.

In the main pseudocode consists of two non-nested iterations. In the first iteration, we use iterator **X** to iterate rows in Table 1. In each step of the iteration, we call the procedure **AddWord** with the parameters **X**, **Sum** and **Count**. The return value is stored in the variable **Sum** and **Count**. That is, the variable **Sum** is added with the letter count of current word and **Count** is incremented by one in each step. At the end of the iteration, the variable **Sum** and **Count** captures the sum of letter count and the number of words in the data set.

#	Word	LC	Sum	Count	#	Word	LC	Sum	Count
0	It	2	2	1	33	the	3	173	34
1	was	3	5	2	34	Monday	6	179	35
2	Monday	6	11	3	35	mood	4	183	36
3	morning.	7	18	4	36	of	2	185	37
4	Swaminathan	11	29	5	37	work	4	189	38
5	was	3	32	6	38	and	3	192	39
6	reluctant	9	41	7	39	discipline.	10	202	40
7	to	2	43	8	40	He	2	204	41
8	open	4	47	9	41	shuddered	9	213	42
9	his	3	50	10	42	at	2	215	43
10	eyes.	4	54	11	43	the	3	218	44
11	He	2	56	12	44	very	4	222	45
12	considered	10	66	13	45	thought	7	229	46
13	Monday	6	72	14	46	of	2	231	47
14	specially	9	81	15	47	school:	6	237	48
15	unpleasant	10	91	16	48	the	3	240	49
16	in	2	93	17	49	dismal	6	246	50
17	the	3	96	18	50	yellow	6	252	51
18	calendar.	8	104	19	51	building;	8	260	52
19	After	5	109	20	52	the	3	263	53
20	the	3	112	21	53	fire-eyed	8	271	54
21	delicious	9	121	22	54	Vedanayagam,	11	282	55
22	freedom	7	128	23	55	his	3	285	56
23	of	2	130	24	56	class	5	290	57
24	Saturday	8	138	25	57	teacher,	7	297	58
25	And	3	141	26	58	and	3	300	59
26	Sunday,	6	147	27	59	headmaster	10	310	60
27	it	2	149	28	60	with	4	314	61
28	was	3	152	29	61	his	3	317	62
29	difficult	9	161	30	62	thin	4	321	63
30	to	2	163	31	63	long	4	325	64
31	get	3	166	32	64	cane...	4	329	65
32	into	4	170	33					

Tabulate the content of the data set and execute it on the pseudocode. At the end of the execution, **Sum** = 329 and **Count** = 65. Therefore, $B = 329 / 65 \simeq 5.06$.

In the second iteration, we find average letter count of each sentence. In the if-condition inside the iteration, we check whether the current word is the end of the sentence. If the condition evaluates to **True**, then we call the procedure **DoSomething** to check whether the average letter count of the current sentence less than the average letter count of the data set. The variable **A** is incremented for each sentence with average letter count less than the average letter count of the dataset. The variable **A** captures the number of sentences with average letter count less than average letter count of the dataset. In the dataset, there are five sentences with average letter count 4.5, 5.14, 6.25, 4.67 and 5.08. Therefore the value of the variable **A** is 2.



COMPUTATIONAL THINKING
Week - 4

Practice Assignment Solution

1. The given pseudocode is executed using the “Scores” table. What will the value of **Count** represent at the end of the execution?

```
Count = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    Move X to Table 2
    while (Table 1 has more rows) {
        Read the first row Y in Table 1
        Count = Count + DoSomething(X, Y)
        Move Y to Table 3
    }
    Move all rows from Table 3 to Table 1
}

Procedure DoSomething(A, B)
    if (A.Gender == B.Gender and A.CityTown == B.CityTown) {
        return (1)
    }
    else {
        return (0)
    }
End DoSomething
```

- a. Number of pairs of students with the same gender
 - b. Number of pairs of students with the same city/town
 - c. Number of pairs of students with the same gender and the same City/Town
 - d. Number of pairs of students with the same gender or the same City/Town
-

Solution: There are two portions in this pseudocode. The first one is main part of the pseudocode and the second one is the procedure **DoSomething**. First we look at the procedure **DoSomething**. It is called with two parameters **A** and **B**. From the context of this code, both these variables are representing some rows in “Scores” table. The if-condition

A.Gender == B.Gender and A.CityTown == B.CityTown

captures whether the City/Town and Gender of the two students the same. Therefore, the procedure **DoSomething** returns 1 when the gender and City/Town of the students that are passed as parameter are the same. Otherwise, it returns 0.

In the main pseudocode, we are executing the nested iteration in this pseudocode. A variable named **Count** is initialized with value 0. In the first iteration, we read the first row in the Table 1 using the iterator **X** and moving it to Table 2. In the second iteration, we read the first row in the Table 1 (after reading the row **X**) using the iterator **Y** and moving it to Table 3. We call the procedure **DoSomething** by passing **X** and **Y** as parameters. By definition of the procedure **Dosomething**, the variable **Count** is getting incremented when the gender and City/Town of both **X** and **Y** are the same. After the inner iteration, we move back all rows from the Table 3 to Table 1. Then we continue the outer iteration. With the nested iterations, we check every pair of rows in the Table 1 against the gender and City/Town equality. Therefore, the variable count captures the

Number of pairs of students with the same gender and the same City/Town.

2. Consider the pseudocode given in Question 1. The procedure **DoSomething** is rewritten such that the pseudocode computes the number of pairs of students with the same marks in at least one subject. Choose the correct choice to complete the pseudocode.

Procedure DoSomething(A, B)

* Fill the code *

End DoSomething

- a.
- ```
if (A.Physics == B.Physics and A.Chemistry == B.Chemistry
 and A.Mathematics == B.Mathematics) {
 return (1)
}
else {
 return (0)
}
```
- b.
- ```
if (A.Physics == B.Physics or A.Chemistry == B.Chemistry
    or A.Mathematics == B.Mathematics) {
    return (1)
}
```
- c.
- ```
if (A.Physics == B.Physics or A.Chemistry == B.Chemistry
 or A.Mathematics == B.Mathematics) {
 return (1)
}
else {
 return (0)
}
```
- d.
- ```
if (A.Physics == B.Physics and A.Chemistry == B.Chemistry
    and A.Mathematics == B.Mathematics) {
    return (1)
}
```

Solution: In this question, the purpose of the procedure **DoSomething** is as follows: To check whether students (**A** and **B**) must have equal marks in at least one subject. The if-condition **A.Physics == B.Physics** check whether both have equal marks in Physics. Our aim is to check for all subjects that at least in one subject both have the same marks. Therefore, we have to use **or** operator to combine the conditions, to make a single if-condition. Thus, the condition is

A.Physics == B.Physics or A.Chemistry == B.Chemistry or A.Mathematics == B.Mathematics.

We have to return 1 if the above condition is **True** and return 0 otherwise. Finally, the following code-block can complete the procedure **DoSomething**.

```
if (A.Physics == B.Physics or A.Chemistry == B.Chemistry
    or A.Mathematics == B.Mathematics) {
    return (1)
}
else {
    return (0)
}
```

3. The given pseudocode is executed using the “Words” table to count the number of pairs of words where both have the same part of speech and letter count. But the pseudocode have some mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ).

```
1  Count = 0
3  while (Table 1 has more rows) {
4      Read the first row X in Table 1
5      Move X to Table 2
6      while (Table 1 has more rows) {
7          Read the first row Y in Table 1
8          Count = Count + DoTheyMatch(X, Y)
9          Move Y to Table 3
10     }
11     Move all rows from Table 3 to Table 2
12 }

13 Procedure DoTheyMatch(A, B)
14     if (A.PartOfSpeech == B.PartOfSpeech or A.LetterCount == B.LetterCount) {
15         return (1)
16     }
17     else {
18         return (0)
19     }
20 End DoTheyMatch
```

- a. Error in Line 1
- b. Error in Line 7
- c. Error in Line 11
- d. Error in Line 13
- e. Error in Line 14
- f. Error in Line 18

Solution: We start with the main pseudocode. A variable named **Count** is initialed to value 0. As mentioned in the question statement, the purpose of the variable **Count** is to capture the number of pairs of words with the same part of speech and the same letter count. Since, we have to compare every pair of words, we are executing the nested iteration. In the first (outer) iteration, we use the iterator (variable) **X** to iterate the rows in the Table 1 and moving it to Table 2. In the inner iteration, we use the iterator **Y** to iterate the rows of the Table 1 and moving it to Table 3. We call the procedure **DoTheyMatch** to check the condition to increment the variable **Count**. After the inner iteration, we have to move the rows of Table 3 to Table 1. In Line 11, it is mentioned that [Move all rows from Table 3 to Table 2](#). This is incorrect statement as per the nested iteration procedure. Therefore, [Error in Line 11](#). The rest of the part of the main pseudocode is correct.

In the procedure **DoTheyMatch**, two rows **A** and **B** are passed as parameters. The purpose of the procedure is to return 1 when the both words in the row are having the same part of speech and the letter count. Then, this should be an **and** condition since we need to satisfy both part of speech equality and letter count equality. Thus, the if-condition given in Line 14 is incorrect as it uses **or** operator to combine the conditions. Therefore, [Error in Line 14](#).

4. The following pseudocode is executed using the “Scores” table. Assume that the procedure **GetCountOfMatched** will take a table as a parameter, and it will return the number of pairs of students in the input table where both have the same date of birth. Then, what will the value of **Count** represent at the end of the execution?

```
while (Table 1 has more rows) {  
    Read the first row X in Table 1  
    if (“JAN 01” ≤ X.DoB ≤ “MAR 31”){  
        Move X to T1  
    }  
    if (“APR 01” ≤ X.DoB ≤ “JUN 30”){  
        Move X to T2  
    }  
    if (“JUL 01” ≤ X.DoB ≤ “SEP 30”){  
        Move X to T3  
    }  
    if (“OCT 01” ≤ X.DoB ≤ “DEC 31”){  
        Move X to T4  
    }  
}  
Count = GetCountOfMatched(T1)  
Count = Count + GetCountOfMatched(T2)  
Count = Count + GetCountOfMatched(T3)  
Count = Count + GetCountOfMatched(T4)
```

- a. Number of pairs of students with the same date of birth
- b. Pairs of students with the same date of birth
- c. Number of pairs of students with the same date of birth where the month of birth is on or before MARCH
- d. Number of pairs of students in each table

Solution: First, assume that there exists a procedure named **GetCountOfMatched** that takes a table as a parameter, and return the number of pair of students with the same date of birth in the input table.

In the main pseudocode, we are executing an iteration on the Table 1. We use the iterator **X** to iterate the rows of the Table 1 and moving it to different tables based on the date of birth. There are four if-conditions inside the iteration.

- The first if-condition captures only those students who born on the months January, February or March, and move the row **X** to the Table T1.
- The second if-condition captures only those students who born on the months April, May or June, and move the row **X** to the Table T2.
- The third if-condition captures only those students who born on the months July, august or September, and move the row **X** to the Table T3.
- The first if-condition captures only those students born on the months October, November or December, and move the row **X** to the Table T4.

Therefore, in the first iteration, we bins the rows based on month of birth of the students. The procedure **GetCountMatched** is called with the table T1, and it will return the number of pairs of students with the same date of birth in the table T1. We are storing it in the variable **Count**. Further, we call the procedure **GetCountMatched** with the tables T2, T3 and T4, and add the return values to the variable **Count**. Thus, the variable **Count** is capturing the sum of number of pairs of the students in each table with the same date of birth. Therefore, the variable **Count** capture the

Number of pairs of students with the same date of birth.

5. What does the given pseudocode compute using the “Words” table?

```
while (Table 1 has more rows) {  
    Read the first row X in Table 1  
    if ( $0 \leq \mathbf{DoSomething}(\mathbf{X}) \leq 1$ ) {  
        Move X to T1  
    }  
    if ( $2 \leq \mathbf{DoSomething}(\mathbf{X}) \leq 3$ ) {  
        Move X to T2  
    }  
    if ( $4 \leq \mathbf{DoSomething}(\mathbf{X}) \leq 5$ ) {  
        Move X to T3  
    }  
    if ( $\mathbf{DoSomething}(\mathbf{X}) \geq 6$ ) {  
        Move X to T4  
    }  
}  
  
Procedure DoSomething (Y)  
    i = 1  
    Count = 0  
    while (i ≤ Y.LetterCount) {  
        if (ith letter of Y.Word is a vowel) {  
            Count = Count + 1  
        }  
        i = i + 1  
    }  
    return (Count)  
End DoSomething
```

- Bins the rows in the “Words” table into four tables based on the letter count
- Bins the rows in the “Words” table into four tables based on the vowel count of the words
- Bins the rows in the “Words” table into four tables based on the letter count and the vowel count
- Bins the rows in the “Words” table into four tables based on the letter count or the vowel count

Solution: First we look at the procedure **DoSomething**. It is called with a parameter **Y**. In each of the procedure-call, we pass a row of the “Words” table as a parameter. Therefore, the parameter **Y** is representing a row in the “Words” table. In the procedure **DoSomething**, we declared two variables **i** and **Count** with initial values 1 and 0 respectively. We are iterating the letters in the word in the row **Y**. The role of the variable **i** is to point the index of the current letter in the word. We have an if-condition in the iteration that checks whether the current (that is the i^{th} letter of the word) letter pointed in the index **i** is a vowel. The variable **Count** is getting incremented when a vowel is encountered in the word. At the end of the iteration, the variable **Count** captures the number of vowels in the word in the row **Y**. Therefore, the procedure **DoSomething** returns the number of vowels in the word in the row **Y** which is passed as the parameter.

In the main pseudocode, we are executing an iteration on the Table 1. We use the iterator **X** to iterate the rows of the Table 1 and moving it to different tables based on the date of birth. There are four if-conditions inside the iteration.

- The first if-condition captures only those words with vowel count 0 or 1, and move the row **X** to the Table T1.
- The second if-condition captures only those words with vowel count 2 or 3, and move the row **X** to the Table T2.
- The third if-condition captures only those words with vowel count 4 or 5, and move the row **X** to the Table T3.
- The first if-condition captures only those words with vowel count at least 6 and move the row **X** to the Table T4.

Therefore, the pseudocode

Bins the rows in the “Words” table into four tables based on the vowel count of the words.

6. The following pseudocode is executed using the “Words” table. Consider the tables T1, T2, T3 and T4 computed in Question 5. Also, consider the procedure **DoSomething** in Question 5. Then, what will the value of **Count** represent at the end of the execution?

```
Count = GetCountOfMatched(T1)
Count = Count + GetCountOfMatched(T2)
Count = Count + GetCountOfMatched(T3)
Count = Count + GetCountOfMatched(T4)

Procedure GetCountOfMatched (Table 1)
  CountA = 0
  while (Table 1 has more rows) {
    Read the first row X from Table 1
    Move X to Table 2
    while (Table 1 has more rows) {
      Read the first row Y from Table 1
      CountA = CountA + DoTheyMatch(X, Y)
      Move Y to Table 3
    }
    Move all rows from Table 3 to Table 1
  }
  return (CountA)
End GetCountOfMatched

Procedure DoTheyMatch (A, B)
  if (A.LetterCount == B.LetterCount
    and DoSomething(A) == DoSomething(B)) {
    return (1)
  }
  return (0)
End DoTheyMatch
```

- a. Number of pairs of words with the same letter count
- b. Number of pairs of words with the same letter count as well as vowel count
- c. Number of pairs of words with the same letter count or the same vowel count
- d. Number of pairs of words with the same letter count but the different vowel count

Solution: First, assume that there exists a procedure named **DoSomething** that takes a row from the “Words” table as a parameter, and return the number of vowels in the word of the input row.

There are three parts in this pseudocode. The first one is main part of the pseudocode, the second one is the procedure **GetCountOfMatched**, and the third is the procedure **DoTheyMatch**. First we look at the procedure **DoTheyMatch**. It is called with two parameters **A** and **B**. From the context of this code, both these variables are representing some rows in “Words” table. The if-condition

A.LetterCount == B.LetterCount and DoSomething(A) == DoSomething(B)

captures whether the letter count and the vowel count of the two words are the same. Therefore, the procedure **DoTheyMatch** returns 1 when the letter count and the vowel count of the two words are the same. Otherwise, it returns 0.

In the procedure **GetCountOfMatched**, a table named “Table 1” is passed a parameter. We are executing nested iterations in this procedure. In the first (outer) iteration, we use the iterator (variable) **X** to iterate the rows in the Table 1 and moving it to Table 2. In the inner iteration, we use the iterator **Y** to iterate the rows of the Table 1 and moving it to Table 3. We call the procedure **DoTheyMatch** to update the variable **CountA**. After the inner iteration, we have to move the rows of Table 3 to Table 1. Based on the definition of the procedure **DoTheyMatch**, the variable **CountA** is getting incremented if the words of the rows **X** and **Y** are having the same letter count and the same vowel count. Since we try for all pairs of words in Table 1, the variable **CountA** captures the number of pairs of words in Table 1 with the same letter count and the same vowel count. Therefore, the procedure **GetCountOfMatched** returns the number of pairs of words in Table 1 with the same letter count and the same vowel count.

The tables T1, T2, T3 and T4 are the four bins obtained from pseudocode of Question 5. The procedure **GetCountMatched** is called with the table T1, and the return value is stored in the variable **Count**. Further, we call the procedure **GetCountMatched** with the tables T2, T3 and T4, and add the return values to the variable **Count**. Thus, the variable **Count** is capturing the sum of the number of pairs of words with the same letter count and the same vowel count in each table. Therefore, the variable **Count** capture the

Number of pairs of words with the same letter count as well as vowel count.

7. Let **X** and **Y** be two rows from the “Scores” table. Let **isFairPair(X, Y)** be a procedure to find whether **X** and **Y** is fair pair. Based on the procedure **isFairPair**, choose the correct option for the definition of fairness.

```
Procedure isFairPair (X, Y)
  XCount = 0, YCount = 0
  if (X.Physics > Y.Physics) {
    XCount = XCount + 1
  }
  if (X.Physics < Y.Physics) {
    YCount = YCount + 1
  }
  if (X.Chemistry > Y.Chemistry) {
    XCount = XCount + 1
  }
  if (X.Chemistry < Y.Chemistry) {
    YCount = YCount + 1
  }
  if (X.Mathematics > Y.Mathematics) {
    XCount = XCount + 1
  }
  if (X.Mathematics < Y.Mathematics) {
    YCount = YCount + 1
  }
  if (XCount > 0 and YCount > 0) {
    return (1)
  }
  else {
    return (0)
  }
End isFairPair
```

- a. **X** and **Y** is said to be a *fair pair* if and only if **X** has more marks than **Y** in Physics and **Y** has more marks than **X** in Mathematics.
- b. **X** and **Y** is said to be a *fair pair* if and only if **X** has more marks than **Y** in Mathematics and **Y** has more marks than **X** in Physics.
- c. **X** and **Y** is said to be a *fair pair* if and only if **X** has more marks than **Y** in at least one subject and **Y** has more marks than **X** in at least one subject.
- d. **X** and **Y** is said to be a *fair pair* if and only if **X** has more marks than **Y** in one subject, **Y** has more marks than **X** in one subject and both got the same marks in one subject.

Solution: We are given two rows (students data) from the “Scores” table. The procedure **isFairPair** checks whether the pair of students is fair. In the procedure **isFairPair**, we declared two variables named **XCount** and **YCount**, and initialized with value 0. There are seven if-conditions in the procedure code-block. The first six if-conditions are checking subject marks of the student **X** with the subject marks of student **Y**. For example, the first if-condition checks $X.Physics > Y.Physics$ and increments the variable **XCount** when the condition is evaluated to **True**. In the second if-condition check $X.Physics < Y.Physics$ and increments the variable **XCount** when the condition is evaluated to **True**. Thus, the variable **XCount** is incremented when there is a subject with the student **X** scored better than the student **Y**. Similarly, the variable **XCount** is incremented when there is a subject with the student **Y** scored better than the student **X**. Therefore, the variable **XCount** captures the number of subjects in which the student **X** scored better than the student **Y**, and the variable **YCount** captures the number of subjects in which the student **Y** scored better than the student **X**.

The final if-condition $XCount > 0 \text{ and } YCount > 0$ checks that both variables **XCount** and **YCount** are more than zero. The above condition is evaluated to **True** when the student **X** has scored more marks than **Y** in at least one subject and the student **Y** has scored more marks than **X** in at least one subject. Therefore, the definition of the fairness is as follows:

X and **Y** is said to be a *fair pair* if and only if **X** has more marks than **Y** in at least one subject and **Y** has more marks than **X** in at least one subject.

8. Let **X** and **Y** be two words from the “Words” table. The given procedure **CountPair** counts the number of sentences that contain both words **X** and **Y**. But the procedure may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ).

```
Procedure CountPair (X, Y)  
  XCount = 0, YCount = 0, Count = 0  
  while (Table 1 has more rows) {  
    Read the first row Z in Table 1  
    if (Z.Word == X) {  
      XCount = XCount + 1  
    }  
    if (Z.Word == Y) {  
      YCount = YCount + 1  
    }  
    if (Z.Word ends with a full stop) {  
      if (XCount > 0 or YCount > 0) {  
        Count = Count + 1  
      }  
    }  
    Move Z to Table 2  
  }  
  return (Count)  
End CountPair
```

- a. Condition to increment **XCount** is incorrect
- b. Condition to increment **YCount** is incorrect
- c. Condition to increment **Count** is incorrect
- d. **XCount** and **YCount** are not reinitialized after reading the end of a sentence

Solution: We are given two words from the “Words” table. The procedure **CountPair** counts the number of sentences that contain both the words **X** and **Y**. In the procedure **CountPair**, three variables **XCount**, **YCount** and **Count** are declared and initialized with value zero. The purpose of the variables **XCount** and **YCount** are to count the number of times the words **X** and **Y** are appeared in a sentence respectively.

We are executing an iteration on the Table 1. We use the iterator **Z** to iterate the rows of Table 1. The first if-condition in the iteration checks that the word in **Z** is the same as **X**, and increment the variable **XCount** when the condition is evaluated to **True**. The second if-condition in the iteration checks that the word in **Z** is the same as **Y**, and increment the variable **YCount** when the condition is evaluated to **True**. The third if-condition in the iteration checks that the word **Z** is an end of sentence. If it is evaluated to **True**, then the if-condition (inner if-condition in nested) checks that the both variables **XCount** and **YCount** are more than one. But the if-condition **XCount > 0 or YCount > 0** is incorrect to update the variable **Count**. Therefore, the

condition to increment **Count** is incorrect.

After reading the end of a sentence, we need to reset the variable **XCount** and **YCount**. But, it is not implemented in the iteration. Therefore, the variables

XCount and **YCount** are not reinitialized after reading the end of a sentence.

After completion of the iteration, the variable **Count** is returned by the procedure.

COMPUTATIONAL THINKING

Mock Qualifier Solution

Total Marks: 25

Duration: 45 mins

1. The following pseudocode is executed using the “Scores” table. What will the value of **A** represent at the end of the execution? It is a Multiple Select Question (MSQ). [4 Marks]

```
A = 0
while (Table 1 has more rows) {
  Read the first row X in Table 1
  if (X.Gender == “M” or X.Total > 80) {
    A = A + 1
  }
  Move X to Table 2
}
```

- a. Number of male students with total marks more than 80
- b. Number of male students + Number of female students with total marks more than 80
- c. Number of male students
- d. Number of male students with total marks more than 80 + Number of female students with total marks at most 80
- e. Number of students with total marks more than 80 + Number of male students with total marks at most 80

Solution: The pseudocode has an iteration. We use the variable **X** to iterate the rows of Table 1. In the iteration process, we have a filter **X.Gender == "M" or X.Total > 80**. The filter has two conditional statements that are merged by an **or** operator. The first condition **X.Gender == "M"** (refer it as C1) filters only the male students, and the second condition **X.Total > 80** (refer it as C2) filters only the students with total marks more than 80. Since the conditions are merged with **or** operator, for any row, either one of the condition is **True** then the variable **A** will be incremented.

Evaluation of conditional statement with **or** operator: Let **C1 or C2** be the conditional statement. The condition is evaluated as follows:

- If C1 is evaluated **True** then execute the **if-block**.
- If C1 is evaluated **False** then evaluate C2. If C2 is evaluated **True** then execute the **if-block**. Otherwise, do not execute the **if-block**.

Let **Y** be the set of students who are filtered by C1, and let **Z** be the set of students who are filtered by C2. Note that the sets **Y** and **Z** is need not to be disjoint set. There can be some students presented in both sets. So we have to exclude the students who are presented in the set **Y** from **Z**. Therefore, the set **Z** can be redefined such that

Z- Set of all students who are filtered by C2 but not filtered by C1

By the above definition,

- The size of the set **Y** is the number of male students, and
- the size of the set **Z** is the number of female students with total marks more than 80.

Therefore, at the end of the execution, the variable **A** captures the following:

Number of male students + Number of female students with total marks more than 80

Similarly, **C1 or C2** is equivalent to **C2 or C1**. By applying first filter as C2, let **Y** be the set of students who are filtered by C2, and let **Z** be the set of students who are filtered by C2 but not filtered by C1. By the above definition,

- The size of the set **Y** is the number of students with total marks more than 80, and
- the size of the set **Z** is the number of male students with total marks at most 80.

Therefore, at the end of the execution, the variable **A** captures the following:

Number of students with total marks more than 80 + Number of male students with total marks at most 80

2. Consider the procedure **doCounting** for the following questions.

[12 Marks]

```
1  Procedure doCounting ()
2      A = 0, B = 0, C = 0
3      P = False, Q = False
4      while (Table 1 has more rows) {
5          Read the first row X in Table 1
6          if (X.PartOfSpeech == "Pronoun" and Q) {
7              B = B + X.LetterCount
8          }
9          if (X.PartOfSpeech == "Noun") {
10             if (P) {
11                 C = C + X.LetterCount
12             }
13             P = True
14         }
15         else {
16             P = False
17         }
18         Q = True
19         if (X.Word ends with a full stop) {
20             A = A + X.LetterCount
21             Q = False
22         }
23         Move X to Table 2
24     }
25     return ([A, B, C])
26 End doCounting
```

- (i) What will the value of **A** represent at the end of the procedure?
- Sum of letter count of words which are the end of each sentence and the part of speech is "Pronoun"
 - Sum of letter count of words which are the end of each sentence except for the first sentence.
 - Sum of letter count of words which are the end of each sentence
 - Sum of letter count of words which are the end of each sentence but not the end of the paragraph

Solution: The variable **A** is initialized with value 0. It is getting updated in one place during the course of execution. At Line 20, the variable **A** is added with the value of the letter count of the current word which is filtered by the condition

X. Word ends with a full stop.

That is, the variable **A** sums up the letter counts of the words which are the end of each sentence. Therefore, the variable **A** captures the following:

Sum of letter count of words which are the end of each sentence

- (ii) What will the value of **B** represent at the end of the procedure?
- Sum of letter count of all pronouns which appear at the end of a sentence
 - Sum of letter count of all pronouns except the pronouns which appear at the end of a sentence
 - Sum of letter count of all pronouns except the pronouns which appear at the beginning of a sentence
 - Sum of letter count of all pronouns which appear at the beginning of a sentence

Solution: The variable **B** is initialized with value 0. It is getting updated in one place during the course of execution. At Line 7, the variable **B** is added with the value of the letter count of the current word which is filtered by the condition

X.PartOfSpeech == "Pronoun" and Q.

The filter is based on two conditions merged with an **and** operator. The first condition is to check the part of speech is "Pronoun" and the second condition is to check the variable **Q** is evaluated to **True**. We have to find what is the role of the variable **Q** in this procedure. The variable **Q** is initially set to **False**. During the course of execution, **Q** is set to **True** in line 18 and set to **False** in Line 21. For each row in the table, after reading the row, **Q** is set to **True**. But, while reading the end of a sentence, **Q** is set to **False**. That is, when we read the first word of each sentence, **Q** is set to **False**. Therefore, the **Q** is set to **True** when the current word is not a beginning of a sentence.

By combining the above two conditions, the filter in Line 6 captures the pronouns which are not the beginning of a sentence. Therefore, the variable **B** captures the following:

Sum of letter count of all pronouns except the pronouns which appear at the beginning of a sentence.

- (iii) What will the value of **C** represent at the end of the procedure?
- Sum of letter count of all nouns except the first in each sentence
 - Sum of letter count of alternative nouns
 - Sum of letter count of alternative nouns in each consecutive noun sequence
 - Sum of letter count of all consecutive nouns except the first in each such sequence

Solution: The variable **C** is initialized with value 0. It is getting updated in one place during the course of execution. At Line 11, the variable **C** is added with the value of the letter count of the current word which is filtered by the condition

X.PartOfSpeech == "Noun" and P.

Note that the nested if-conditions are merged using the **and** operator. The first condition is to check the part of speech is "Noun" and the second condition is to check the variable **P** is evaluated to **True**. We have to find what is the role of the variable **P** in this procedure. The variable **P** is initially set to **False**. During the course of execution, **P** is set to **True** in line 13 and set to **False** in Line 16. For each row in the table, after reading a noun, **P** is set to **True**. But, while reading a word which is not a noun, **P** is set to **False**. That is, when we read a noun immediate to a next noun in the same sentence, **P** is set to **True**. Therefore, the **P** is set to **True** for all nouns in a sequence except for the first noun in the sequence.

By combining the above two conditions, the filters in Line 9 and 10 together capture the nouns which are not the first in a sequence of nouns. Therefore, the variable **C** captures the following:

Sum of letter count of all consecutive nouns except the first in each such sequence

- (iv) If the **and** operator in Line 6 is replaced by the **or** operator, then what will the value of **B** represent at the end of the procedure?
- Sum of letter count of all pronouns except the first in each sentence
 - Sum of letter count of all pronouns or the first word of each sentence
 - Sum of letter count of all words except the pronouns which are at the beginning of a sentence
 - Sum of letter count of all words except the words which are the first word of a sentence and the part of speech is not "Pronoun"

Solution: The variable **B** is initialized with value 0. It is getting updated in one place during the course of execution. At Line 7, the variable **B** is added with the value of the letter count of the current word which is filtered by the condition

X.PartOfSpeech == "Pronoun" or **Q**.

The filter is based on two conditions merged with an **or** operator. The first condition is to check the part of speech is "Pronoun" and the second condition is to check the variable **Q** is evaluated to **True**. We have to find what is the role of the variable **Q** in this procedure. The variable **Q** is initially set to **False**. During the course of execution, **Q** is set to **True** in line 18 and set to **False** in Line 21. For each row in the table, after reading the row, **Q** is set to **True**. But, while reading the end of a sentence, **Q** is set to **False**. That is, when we read the first word of each sentence, **Q** is set to **False**. Therefore, the **Q** is set to **True** when the current word is not a beginning of a sentence.

By combining the above two conditions, the filter in Line 6 captures all the pronouns and the all the words which are not the beginning of a sentence. But pronouns in the beginning of a sentence were filtered by the condition. Therefore, the variable **B** captures the following:

Sum of letter count of all words except the words which are the first word of a sentence and the part of speech is not "Pronoun".

3. The following pseudocode is executed using the “Scores” table. At the end of the execution, **Z** captures the number of pairs of students with the sum of difference of each subject marks is the same as the difference of the total marks. Choose the correct choice to complete the pseudocode. [4 marks]

```

1  Z = 1
2  while (Table 1 has more rows) {
3      Read the first row X in Table 1
4      Move X to Table 2
5      while (Table 1 has more rows) {
6          Read the first row Y in Table 1
7          value = Proc2(X.Physics, Y.Physics)
8          value = value + Proc2(X.Chemistry, Y.Chemistry)
9          value = value + Proc2(X.Mathematics, Y.Mathematics)
10         value = value + Proc2(X.Total, Y.Total)
11         if (value == 0) {
12             Z = Z + 1
13         }
14         Move Y to Table 3
15     }
16     Move all rows from Table 3 to Table 1
17 }
18
19 Procedure Proc2(C, D)
20     if (C < D) {
21         return (C - D)
22     }
23     else {
24         return (D - C)
25     }
26 End Proc2

```

- a. Error in Line 1
- b. Error in Line 7
- c. Error in Line 8
- d. Error in Line 9
- e. Error in Line 10
- f. Error in Line 14
- g. Error in Line 20
- h. No error

Solution: **X** and **Y** are two students in the dataset. The difference of the marks in each subject is as follows:

- $| \mathbf{X}.Mathematics - \mathbf{Y}.Mathematics |$
- $| \mathbf{X}.Physics - \mathbf{Y}.Physics |$
- $| \mathbf{X}.Chemistry - \mathbf{Y}.Chemistry |$

The sum of the differences is given by the following expression:

$$| \mathbf{X}.Mathematics - \mathbf{Y}.Mathematics | + | \mathbf{X}.Physics - \mathbf{Y}.Physics | + | \mathbf{X}.Chemistry - \mathbf{Y}.Chemistry |$$

According to the condition given in the question, we wish to count all pairs of students (**X**, **Y**) such that the sum of the differences given above is equal to the difference in the total marks. In other words, we want to count those pairs for which the following condition is satisfied:

$$| \mathbf{X}.Mathematics - \mathbf{Y}.Mathematics | + | \mathbf{X}.Physics - \mathbf{Y}.Physics | + | \mathbf{X}.Chemistry - \mathbf{Y}.Chemistry | \\ = | \mathbf{X}.Total - \mathbf{Y}.Total |$$

A little bit of rearranging gives:

$$| \mathbf{X}.Mathematics - \mathbf{Y}.Mathematics | + | \mathbf{X}.Physics - \mathbf{Y}.Physics | + | \mathbf{X}.Chemistry - \mathbf{Y}.Chemistry | \\ - | \mathbf{X}.Total - \mathbf{Y}.Total | = 0$$

If we wish to write a program to achieve this, we can use a variable called **value** that accumulates the differences of the individual subjects.

This is exactly what is happening in lines 7, 8 and 9 of the pseudocode. So, there is nothing wrong in any of these lines. After adding the subject wise differences, we can subtract the difference in the total marks. According to the condition given above, this should result in 0. In line 10, instead of subtracting the difference in the total marks, we are adding it. This is an error. So there is an error in line 10. The correct statement should be:

value = **value** - **Proc2**(**X.Total** , **Y.Total**)

Now, in line 11 we are checking if **value** is 0. This is correct. Line 12 is incrementing **Z**. This is also correct. But this brings us back to the very first line. In line 1, **Z** is initialized to zero. This is an error. So far, lines 1 and 10 have errors in them.

That leaves us with the procedure **Proc2**. Notice that the procedure is returning **C** - **D** if **C** < **D**. That is, it is returning the negative difference between **C** and **D**. At the outset, this seems like an error. But if we look carefully, we are consistently following this throughout. While calling the procedure, **C** always takes the marks of **X** and **D** always takes the marks of **Y**.

To summarize, the pseudocode has two errors, one in line 1 and the other in line 10.

4. Let **X** and **Y** be two rows in the “Library” table. We call **X** and **Y** compatible if either published in the same year or the same genre but not both. Let **CompatiblePair(X, Y)** be a procedure to find whether **X** and **Y** are compatible. Choose the correct implementation of the procedure **CompatiblePair**. [5 marks]

a.

```
Procedure CompatiblePair (X, Y)  
  A = False, B = False, C = False  
  if (X.Year == Y.Year) {  
    A = True  
  }  
  if (X.Genre == Y.Genre) {  
    B = True  
  }  
  if (A or B) {  
    return (True)  
  }  
  return (False)  
End CompatiblePair
```

b.

```
Procedure CompatiblePair (X, Y)  
  A = False, B = False, C = False  
  if (X.Year == Y.Year or X.Genre == Y.Genre ) {  
    return (True)  
  }  
  return (False)  
End CompatiblePair
```

c.

```
Procedure CompatiblePair (X, Y)  
  A = False, B = False, C = False  
  if (X.Year == Y.Year) {  
    A = True  
  }  
  if (X.Genre == Y.Genre) {  
    B = True  
  }  
  if ((A and B) == False or (A or B) == True) {  
    return (True)  
  }  
  return False  
End CompatiblePair
```

d.

```
Procedure CompatiblePair (X, Y)  
  A = False, B = False, C = False  
  if (X.Year == Y.Year) {  
    A = True  
  }  
  if (X.Genre == Y.Genre) {  
    B = True  
  }  
  if ((A and B) == False and (A or B) == True) {  
    return (True)  
  }  
  return False  
End CompatiblePair
```

Solution: We have to use two boolean variables to check that the published year and the genre of the books are the same. In the procedure, we are using **A** to check the publishing year and **B** to check the genre. This can be done by the following piece of codes.

```
if (X.Year == Y.Year) {  
    A = True  
}
```

and

```
if (X.Genre == Y.Genre) {  
    B = True  
}
```

Finally, we have to check the condition that either **A** or **B** must be **True** but not both. This is possible only when one of the variable is set to **True** and the other one is set to **False**. Therefore, the following filter captures the condition:

- One of the variable is set to **True**:

(A or B) == True

- One of the variable is set to **False**:

(A and B) == False

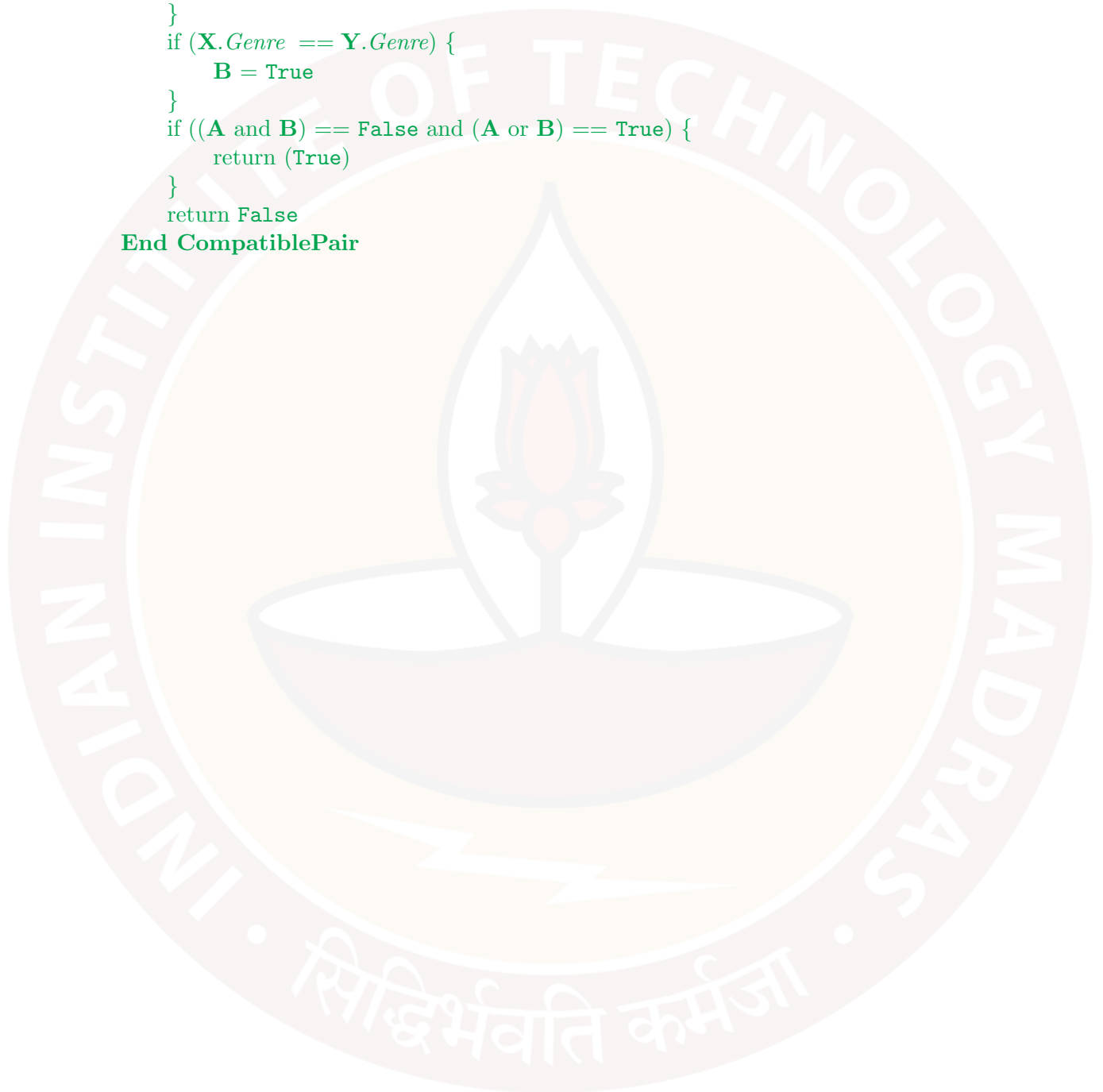
- by combining above two conditions, we capture the condition that either **A** or **B** is **True** but not both:

(A or B) == True and (A and B) == False

For the above condition, the procedure **CompatiblePair** returns **True**. That is,

```
if ((A or B) == True and (A and B) == False) {  
    return True  
}
```

Otherwise, the procedure returns **False**. By combine all the above piece of codes, we get the following procedure implementation.

Procedure CompatiblePair (X, Y)**A = False, B = False, C = False****if (X.Year == Y.Year) {****A = True****}****if (X.Genre == Y.Genre) {****B = True****}****if ((A and B) == False and (A or B) == True) {****return (True)****}****return False****End CompatiblePair**

BSCCS1001: Week-5, Practice Assignment

1. The following pseudocode is executed on the “Scores” table. What does the value **A** represent at the end of the execution? It is a Multiple Select Questions (MSQ).

```
S = [ ]
while (Table 1 has more rows) {
    Read the first row X in Table 1
    if (X.CityTown == “Madurai” or X.Gender == “F”) {
        S = S ++ [X.SeqNo]
    }
    Move X to Table 2
}
A = length(S)
```

- ☐ Number of female students from Madurai
- ☐ (Number of female students) + (Number of students from Madurai)
- ☒ (Number of female students) + (Number of students from Madurai) - (Number of female students from Madurai)
- ☒ (Number of female students) + (Number of male students from Madurai)
- ☒ (Number of female students not from Madurai) + (Number of students from Madurai)

Solution: The first line in the pseudocode initializes the variable **S** with an empty list. Then, we enter a while loop that goes through each row in the “Scores” table. Inside the while loop, there is an if-statement that is composed of two conditions:

- Is the student from Madurai?
- Is the student a female?

Since these two conditions are connected by “or”, the if-statement will evaluate to true if at least one of these two conditions evaluates to true. Inside the if-block, we are appending the student to the list **S** whenever the if-statement evaluates to true. Finally, the variable **A** stores the number of elements in the list **S**.

The question is asking us to find what the variable **A** represents. Since **A** is the number of elements in list **S**, we need to find what kind of students are being added to **S**. Let us go ahead and figure that out.

Consider the following table:

	Madurai	Not Madurai
Male	mM	mN
Female	fM	fN

We want all students who are from Madurai or who are females. This gives us the following expression:

$$mM + fM + fN$$

The above expression can be grouped in several ways. Each grouping gives us a different way of looking at the same expression:

- $(mM + fM) + fN$: This is the number of students from Madurai + number of females not from Madurai
- $mM + (fM + fN)$: This is the number of female students + the number of male students from Madurai

The above two groupings correspond to the last two options in the question, and are therefore correct. Now, we take up the first grouping:

$$(mM + fM) + fN$$

If we add and subtract fM , we get the following grouping:

$$(mM + fM) + (fN + fM) - fM$$

This is nothing but the following expression:

(number of students from Madurai) + (number of female students) - (number of female students from Madurai)

The above expression corresponds to the third option in the question. Another way of understanding this is, among the students from Madurai, some are females; among the females, some are from Madurai. We are double counting the number of females from Madurai, so we subtract it once.

We have seen three different ways of expressing the same thing. Approaching a problem from multiple perspectives helps in broadening our understanding.

2. **topMath** and **topPhy** are lists of cards having Mathematics marks and Physics marks greater than 80 respectively. Each element in both lists is a record. The fields in the record are the same as the ones in the header of the “Scores” table. What will the value of the list **someList** represent at the end of the execution of the pseudocode given below?

```
someList = [ ]
foreach X in topMath {
    foreach Y in topPhy {
        if (X.SeqNo == Y.SeqNo and X.Chemistry > 80) {
            someList = someList ++ [X.Name]
        }
    }
}
```

- ☐ It stores the names of students who have scored above 80 in Chemistry.
- ☐ It stores the names of students who have scored above 80 in both Mathematics and Physics.
- ☐ It stores the names of students who have scored above 80 in at least one subject.
- ✓ It stores the names of students who have scored above 80 in all three subjects.

Solution: The variable **someList** is initialized to an empty list in the first line. Next, we have a nested iteration. The outer loop goes through all elements in the list **topMath**. For each element **X** in **topMath**, the inner loop **topPhy** goes through each element **Y** in **topPhy**. Inside the nested iteration, we have an if-statement that has the following conditions:

- Do **X** and **Y** refer to the same students?
- Has **X** scored more than 80 in Chemistry?

Since these two conditions are connected by “and”, both these conditions must evaluate to true for statement inside the if-block to be evaluated. So, inside the if-block, we have **someList** storing the names of students present in both the lists, **topMath** and **topPhy**, who have scored above 80 in Chemistry. Since the students in the list **topMath** and **topPhysics** have scored above 80 in Maths and Physics respectively, the students in **someList** have scored above 80 in all three subjects.

3. We call two sentences *similar* if both of them have the same number of words and satisfy the following condition:

The i^{th} word in the first sentence has the same part of speech as the i^{th} word in the second sentence, for $1 \leq i \leq L$, where L is the total number of words in either sentence.

aList and **bList** are lists that contain the part of speech of words in two sentences **A** and **B** respectively. **isSimilar** is a procedure that accepts these two lists as parameters and checks for the similarity of **A** and **B**. Select the correct code fragment to complete the pseudocode.

```
Procedure isSimilar(aList, bList)
    if (length(aList)  $\neq$  length(bList)) {
        return (False)
    }
    cList = bList
    *****
    *           Fill the code           *
    *****
End isSimilar
```

- ☐ foreach **x** in **aList** {
 if (**x** == first(cList)) {
 return (True)
 }
 cList = init(cList)
}
- ☐ foreach **x** in **aList** {
 if (**x** $\neq$ first(cList)) {
 return (False)
 }
 cList = init(cList)
}

```

○ foreach x in aList {
    if (x ≠ first(cList)) {
        return (False)
    }
    cList = rest(cList)
}

```

```

○ foreach x in aList {
    if (x ≠ first(cList)) {
        return (False)
    }
    cList = rest(cList)
}
return (False)

```

```

✓ foreach x in aList {
    if (x ≠ first(cList)) {
        return (False)
    }
    cList = rest(cList)
}
return (True)

```

Solution: Let us first study the procedure and see the lines of code already given to us. The first few lines are checking if the two lists are of the same size. If not, the procedure returns false. It becomes clear that the rest of the code below the if-block is executed if and only if the two lists are of the same length. Moving forward, we can safely assume that the two lists are of the same length. The rest of the code should check if the two lists **aList** and **bList** are equal. This is because, two sentences **A** and **B** are similar if and only if the corresponding part of speech lists are equal.

We now see a new list, **cList**, that is a copy of **bList**. From looking at the options, we can see that the rest of the code manipulates the two lists, **aList** and **cList**. We need to check if the i^{th} element of **aList** is the same as the i^{th} element of **bList**. Notice that **bList** doesn't appear anywhere in the solution. The reason for this will become clear soon.

Now, we need to iterate through each element **x** in **aList** and check if the corresponding element in **bList** is equal to **x**. How do we go about comparing the i^{th} element of two lists? Let us take an example. Assume that we have two lists:

aList = ["noun", "verb", "article", "adjective"]

bList = ["noun", "verb", "article", "adjective"]

Let us now iterate through **aList**. We take the first element in **aList** and call it **x**. That is, we have **x** = “noun”. How do we check if this is equal to the first element in **bList**? We can do the following, is **x** == **first(bList)**? If the answer is true, then we can go to the next element in **aList**. In the example we have taken up, this is true, so let us move to the second element in **aList**.

Now, we have **x** = “verb”. How do we get the second element of **bList**? Here, we have an important observation: for comparing the second element of two lists, we no longer need the first element in both the lists. Since we are iterating through **aList**, we have naturally moved onto the second element. How do we get rid of the first element in **bList**? We can reassign it as follows: **bList** = **rest(bList)**. But this reassignment changes **bList**. This is a side-effect that we don’t want. To avoid this, we create a copy of **bList** and call it **cList** and perform the following operation: **cList** = **rest(cList)**.

Extending this argument, to compare the i^{th} elements of two lists, we do not need the first $i - 1$ elements. In every step of the iteration, we keep eliminating the first element in **cList**, while retaining the rest of the elements as they are. The following pseudocode expresses this idea:

```
foreach x in aList {  
    if (x ≠ first(cList)) {  
        return (False)  
    }  
    cList = rest(cList)  
}  
return (True)
```

We iterate through each element **x** in **aList**. We check if the first element in **cList** is equal to **x**. If not, then the two lists are not equal and we return False right away. The procedure terminates then and there and the rest of the steps in the iteration are not executed. If they are equal, then we need to move onto the second element in **cList**. We do this by reassigning **cList** to **rest(cList)**, which gets rid of the first element.

Now, when we go for the second step of the iteration, **x** becomes the second element in **aList**. The first element in **cList** is nothing but the second element in **bList**. In the next step of the iteration, the first element of **cList** will be the third element of **bList**. We keep doing this until all elements have been checked. After coming out of the loop, we see that every element in **aList** is equal to the corresponding element in **bList**. This means that the two lists are equal, and therefore we return True.

4. For a given train, we have a list called **days** that contains the sequence of entries contained in the column *Day* in the “Trains” dataset. As an example, for the train “12281”, **days** will be the following list of integers: [1, 1, 1, 1, 2, 2].

Madhuram is a passenger who boards this train at its starting station and gets down at its ending station. He has a peculiar habit of treating himself to exactly one sweet at midnight whenever he is on a train. Which of the following expressions gives the number of “midnight sweets” that he consumes during the course of his journey? It is a Multiple Select Question (MSQ).

- ☒ **last(days) - first(days)**
- ☐ **last(days) - first(days) + 1**
- ☐ **last(init(days)) - first(rest(days))**
- ☐ **length(days)**
- ☒ **last(days) - 1**
- ☐ **length(days) - 1**

Solution: Let us take one of the trains from the dataset and look at the list **days**. Let us also consider the corresponding arrival and departure times for the same train. For the train 12281, we have:

days = [1, 1, 1, 1, 2, 2]

arrival = [“None”, “10:44”, “14:56”, “20:45”, “00:50”, “06:25”]

departure = [“07:40”, “10:49”, “15:00”, “20:56”, “00:55”, “None”]

We use “None” if there is no entry in the dataset corresponding to that field. The first four entries in **days** are all 1. This means that all four entries correspond to the same day. We can verify that this is the case using the other two lists.

The last two entries in **days** are 2. This corresponds to the second day of travel. Again, the other two lists help us see that this is the case. We can see that this transition from day 1 to day 2 happens at midnight.

Therefore, the number of midnight sweets that Madhuram eats will be equal to the number of such transitions in **days**. If a train reaches the last station on the third day of its journey, then there are two nights involved. Likewise, if a train completes the journey on the n^{th} day, then we have $n - 1$ nights. In general, we have:

last(days) - 1

Since the first element of **days** is always 1, we also have the following answer:

last(days) - first(days)

5. **trainList** is a non-empty list that contains information about all trains associated with a station. Each element in **trainList** is a list of running days for a train. For example, given a station that handles two trains, we could have `[["M", "W"], ["Sa"]]`. Assume that all the elements of **trainList** are non-empty lists. Consider the following pseudocode.

```
m = 0, tu = 0, w = 0, th = 0, fr = 0, sa = 0, su = 0
foreach x in trainList {
    foreach y in x {
        if (y == "M") {
            m = 1
        }
        if (y == "Tu") {
            tu = 1
        }
        if (y == "W") {
            w = 1
        }
        if (y == "Th") {
            th = 1
        }
        if (y == "F") {
            f = 1
        }
        if (y == "Sa") {
            sa = 1
        }
        if (y == "Su") {
            su = 1
        }
    }
}
```

Solution: Let us take a look at the code and try to understand it before going to the questions. Though it looks lengthy, there are some repetitive elements in it. The first line in the code initializes seven variables, one for each day of the week, to zero.

This is followed by a nested iteration. The outer loop goes through each element in **trainList**. Each element in **trainList** is nothing but a list of days. The inner loop goes through each of these days. Inside the nested iteration, we have seven if-blocks. Each block corresponds to one day of the week, and checks if the element **y** is equal to that particular day.

As an example, let us take the case where **y** = “W”. The third if-condition evaluates to true, and the variable **w** is set to one. Notice that, henceforth, the variable **w** always holds the value one and is never reset to zero. At the end of execution, if a variable (say **tu** is zero), that means no train stops at the station on that day. If the variable is one, then some train stops at the station on that day. In other words, each of the variables **m**, **tu**, $\dots$, **su** tells us if the station is busy on that day.

The above pseudocode is executed on the **trainList** for some arbitrary station. After execution, we run the following code.

```
someVar = 0
someVar = 7 - (m + tu + w + th + f + sa + su)
```

- (a) What does the variable **someVar** store at the end of execution?
- ☐ It stores the number of days in a week when at least one train visits the station.
 - ☒ It stores the number of days in a week when no train visits the station.
 - ☐ It stores the number of trains that visit the station in a week.
 - ☐ It does not store anything meaningful.

Solution: We already know that each of the variables **m**, **tu**, $\dots$, **su** holds the value one if a train stops at the station and zero otherwise. So, the sum of these seven variables gives us the number of days on which the station is busy. We have the following expression:

someVar = 7 - (**m** + **tu** + **w** + **th** + **f** + **sa** + **su**)

This is nothing but the number of days on which the station is not busy. In other words, it is the number of days on which no train visits the station.

(b) Which of the following is an accurate bound for the variable **someVar**?

- ☐ $0 \leq \text{someVar} \leq 7$
- ☐ $0 < \text{someVar} \leq 7$
- ☒ $0 \leq \text{someVar} < 7$
- ☐ $0 < \text{someVar} < 7$

Solution: In the question, we are given the fact that **trainList** is non-empty. Additionally, we are also given that each element of **trainList** is non-empty. Consider the main pseudocode. Each time we go through the inner loop of the nested iteration, at least one of the seven variables will be set to one. This is because none of the inner lists are empty. At the extreme case, we could have all seven variables set to one at the end of execution.

Now, we come to the code that computes **someVar**. Before that, let us first consider the sum of the seven variables:

$$\mathbf{m + tu + w + th + f + sa + su}$$

It is already clear that all seven variables cannot be zero. This means that the expression given above is at least one. At the extreme case, we could have all seven variables equal to one. This leaves us with the following inequality:

$$1 \leq \mathbf{m + tu + w + th + f + sa + su} \leq 7$$

From this it follows that:

$$0 \leq 7 - (\mathbf{m + tu + w + th + f + sa + su}) < 7$$

Therefore, we have:

$$0 \leq \mathbf{someVar} < 7$$

BSCCS1001: Week-5, Graded Assignment

1. (2 points) In the “Scores” table, let **S** be the list of sequence numbers of rows which have marks greater than 75 in at least one subject. What does the variable **count** represent at the end of execution of the pseudocode given below?

```
count = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    foreach c in S {
        if (X.SeqNo == c) {
            if (X.Mathematics < 75 and X.Physics < 75) {
                count = count + 1
            }
        }
    }
    Move X to Table 2
}
```

- ☐ Number of students who have scored less than 75 in both Mathematics and Physics
- ☐ Number of students who have scored more than 75 in Chemistry
- ☒ Number of students who have scored less than 75 in both Mathematics and Physics, but more than 75 in Chemistry
- ☐ Number of students who have scored less than 75 in all three subjects

Solution: There is a nested iteration in the code. The interesting aspect is that the outer loop is a “while” and the inner loop is a “foreach”. The outer loop is going over the rows in the “Scores” table. For each row in the table, the inner loop is going over the elements in the list **S**.

Inside the inner loop, we have two if-conditions that are nested. The outer if-condition checks if the row **X** from the table and element **c** from the list corresponds to the same student. If yes, then we move on and check if the mathematics and physics marks of this student is less than 75. If that is true, then **count** is incremented. But note that we are looking at elements in the list **S**. These elements are the sequence numbers of students who have scored greater than 75 in at least one subject.

So, we have the following observations:

- The student has scored greater than 75 in at least one subject. This constraint comes from the sequence number being in the list **S**.
- The student has scored less than 75 in both mathematics and physics. This comes from the inner if-condition.

Since these two conditions are nested, we have to look at those students who satisfy both the conditions. If we take the second condition first, then we are looking at students who have scored less than 75 in both mathematics and physics. Now we bring in the first condition. The only way a student can satisfy this condition is if he has scored greater than 75 in chemistry.

Putting things together, **count** represents the number of students who have scored below 75 in both mathematics and physics, but above 75 in chemistry.

2. (2 points) The following pseudocode is executed on the “Words” table. Assume that the rows in Table 1 are sorted in the increasing order of sequence number. What does the list **L** contain at the end of execution?

```
L = [ ]
A = “None”
Read the first row X in Table 1
A = X.PartOfSpeech
Move X to Table 2
while (Table 1 has more rows) {
    Read the first row X in Table 1
    if (X.PartOfSpeech == “Noun”) {
        if (A == “Adjective”) {
            L = L ++ [X.Word]
        }
    }
    A = X.PartOfSpeech
    Move X to Table 2
}
```

- ☒ The list of nouns that come immediately after an adjective.
- ☐ The list of adjectives that come immediately after a noun.
- ☐ The list of nouns that come immediately before an adjective.
- ☐ The list of adjectives that come immediately before a noun.

Solution: Let us look at the first few lines. List **L** is initialized to the empty list and variable **A** is initialized to “None”. At first sight, the next line about reading the first row from table 1, looks a bit out of place. So far, we have only encountered this particular line inside a loop. But nothing stops us from reading the first row in the table outside a loop. As the rows have been sorted in the increasing order of sequence number, we have the variable **A** store the part of speech of the first word in the paragraph.

Now, we enter the while loop. There is a nested if-condition. The outer if-condition checks if the part of speech of the current word is a noun. Whenever that is true, we move on to the inner if-condition, which checks if the variable **A** stores the value “Adjective”. If that is true, then the current word is appended to the list **L**.

It becomes clear that we are appending nouns to the list **L**. This is because the outer if-condition stipulates that the part of speech of **X** should be “Noun”. But we are not adding all nouns to the list **L**. To see when we append a noun to the list, we have to understand the role played by the variable **A**.

Once we get out of the nested if-block, variable **A** is assigned the part of speech of the current word. So, in the next step of the iteration, the variable **A** will be storing the part of speech of the previous word.

Let us summarize and look at the two conditions that have to be satisfied if the current word has to be appended to the list **L**:

- The part of speech of the current word is “Noun”.
- The part of speech of the previous word is “Adjective”.

Therefore, the list **L** stores all nouns that come immediately after an adjective. Let us now highlight a few features of this code that are worth noting:

- Notice that the while loop starts iterating from the second row in the table. Why not start from the first row as usual? We start from the second row because the first word in the paragraph has no previous word. So, it only makes sense to start the iteration from the second row in the table.
- Why do we assign the part of speech of the first word in the paragraph to **A** before the start of the iteration? In the first step of the iteration, we are looking at the second row of the table. At this stage, we wish to know if the previous word (first word in this case) is an adjective or not. This is the reason we make this assignment.
- Why do we assign the part of speech of the current word to **A** just before the end the while loop? This is done so that the variable **A** can store the part of speech of the previous word in the next step of the iteration.

3. (2 points) We have a non-empty list called **authList** that stores the names of all authors in the “Library” table sorted in alphabetical order of author names. There is one element corresponding to each book in the table. This results in many duplicates. The following procedure attempts to extract the unique list of authors, while preserving the sorted order. The pseudocode may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ).

```
1  uniqueList = [ ]
2  uniqueList = uniqueList ++ first(authList)
3  prev = first(authList)
4  foreach x in rest(authList) {
5      if (x == prev) {
6          uniqueList = uniqueList ++ [x]
7      }
8      prev = x
9  }
```

- ☐ Error in line 1
- ☒ Error in line 2
- ☐ Error in line 3
- ☒ Error in line 5
- ☐ Error in line 8
- ☐ No error

Solution: A good approach for problems of this kind is to quickly go through the entire code once and get an overall understanding of it. We can then pause at each line and see if it has any errors.

In the first line, **uniqueList** is initialized to an empty list. The name of the variable is very suggestive. This is probably going to be the list that stores the list of unique authors. Going down, we see that we are looping through all the elements in **rest(authList)** and adding elements to **uniqueList** whenever some condition is satisfied.

Let us go line by line. There is no error in line 1. In line 2, the intention is to add the first element in **authList** to **uniqueList**. There is an error in the way it is done. Only two lists can be combined with the “++” operation, whereas a list and a string are being combined here. The correct code is as follows:

```
uniqueList = uniqueList ++ [first(authList)]
```

There is no obvious error in the next line. The variable **prev** is storing the first element in the list **authList**. Again, the variable naming is suggestive. **prev** probably stands for previous. So, the **prev** is in all likelihood storing the previous element.

There is no error in line 4. That is clear from the options. We are iterating through all elements in **authList** except the first. This is because of the term **rest(authList)**.

Inside the foreach loop, we have an if-condition. This if-condition is checking if the previous element is equal to the current element. If they are equal, then this element is added to **uniqueList**.

We know that **authList** is a list of author names sorted in alphabetical order of author names. If there are duplicates, then they will appear as adjacent elements in the list. For example, consider the following sorted list:

["Kalam", "Kalam", "Narayan", "Roy", "Roy"]

According to the current version of the code, whenever the previous name matches the current name, we are adding it to **uniqueList**. This is exactly the behavior that we don't want to see. Note that we start by adding the first element in **authList** to **uniqueList**. From then on, we only add the current element if it is not the same as the previous element. Therefore, there is an error in line number 6. The corrected version should be as follows:

if (**x** $\neq$ **prev**)

Finally, in line 8, the variable **prev** is being updated to the current author name. There is no error in this line. This assignment statement is similar to the one that we saw in the previous question. This is done so that in the next step of the iteration, the variable **prev** will store the previous element in the list.

4. (3 points) The procedure **visitedShop** returns the list of names of people who have visited a particular shop in the “Shopping Bills” dataset. It accepts the shop’s name as a parameter. Additionally, each customer must be represented exactly once in the returned list. The following pseudocode may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ).

```
1  Procedure visitedShop(shop)
2      S = “None”
3      while (Pile 1 has more cards) {
4          Read the top card X from Pile 1
5          if (X.ShopName == shop) {
6              if (not(member(S, X.CustomerName))) {
7                  S = S ++ [X.ShopName]
8              }
9          }
10         Move X to Pile 2
11     }
12     return (S)
13 End visitedShop

14 Procedure member(L, name)
15     present = True
16     foreach x in L {
17         if (x == name) {
18             present = True
19             exitloop
20         }
21     }
22     return (present)
23 End member
```

- ✓ Error in line 2
- ☐ Error in line 6
- ✓ Error in line 7
- ✓ Error in line 15
- ☐ Error in line 17
- ☐ Error in line 18
- ☐ No error

Solution: First, a quick look at the code. The while loop in the procedure **visitedShop** goes through all cards in the “Shopping bills” dataset. Inside, there is a nested if-block. The outer if-condition checks if the shop name of the current card matches the input parameter. If it is true, then the inner if-condition checks for some other condition that involves the procedure **member**. If this condition evaluates to True, we go ahead and add the shop name to **S**.

Let us now look at the procedure **member**. This procedure should be familiar as we have already seen it in the lectures and tutorials. It accepts two parameters. The first parameter is a list **L** and the second parameter is some variable called **name**. It is supposed to return True if the element **name** is in the list and False otherwise.

With this background, let us go through the code line by line. In line 2, we see that **S** has been initialized to “None”. This is an error. This is because **S** is the variable that is being returned by **visitedShop** and this procedure is supposed to return a list. The correct version of line 2 should be:

S = []

Lines 3 and 4 are error free. Line 5 doesn’t have any error either. This procedure is concerned with those people who have visited a particular shop. So, the if-condition in line-5 is checking for exactly that.

Line 6 is a bit involved. So, let us unpack it. **member(S, X.CustomerName)** returns the value True if the customer’s name is present in the list. This means that **not(member(S, X.CustomerName))** returns False if the customer is present in the list. For now, let us suspend our judgement regarding line 6 and move on.

In line 7, we are adding the shop name to the list if the customer’s name is not present in the list. Let us go back to the question statement and see if this is what we expect. We wish to find the list of names of people who have visited the shop. But, if someone has visited it multiple times, then we want only one entry corresponding to this person in the list **S**. This is the reason we are checking if **X.CustomerName** is present in the list. We should add a name only if it is not already present in **S**. In other words, only if **not(member(S, X.CustomerName))** evaluates to True.

However, there is a subtle error in line 7. Instead of appending the customer's name to the list, we are appending the shop's name. So the correct version of line 7 is:

S = **S** ++ [**X**.*CustomerName*]

There are no more errors in the procedure **visitedShop**. That leaves us with the procedure **member**. Line 15 looks wrong. Initially, we don't know if **name** is present in the list or not. So, it is best to initialize **present** to False. Let us go through the remaining code to see if this is indeed an error.

Line 16 looks fine. In line 17, we see that if **name** is already present in the list **L**, we set the variable **present** to True and then exit the loop. This is the expected behavior, which means that there is nothing wrong with lines 16-21.

In line 22, we return **present**. No error there as well. Now we go back to line 15. It becomes clear that there is an error in this line. **present** should be False to begin with. If, we find **name** in the list **L**, then we update **present** to True, if not we let it remain as it is.

In summary, lines 2, 7 and 15 have errors in them.

5. (4 points) Let **marks** be a non-empty list of physics marks sorted in ascending order. **mostFrequent** is a procedure that accepts a sorted list as a parameter and returns the value which occurs the most number of times in it. For example, if the list is [80, 80, 80, 80, 90, 90, 100], then 80 is returned. If multiple elements satisfy the above condition, the smallest among them is returned. Select the correct code fragment to complete the pseudocode.

```
Procedure mostFrequent(marks)
  prev = first(marks)
  mfreq = prev
  count = 1, max = 1
  foreach x in rest(marks) {
    if (x == prev) {
      *****
      *           Fill the code           *
      *****
    }
    else {
      count = 1
    }
    prev = x
  }
  return (mfreq)
End mostFrequent
```

- ✓ `count = count + 1`
 `if (count > max) {`
 `max = count`
 `mfreq = x`
 `}`
- `if (count ≥ max) {`
 `max = count`
 `mfreq = prev`
 `}`
- `count = count + 1`
 `if (count > max) {`
 `max = count`
 `mfreq = prev`
 `count = 0`
 `}`
- `if (count ≥ max) {`
 `max = count`
 `mfreq = x`
 `count = 0`
 `}`

Solution: Let us look at the code that is already available to us. The variable **prev** stores the first element in the list **marks**. As before, the naming is suggestive and **prev** is mostly referring to the previous element. **mfreq** is initialized to the first element in **marks**. Since **mfreq** is the variable that is being returned by the procedure, that must be the one used to keep track of the most frequent element in the list. **count** and **max** are then initialized to 1.

Next, we have a foreach loop that goes through the elements of **rest(marks)**. This means, we will be starting from the second element of **marks** and going all the way up to the last element. Inside the loop, we check if the current element is equal to the previous one. This is the code-block we have to fill. If the current element is not equal to the previous element, we enter the else block. In the else-block, **count** is being updated to 1.

At this stage, let us go back to the question. We have one additional piece of information that we have not used. This is the fact that the list **marks** is sorted in the ascending order. Consider the example given in the question:

[80, 80, 80, 80, 90, 100]

As the list is sorted, duplicates will occur adjacent to each other. This gives us a way to compute the most frequent element. In every step of the iteration, we check if the current element **x** is equal to the previous element **prev**. If True, then we increment **count**. At this stage, we do the following thing.

- Check if the frequency of the current element is greater than **max**.
- If it is greater, then we update **max** and also the value of **mfreq**.

This idea is captured in the following pseudocode:

```
count = count + 1
if (count > max) {
    max = count
    mfreq = prev
}
```

6. (4 = 2 + 2 points) The following pseudocode is executed on the “Words” table. Assume that the rows in the table are sorted in the ascending order of sequence number.

```
start = True
outList = [ ]
while (Table 1 has more rows) {
    Read the first row X in Table 1
    if (start) {
        inList = [ ]
        start = False
    }
    inList = inList ++ [ [X.Word, X.PartOfSpeech] ]
    if (X.Word ends with a full stop) {
        outList = outList ++ [inList]
        start = True
    }
    Move X to Table 2
}
```

- (a) At the end of execution, which of the following statements are true about the variable **outList**? It is a Multiple Select Question (MSQ).
- ☒ Each element in **outList** corresponds to one sentence from the paragraph.
 - ☐ Each element in **outList** corresponds to one word from the paragraph.
 - ☐ Each element in **outList** corresponds to one character from the paragraph.
 - ☒ **outList** is a list of lists.

Solution: Consider the first if-condition inside the while loop. In the first step of the iteration, the variable **start** is already True. So, the if-block is executed. Coming out of this if-block, **start** holds the value False and **inList** is initialized to an empty list.

Moving on, the current word (in this case, the first word) and its part of speech are added as an element to **inList**. Note that this element is itself a list consisting of two elements. **inList** will now look like this:

[[“It”, “Preposition”]]

As the word doesn’t end with a full stop, the second if-block is not executed. In the next step of the iteration, **start** is False, so the first if-block is not executed.

The second word and its part of speech are now added to the list **inList**. So, **inList** now looks as follows:

[[“It”, “Preposition”], [“was”, “Verb”]]

As this word also doesn’t end with a full stop, we move on to the next step. We keep doing this until we reach a word that ends with a full stop. At this point, the second if-block is executed. Here, the list **inList** is added to **outList** and the variable **start** is set to True. So, the first element of **outList** is a list and corresponds to the first sentence in the paragraph.

At the next step, the first if-block is going to get executed because the current value of the variable **start** is True. After coming out of this if-block, **inList** is again empty and **start** holds the value False.

The pattern would have become clear by now. **inList** will start holding the contents of the second sentence. Once we reach the end of the second sentence, **inList** will be added to **outList**.

So, each element of **outList** corresponds to one sentence form the paragraph. Each element of **outList** is itself a list. This means that **outList** is a list of lists.

- (b) Which of the following statements are true about the variable **start** during the course of the execution of the above pseudocode? It is a Multiple Select Question (MSQ).

- ☐ The variable **start** is set to False at the last word of every sentence.
- ✓ ☒ The variable **start** is set to True at the last word of every sentence.
- ✓ ☒ The variable **start** is set to False at the first word of every sentence.
- ☐ The variable **start** is set to True at the first word of every sentence.

Solution:

start is set to True to begin with. After seeing the first word of the paragraph, which also happens to be the first word of the first sentence, **start** is set to False. Every time we come to the last word of a sentence, **start** is set to True. The next step of the iteration is nothing but the first word of the next sentence. After processing this, **start** is again set to False. From this it becomes clear that **start** is set to True at the last word of every sentence and is set to False at the first word of every sentence.

7. (7 = 2 + 2 + 3 points) We take the variable **outList** obtained at the end of execution of the pseudocode in the **previous question**. Now, we execute the following pseudocode.

```
someVar = 0, count = 0, i = 0
foreach x in outList {
    someVar = someVar + length(x)
    foreach y in x {
        if (last(y) == "Noun") {
            count = count + 1
        }
    }
    i = i + 1
}
```

- (a) What does the variable **count** store at the end of execution?
- ✓ It is the number of nouns in the paragraph.
 - ☐ It is the number of nouns in the first sentence of the paragraph.
 - ☐ It is the number of sentences in the paragraph that begin with a noun.
 - ☐ None of the above.

Solution: We have a nested iteration. The outer loop goes through each element **x** in **outList**. **x** corresponds to a sentence in the paragraph. Each element **y** in **x** corresponds to a word in the sentence. **y** itself is a list of two elements, whose first element is the word and last element is its part of speech.

The inner loop iterates through the list **x**. The if-condition inside the inner loop is checking if a word is a noun. Note that **y** itself is a list, whose last (second in this case) element is the part of speech of the word. So, **count** stores the number of nouns in the paragraph.

(b) What does the variable **someVar** store at the end of execution?

- ✓ It is the number of words in the paragraph.
- ☐ It is the number of sentences in the paragraph.
- ☐ It is the number of characters in the paragraph.
- ☐ None of the above.

Solution:

x corresponds to a sentence. **length(x)** is nothing but the number of words in the sentence represented by **x**. So, **someVar** is the sum of the number of words across all sentences. Therefore, it will represent the number of words in the paragraph.

(c) Which of the following statements are true about the variable **i** at the end of execution? It is a Multiple Select Question.

- ☐ It is the number of words in the paragraph.
- ✓ It is the number of sentences in the paragraph.
- ✓ It is equal to the number of elements in **outList**
- ☐ It is equal to the number of elements in **inList**
- ✓ It is equal to **length(outList)**
- ☐ It is the number of characters in the paragraph.

Solution: **i** gets incremented every time we go through a sentence **x** in the paragraph. So, at the end of execution, **i** will store the number of sentences in the paragraph. This is the same as the number of elements in the list **outList**, which is the same as **length(outList)**.

8. (4 = 2 + 2 points) **stations** is a list that contains the sequence of stations visited by a train from the “Trains” dataset. Each element in **stations** is a list: [*Name*, *Distance*], the first entry is the name of the station, while the second entry is the distance of this station from the first station in the list.

minDist is a procedure that accepts **stations** as a parameter and returns the names of a pair of consecutive stations which have the shortest distance between them on this route. Complete the following procedure.

```
Procedure minDist(stations)
  pair = ["None", "None"]
  min = 10000
  prev = first(stations)
  foreach x in rest(stations) {
    diff = last(x) - last(prev)
    *****
    *           Fill the code           *
    *****

    prev = x
  }
  return (pair)
End minDist
```

- (a) There may be multiple pairs having the same minimum distance. If we wish to find a pair of stations closest to the first station in the list, which of the following is the correct code fragment?

- ☒ if (**diff** < **min**) {
 min = **diff**
 pair = [**first**(**prev**), **first**(**x**)]
}
- ☐ if (**diff** ≤ **min**) {
 min = **diff**
 pair = [**first**(**prev**), **first**(**x**)]
}

- if (**diff** < **min**) {
 min = **diff**
 pair = [**last**(**prev**), **last**(**x**)]
 }
- if (**diff** ≤ **min**) {
 min = **diff**
 pair = [**last**(**prev**), **last**(**x**)]
 }

Solution:

pair is the list returned by this procedure. It is initialized to ["None", "None"] as it will be storing the names of two stations, both of which are strings. As we are finding the minimum distance, **min** is initialized to 10000. In the context of distances between stations, 10000 is a very high value. So this initialization is reasonable.

Similar to the questions that we have seen before, we have a variable **prev** that stores the information of the previous element in the list. We initialize it with the first element in the list and start the iteration from the second element.

Now, we loop through each station in **rest(stations)**. The variable **diff** captures the distance between the previous station and the current station. The code that we need to complete should update the variable **min** and **pair** whenever we come across a pair of stations that have a shorter distance between them than the current minimum.

The correct code fragment is as follows:

```
if (diff < min) {
    min = diff
    pair = [first(prev), first(x)]
}
```

We will try to understand why this is correct. Let us see what happens if there are a pair of stations that have the same minimum distance. Consider the following example:

[["stn-1", 100], ["stn-2", 120], ..., ["stn-7", 1000], ["stn-8", 1020], ...]

Assume that the minimum distance is 20 and there are two pairs of stations having this distance:

["stn-1", "stn-2"] and ["stn-7", "stn-8"]

Our code is supposed to return ["stn-1", "stn-2"] as this is the pair that is closest to the first station on the list. After processing the element ["stn-2", 120] in the loop, **diff** will hold the value 20, **min** would be set to 20 and **pair** will be ["stn-1", "stn-2"]. When we come to the element ["stn-8", 1020], **diff** will again be 20. Now look at the if-condition:

```
if (diff < min)
```

Note that we don't want **pair** to be updated at this stage. This behavior is guaranteed only if the symbol is $<$. Had it been $\leq$, **pair** would have been updated with the value ["stn-7", "stn-8"]. This is because $20 \leq 20$ will evaluate to true.

- (b) There may be multiple pairs having the same minimum distance. If we wish to find a pair of stations closest to the last station in the list, which of the following is the correct code fragment?

☐ if (**diff** < **min**) {
 min = **diff**
 pair = [**first**(**prev**), **first**(**x**)]
}

☒ if (**diff** $\leq$ **min**) {
 min = **diff**
 pair = [**first**(**prev**), **first**(**x**)]
}

☐ if (**diff** < **min**) {
 min = **diff**
 pair = [**last**(**prev**), **last**(**x**)]
}

☐ if (**diff** $\leq$ **min**) {
 min = **diff**
 pair = [**last**(**prev**), **last**(**x**)]
}

Solution:

The only difference here is that the if-condition should have the $\leq$ symbol:

```
if (diff  $\leq$  min)
```

This way, the pair of stations that is closest to the last element in the list **stations** will be updated. To summarize, if you have (**diff** < **min**), then the first occurrence of minimum will be stored in **pair** and all subsequent occurrences with the same minimum will be ignored. If you have (**diff** $\leq$ **min**), every time the same minimum value occurs, **pair** will be updated, so that when we exit the loop, the last such occurrence will be recorded in **pair**.

9. (4 points) **stns** is a list that contains information about the sequence of stations visited by a train. Specifically, each element in this list is itself a list: *[Arrival, Departure]*. The first element in **stns** corresponds to the starting station for the train and the last element corresponds to the ending station for the train.

waitTime(arr, dep) is a procedure that accepts the arrival time and departure time of a train at a given station as input and returns the waiting time at that station in minutes.

We wish to find the average waiting time across all intermediate stations where the train stops and store this result in variable **avg**. The pseudocode may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ).

```
1  total = 0, count = 0, avg = 0
2  foreach x in init(rest(stns)) {
3      total = total + waitTime(first(x), last(x))
4      count = count + 1
5  }
6  if (count ≠ 0) {
7      avg = total / count
8  }
```

- ☐ Error in line 2
- ☐ Error in line 3
- ☐ Error in line 4
- ☐ Error in line 6
- ☐ Error in line 7

✓ No error

Solution:

As this is a small piece of code, let us go through it line by line. Line 1 seems to be harmless. Line 2 looks quite complicated and needs some unpacking. Let us do it with an example. Consider the following list

stns = [[“None”, “10:50”], [“3:20”, “3:35”], [“6:00”, “6:02”], [“8:00”, “None”]]

The interesting observation about this list is that the first and last stations do not have any waiting time. This is because, the train is starting at the first station, and there is no arrival time for the first element. As the train arrives at the last station and completes its journey, there is no departure time for the last element in the list.

This means that we are interested in only the intermediate stations in the list, excluding the first and the last elements. Is there a way to do this? Yes, the procedures **rest** and **init** come to our rescue. To get all elements other than the first, we use:

```
rest(stns)
```

In the case of our example, this would give the following result:

```
[ ["3:20", "3:35"], ["6:00", "6:02"], ["8:00", "None"] ]
```

Now, to get all elements except the last element, we take the **init** of the list given above:

```
init(rest(stns))
```

This results in:

```
[ ["3:20", "3:35"], ["6:00", "6:02"] ]
```

Now, going back to the pseudocode, we see that there is nothing wrong with line 2. In lines 3 and 4, we are merely accumulating the total waiting time in **total** and counting the number of intermediate stations using **count**. So no problems in the foreach code-block.

Getting out, we check if **count** is non-zero. If true, we go ahead and compute the average. Again, no errors. But why is this check necessary? Consider a train which doesn't stop at any intermediate station. Its list would look like this:

```
stns = [ ["None", "3:35"], ["6:00", "None"] ]
```

In this case, the foreach block would not be executed as **init(rest(stns))** is an empty list. Why? This is because there are only two elements, the first and the last, both of which are ignored. Therefore, **count** and **avg** will be zero. This the reason we need to have a check if **count** is zero or not at line 6.

To summarize, this pseudocode has no errors!

10. (8 = 2 + 2 + 4 points) **trains** is a list that contains information about trains associated with a station **stn**. Specifically, each element in this list is another list: [*Arrival*, *Departure*]. If the arrival or departure time is empty, it is represented as “None”.

```
flag1 = False, flag2 = True
count = 0
foreach x in trains {
    if (first(x) == “None” or last(x) == “None”) {
        flag1 = True
    }
    else {
        count = count + 1
    }
}
if (count == length(trains)) {
    flag2 = False
}
```

- (a) Which of the following statements about the variable **flag1** is true at the end of execution of the above pseudocode?
- ☒ It is True if and only if **stn** is a starting or ending station for at least one train in the list
 - ☐ It is False if and only if **stn** is a starting or ending station for at least one train in the list
 - ☐ It is True if and only if **stn** is a starting station for one train and ending station for some other train in the list
 - ☐ It is False if and only if **stn** is a starting station for one train and ending station for some other train in the list

Solution: **flag1** is False to begin with. Then, we iterate through all the trains in the list **trains**. The only place where **flag1** is updated is inside the if-block within the foreach loop. Let us take a closer look at the if-condition:

first(x) == “None” or last(x) == “None”

flag1 will be updated to True if one of the two conditions evaluates to true. For the sake of argument, let **first(x)** be “None”. What does this mean? **first(x)** is the arrival time of some train at **stn**. If this arrival time is “None”, it means that **stn** is the starting station for this train. Since this train is starting from this station, it only has a departure time and no arrival time.

Likewise, if **last(x)** is “None”, then **stn** is the ending station for this train. **flag1** is set to true whenever one of these two conditions evaluates to true:

- **stn** is the starting station for a train
- **stn** is the ending station for a train

Therefore, at the end of execution, **flag1** is true if and only if **stn** is a starting station or ending station for at least one train in the list.

(b) What does the variable **count** represent at the end of execution of the above pseudocode?

- ☐ It is number of trains associated with **stn**.
- ☐ It is the number of trains for which **stn** is a starting station.
- ☐ It is the number of trains for which **stn** is an ending station.
- ☒ It is the number of trains for which **stn** is neither a starting nor an ending station.

Solution: **count** is incremented inside the else-block. We enter the else-block only if both the conditions inside the if-block evaluate to False. That is, **first(x)** is not “None” and **last(x)** is not “None”. This means, for the train corresponding to **x**, **stn** is neither a starting station nor an ending station. It is only an intermediate station where it stops. Therefore, **count** stores the number of trains for which **stn** is neither a starting nor an ending station.

- (c) At the end of execution of the code given above, what can be said about the values stored by the boolean variables **flag1** and **flag2**?

- ✓ **flag1** and **flag2** always store the same value.
- ☐ **flag1** and **flag2** always store opposite values.
- ☐ **flag1** always stores the value True.
- ☐ **flag2** always stores the value True.
- ☐ There is no relationship between these two variables.

Solution:

flag2 is set to True to begin with. It is updated if the last if-condition evaluates to True. When does that happen? It happens if **count** is equal to the number of elements in **trains**. We already know that **count** is storing the number of trains in the list for which **stn** is neither a starting nor an ending station. If **count** is equal to **length(trains)**, then for every train in the list, **stn** is neither a starting nor ending station. This means that **stn** is only an intermediate station for all trains in the list.

So, **flag2** becomes False if there is no train in the list for which **stn** is a starting or ending station. Consequently, **flag2** is True if there is at least one train in the list for which **stn** is a starting or ending station. But this is exactly the case with **flag1**!

Therefore, **flag1** and **flag2** always store the same value at the end of execution!

COMPUTATIONAL THINKING

Week - 6

Practice Assignment

1. The following pseudocode is executing using the “Scores” table. What will the value of **Z** represent at the end of the execution?

[3 marks]

```
D = { }
A = 0, C = 0
while (Table 1 has more rows){
    Read the first row X in Table 1
    A = A + X.Total
    C = C + 1
    if (isKey(D, X.CityTown)) {
        D[X.CityTown][“Total”] = D[X.CityTown][“Total”] + X.Total
        D[X.CityTown][“Count”] = D[X.CityTown][“Count”] + 1
    }
    else {
        D[X.CityTown] = { “Total”: X.Total, “Count”: 1 }
    }
    Move X to Table 2
}
Avg = A / C
Z = 0
foreach B in keys(D) {
    D[B][“Average”] = D[B][“Total”] / D[B][“Count”]
    if (D[B][“Average”] > Avg) {
        Z = Z + D[B][“Count”]
    }
}
```

- a. Number of students from the cities with the average total marks of the city is more than the average total marks of the dataset
 - b. Number of students from the cities with the average total marks of the city is less than the average total marks of the dataset
 - c. Number of cities with the average total marks less than the average total marks of the dataset
 - d. Number of cities with the average total marks more than the average total marks of the dataset
-

Solution: Using the `while ()` block, the code is iterating through each and every card in the scores dataset. Here we are storing the sum of `X.Total` in variable `A` and number of students in variable `C`. Also we are updating the nested dictionary `D`.

`D` is a nested dictionary, where, `D = { }`, is the outside dictionary with the name of cities as keys and `D[X.CityTown] = { }` is the inner dictionary with “Total” and “Count” as keys. Using the condition `if (isKey(D, X.CityTown)) {...} else {...}` we are updating the dictionary `D`. First we check whether the key `X.CityTown` is present in the dictionary:

- if the key is present then we add the `X.Total` in `D[X.CityTown][“Total”]` and increment `D[X.CityTown][“Count”]`
- if the key is not present then we create key value pair into the dictionary.

So in the dictionary for each city we are storing the sum of total of each city in the key “Total” and total number of students in each city in the key “Count”.

`Avg` stores the average of the total marks of the students in the dataset. Using `foreach` we are iterating through each keys (`CityTown`) in the dictionary, `D`, and we are adding one key, “Average”, to the innermost dictionary and storing the average of each city in `D[X.CityTown][“Average”]`. Then we check the condition `if (Average of students of each city > average of total students)` and when the condition is satisfied we add the number of students in the `CityTown` to variable `Z`.

Therefore `Z` contains the number of students from the cities with the average total marks of the city is more than the average total marks of the dataset

2. The following pseudocode is executed using the “Shopping Bills” dataset. The variable **D** is a dictionary that maintains the following information: the category of the item purchased by a customer. Choose the correct code fragment to complete the pseudocode. [3 Marks]

```

D = { }
while (Pile 1 has more cards) {
    Read the top card X in Pile 1
    *****
    *       Fill the code       *
    *****
    Move X to Pile 2
}

```

- a.
- ```

foreach A in X.ItemList {
 D[X.CustomerName][A] = True
}

```
- b.
- ```

if (isKey(D, X.CustomerName)) {
    D[X.CustomerName] = { }
}
foreach A in X.ItemList {
    D[X.CustomerName][A.Category] = True
}

```
- c.
- ```

if (not isKey(D, X.CustomerName)) {
 D[X.CustomerName] = { }
}
else {
 foreach A in X.ItemList {
 D[X.CustomerName][A] = True
 }
}

```
- d.
- ```

if (not isKey(D, X.CustomerName)) {
    D[X.CustomerName] = { }
}
foreach A in X.ItemList {
    D[X.CustomerName][A.Category] = True
}

```

Solution: The dictionary **D** should contain the following information: the category of the item purchased by a customer. So we need to maintain a nested dictionary with customer as keys in the outer dictionary and category of items as keys in the inner dictionary.

Outer dictionary is already initialised in statement 1 and a **while ()** loop is used to iterate along each and every card of the shopping bill dataset. In the code we need to initialise the

inner dictionary for each and every customer. Also each category in the itemlist of the card we need to update dictionary as $\mathbf{D}[\mathbf{X}.CustomerName][\mathbf{A}.Category] = \mathbf{True}$. So the correct block of code which satisfies the criteria of the code is

```
if (not isKey(D, X.CustomerName)) {  
    D[X.CustomerName] = { }  
}  
foreach A in X.ItemList {  
    D[X.CustomerName][A.Category] = True  
}
```

3. What does the following pseudocode compute when it is executed using the “Shopping bills” dataset. [3 Marks]

```
BB = { }
while (Pile 1 has more cards) {
    Read the top card X in Pile 1
    if (X.ShopName == “Big Bazaar”) {
        BB = UpdateDictionary(BB, X)
    }
    Move X to Pile 2
}
A = GetMaxKeyByKey(BB)

Procedure UpdateDictionary(D, Y)
    foreach A in Y.ItemList {
        if (isKey(D, A.Category)) {
            D[A.Category] = D[A.Category] + 1
        }
        else{
            D[A.Category] = 1
        }
    }
    return D
End UpdateDictionary

Procedure GetMaxKeyByKey(D)
    A = “None”, B = 0
    foreach Y in keys(D) {
        if (B < D[Y]) {
            A = Y
            B = D[Y]
        }
    }
    return A
End GetMaxKeyByKey
```

- Finds the item category from Big Bazaar
- Finds the top item category by item category count from Big Bazaar
- Finds the most frequent item category from Big Bazaar
- Finds the top item category by item quantity from Big Bazaar

Solution: The pseudocode contains a main code block and two procedures (UpdateDictionary and GetMaxKeyByKey).

A `while ( )` loop is used to iterate through each and every card of the shopping bill dataset. And the dictionary **BB** is updated using the procedure `UpdateDictionary(BB,X)` only when the **X.ShopName** is “BigBazaar”.

The parameters passed to the procedure, `UpdateDictionary`, are dictionary **BB** and card **X**. Inside the procedure `foreach` is used to iterate along the itemlist of card **Y** and condition, `if(isKey(D, A.Category) { } else { })` is used to update the dictionary. If the item category is already present in dictionary, the value **D[A.Category]** is incremented else the value **D[A.Category]** is assigned 1. Therefore in the procedure frequency of an item category in the card is added to the values of dictionary where the keys are item category.

So after `while ( )` loop, the dictionary **BB** contains the total frequency of each item category sold by the shop “Big Bazaar”. At last a procedure, `GetMaxKeyByKey`, is called to update the value in variable **A**.

The parameter passed to the procedure, `GetMaxKeyByKey`, is dictionary **BB**. Inside the procedure `foreach` is used to iterate through every keys in the dictionary. And `if (B < D[Y]) { }` condition is used to store the most frequent item category in variable **B** and the name of category in **A**.

Therefore pseudocode finds the most frequent item category from Big Bazaar.

4. The following pseudocode is executed using the “**station_wise**” cards of the “Train” dataset. What will the value of **C** represent at the end of the execution?

[3 Marks]

```

D = { “M”: 0, “Tu”: 0, “W”: 0, “Th”: 0, “F”: 0, “Sa”: 0, “Su”: 0 }
while (Pile 1 has more cards) {
    Read the top card X in Pile 1
    if (X.StationName == “Mumbai”) {
        foreach A in X.TrainList {
            foreach B in A.Days {
                D[B] = D[B] + 1
            }
        }
    }
    Move X to Pile 2
}
C = [], Y = 0
foreach B in keys(D) {
    if (Y == D[B]) {
        C = C ++ [B]
    }
    if (Y < D[B]) {
        C = [B]
        Y = D[B]
    }
}

```

- The busiest day of the Mumbai station
- List of busiest days of the Mumbai station
- The least busy day of the Mumbai station
- List of least busy days of the Mumbai station

Ans: (b)

Solution: Initially a dictionary **D** is initialised where the keys of the dictionary are days of a week and values of each key is also initialised to zero.

A **while ()** loop is used to iterate through each and every “station_wise” cards in the Train dataset. Statements inside the **if (X.StationName == “Mumbai”) { }** condition is executed only for the cards having station name Mumbai. A nested **foreach** is used to iterate along each row in the card Mumbai and along each days in the row. And the value of the dictionary, **D**[Day] is incremented when the key B is same as Day in **A.Days**. So after **while ()** loop, the dictionary contains the information of number of trains passing through Mumbai in each day.

In the second phase of the code **foreach** loop is used to iterate along each and every keys in the dictionary **D**. Inside the loop two if blocks are used. The second if block (**if (Y < D[B]) { }**) is used to find the key which stores the highest value also the key is stored in variable **C** as

a list. The first if block ((if (**Y** == **D[B]**) { })) is used to update the list **C** when there are more keys with same value.

So the **C** stores list of busiest days of the Mumbai station

5. The following pseudocode is executed using the “Words” table. At the end of the execution, **C** captures the number of words with either all letters are distinct, or at least three letters are appeared at least twice. Choose the correct code-fragment to complete the pseudocode. [3 Marks]

```

C = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    i = 1, D = { }
    while (i ≤ X.LetterCount) {
        B = ith letter in X.Word
        if (isKey(D, B)) {
            D[B] = D[B] + 1
        }
        else{
            D[B] = 1
        }
        i = i + 1
    }
    *****
    *      Fill the code      *
    *****
    Move X to Table 2
}

```

a.

```

Found = True, A = 0
foreach Y in keys(D) {
    if (D[Y] > 1) {
        A = A + 1
    }
    else{
        Found = False
    }
}
if (Found or A ≥ 3) {
    C = C + 1
}

```

b.

```
Found = True, A = 0
foreach Y in keys(D) {
    if (D[Y] > 1) {
        Found = False
        A = A + 1
    }
}
if (Found or A ≥ 3) {
    C = C + 1
}
```

c.

```
Found = True, A = 0
foreach Y in keys(D) {
    if (D[Y] > 1) {
        Found = False
        A = A + 1
    }
    else{
        Found = True
    }
}
if (Found or A ≥ 3) {
    C = C + 1
}
```

d.

```
Found = True, A = 0
foreach Y in keys(D) {
    if (D[Y] > 1) {
        Found = False
    }
    else{
        A = A + 1
    }
}
if (Found or A ≥ 3) {
    C = C + 1
}
```

Ans: (b)

Solution: Nested `while ()` loop is used in the pseudocode. The outer `while ()` loop is used to iterate along each and every word of the paragraph dataset and inner `while ()` loop is used to iterate along each letter in the word.

Dictionary **D** is initialised inside the outer while loop, there fore, for each word **X** a dictionary D is initialised. And inside inner while loop an `if (isKey(D, B)) { } else{ }` is used to store the frequency of letters in a in the dictionary **D**.

Here **C** captures the number of words with either all letters are distinct, or at least three letters are appeared at least twice.

For all letters of the word to be distinct, **D[B]** should be equal to 1 for all the letters. Here we use a variable Found which holds the value True when the condition is satisfied.

For at least three letters to be appear at least twice, we need to increment a variable (**A**) when **D[B] > 1** and needs to check whether **A ≥ 3**.

So the correct code block which completes the code is

```
Found = True, A = 0
foreach Y in keys(D) {
    if (D[Y] > 1) {
        Found = False
        A = A + 1
    }
}
if (Found or A ≥ 3) {
    C = C + 1
}
```

COMPUTATIONAL THINKING

Week - 6

Graded Assignment Solution

1. The following pseudocode is executed using the “Scores” table. What will the values of **A** and **B** represent at the end of execution? [3 Marks]

```
D = { }
while (Table 1 has more rows) {
    Read the first row X in Table 1
    if (isKey(D, X.CityTown)) {
        if (D[X.CityTown] > X.Mathematics) {
            D[X.CityTown] = X.Mathematics
        }
    }
    else {
        D[X.CityTown] = X.Mathematics
    }
    Move X to Table 2
}
A = 0, B = 100
foreach Y in keys(D) {
    if (B == D[Y]) {
        A = A + 1
    }
    if (B > D[Y]) {
        A = 1
        B = D[Y]
    }
}
```

- a. **A** = Cities where students score the highest marks in Mathematics
B = The highest marks in Mathematics
 - b. **A** = Number of cities where students score the highest marks in Mathematics
B = The highest marks in Mathematics
 - c. **A** = Number of cities where students score the lowest marks in Mathematics
B = The lowest marks in Mathematics
 - d. **A** = Always more than one
B = The highest marks in Mathematics
-

Solution: **D** is initialized as an empty dictionary. The scores table is iterated using the iterator **X**. In each iteration, the if-condition check whether the **X.CityTown** is a key in **D**. If the condition evaluates to true, then another if-condition (nested inside) checks whether the Mathematics marks of **X** less than **D[X.CityTown]**. If the condition evaluates to true, then value **D[X.CityTown]** is replaced by **X.Mathematics**. More precisely, **D[X.CityTown]** is updated by a new value if **X.Mathematics** is lesser than it. Then, **D[X.CityTown]** stores the lowest mathematics mark of students from **X.CityTown**. There is no else-block for the nested (inner) if-block. Then we come out of the outer if-block.

The else-part of the outer if-block is executed when **X.CityTown** is not a key of **D**. Here, we create a key **X.CityTown** in **D**, and initialize the value for the key as **X.Mathematics**. At the end of the while-block, **D** captures the following information: For each *CityTown* **C**, **D[C]** is the lowest Mathematics marks of the students from the city **C**.

Variables **A** and **B** are initialized with the value zero. The keys of **D** are iterated using the iterator **Y**. In each iteration, The first if-condition checks whether **B** and **D[Y]** are same. If the condition evaluates to true, then the value of **A** is incremented by one. The second if-condition checks **B > D[Y]**. If the condition evaluates to true, then **A** is set to one and **B** is set to **D[Y]**. Based on the usage of variables, **A** is used as a counter variable and **B** is used to store some value and it is updated when there is a smaller value than **B** is assigned for a key in **D**. Therefore, whenever a key **Y** is encountered with **B > D[Y]**, **B** is updated to **D[Y]**, and **A** is set to one. Whenever a key encountered with **B == D[Y]**, the value of **A** is incremented by one.

Thus, **B** captures the lowest value among the values in **D**, and **A** captures the number of occurrence of the lowest value. Since **D** captures the lowest mathematics marks of each city, **A** and **B** capture the following information:

A = Number of cities where students score the lowest marks in Mathematics

B = The lowest marks in Mathematics

2. The following pseudocode is executed using the “Scores” table. What will the value of **B** represent at the end of execution? [4 Marks]

```
D = { }
while (Table 1 has more rows) {
    Read the first row X in Table 1
    D = updateDictByField(D, X.Physics)
    Move X to Table 2
}
B = findAKey(D)

Procedure updateDictByField(D, Value)
    if (isKey(D, Value)) {
        D[Value] = D[Value] + 1
    }
    else {
        D[Value] = 1
    }
    return (D)
End updateDictByField

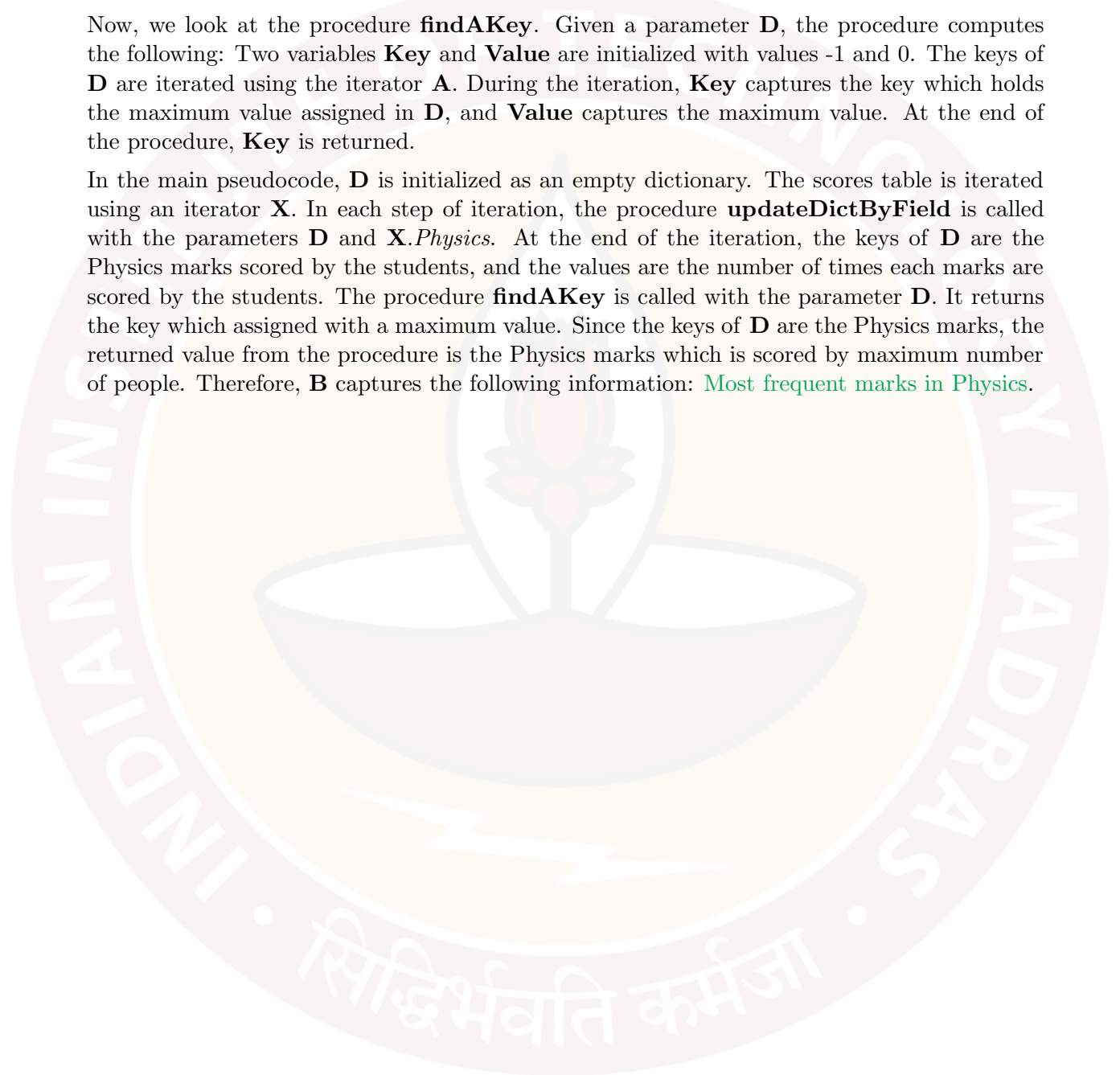
Procedure findAKey(D) {
    Key = -1, Value = 0
    foreach A in keys(D) {
        if (D[A] > Value) {
            Value = D[A]
            Key = A
        }
    }
    return (Key)
End findAKey
```

- a. Number of repeated marks in Physics
- b. Minimum marks in Physics
- c. Maximum marks in Physics
- d. Most frequent marks in Physics

Solution: First we look at the procedure **updateDictByField**. The parameters passed to the procedure are a dictionary and a value. The if-condition checks whether the **Value** is a key of **D**. If the condition evaluates to true, then the value of **D[Value]** is incremented by one. Otherwise, **D[Value]** is initialized with value one. Overall, the procedure updates the dictionary **D** as follows: if **Value** is not a key then create it as a key of **D** and initialize with a value one, otherwise increment the value of **D[Value]** by one.

Now, we look at the procedure **findAKey**. Given a parameter **D**, the procedure computes the following: Two variables **Key** and **Value** are initialized with values -1 and 0. The keys of **D** are iterated using the iterator **A**. During the iteration, **Key** captures the key which holds the maximum value assigned in **D**, and **Value** captures the maximum value. At the end of the procedure, **Key** is returned.

In the main pseudocode, **D** is initialized as an empty dictionary. The scores table is iterated using an iterator **X**. In each step of iteration, the procedure **updateDictByField** is called with the parameters **D** and **X.Physics**. At the end of the iteration, the keys of **D** are the Physics marks scored by the students, and the values are the number of times each marks are scored by the students. The procedure **findAKey** is called with the parameter **D**. It returns the key which assigned with a maximum value. Since the keys of **D** are the Physics marks, the returned value from the procedure is the Physics marks which is scored by maximum number of people. Therefore, **B** captures the following information: **Most frequent marks in Physics**.



3. The following pseudocode is executed using the “Scores” table. What will be the value of **B** at the end of execution? [6 Marks]

```

D = { “Mathematics”: { }, “Physics”: { }, “Chemistry”: { } }, B = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    D = updateDictByField(D, X, “Mathematics”)
    D = updateDictByField(D, X, “Physics”)
    D = updateDictByField(D, X, “Chemistry”)
    Move X to Table 2
}
while (Table 2 has more rows) {
    Read the first row X in Table 2
    if (eligible(D, X) > 1) {
        B = B + 1
    }
    Move X to Table 1
}

Procedure updateDictByField(D, Y, Subject)
    if (isKey(D[Subject], Y.CityTown)) {
        if (D[Subject][Y.CityTown] < Y.Subject) {
            D[Subject][Y.CityTown] = Y.Subject
        }
    }
    else {
        D[Subject][Y.CityTown] = Y.Subject
    }
    return (D)
End updateDictByField

Procedure eligible(D, X) {
    SubList = [ “Mathematics”, “Physics”, “Chemistry” ], C = 0
    foreach Subject in SubList {
        if (D[Subject][X.CityTown] == X.Subject) {
            C = C + 1
        }
    }
    return (C)
End eligible

```

Ans: 10

Solution: The procedure **updateDictByField** receives a dictionary **D**, a row **Y** and a subject **Subject** as parameters. First, we check whether **Y.CityTown** is a key in **D[Subject]**. If it evaluates to true, then we update the value of the key **Y.CityTown** as **Y.Subject** when existing value of the key is less than **Y.Subject**. That is, **D** captures the maximum marks of a student in the subject **Subject** in each city.

The maximum subject marks of the students from each city is given in the following table.

<i>CityTown</i>	<i>Mathematics</i>	<i>Physics</i>	<i>Chemistry</i>
Ambur	81	82	87
Bangalore	97	92	92
Chennai	87	92	92
Erode	72	92	97
Madurai	87	83	89
Nagercoil	62	62	57
Salem	62	45	91
Theni	76	58	90
Trichy	89	86	93
Vellore	93	92	91

The procedure **eligible** receives the dictionary **D** and a student row **X** as parameters. For each subject, we check whether the student is city topper in the subject where the maximum marks of the subject in the city is stored in the dictionary **D**. Therefore, **C** captures the number of subjects in which **X** is a city topper in the subject. The value of **C** for the student information in the “Scores” table can be tabulated as follows.

CardNo	CityTown	Mathematics	Physics	Chemistry	C
6	Ambur	81	82	87	3
11	Bengaluru	97	92	92	3
23	Bengaluru	97	91	88	1
21	Bengaluru	78	69	74	0
9	Bengaluru	63	88	73	0
8	Bengaluru	74	64	51	0
25	Chennai	87	75	92	2
5	Chennai	71	92	84	1
26	Chennai	74	71	82	0
3	Chennai	42	53	78	0
2	Chennai	57	54	77	0
12	Chennai	52	64	71	0
10	Chennai	64	72	68	0
17	Chennai	62	81	67	0
18	Erode	72	92	97	3
0	Erode	68	64	78	0
13	Madurai	65	73	89	1
4	Madurai	87	64	89	2
28	Madurai	74	83	83	1
24	Madurai	44	72	58	0
27	Madurai	81	76	52	0
22	Nagercoil	62	62	57	3
1	Salem	62	45	91	3
15	Theni	76	58	90	3
14	Trichy	89	62	93	2
19	Trichy	56	78	62	0
16	Trichy	87	86	43	1
20	Vellore	93	68	91	2
29	Vellore	72	66	81	0
7	Vellore	84	92	76	1

In the main pseudocode, we check the number of students who are city toppers in more than one subject. That is, the number rows in the above table with **C** value more than one.

4. The following pseudocode is executed using the “Shopping Bills” dataset. At the end of the execution, the variable **D** captures the following information: for each customer **Z**, **D[Z][“Shop”]** stores the shops visited by **Z**, and **D[Z][“Category”]** stores the categories of the items purchased by **Z**. But the pseudocode may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ). [5 marks]

```
1  D = { }
2  while (Pile 1 has more cards) {
3      Read the top card X in Pile 1
4      D = updateDictionary(D, X)
5      Move X to Pile 2
6  }

7  Procedure updateDictionary(D, Y)
8      if (not isKey(D, Y.CustomerName)) {
9          D[Y.CustomerName] = { “Shop”: [], “Category”: [] }
10     }
11     D[Y.CustomerName][“Shop”][Y.ShopName] = True
12     foreach A in Y.ItemList {
13         D[Y.CustomerName][“Shop”][A.Category] = True
14     }
15     return (D)
16 End updateDictionary
```

- a. Error in Line 1
- b. Error in Line 9
- c. Error in Line 12
- d. Error in Line 13
- e. Error in Line 15

Solution: The objective of the pseudocode is to find the following: for each customer **Z**, find the shops visited by **Z**, and the item categories which are bought by **C**. In the pseudocode, **D** is used to store the above information. For each customer **Z**, **D[Z]** is a dictionary with keys “Shop” and “Category”. **D[Z][“Shop”]** stores the shops that are visited by **Z** and **D[Z][“Category”]** stores the categories of the items purchased by **Z**. So, **D[Z][“Shop”]** is a dictionary that assigns true value for all shop visited by **Z**. Similarly, **D[Z][“Category”]** is also a dictionary.

The first step is to read the cards and generate the dictionary **D**. Line 1 - 6 perform the reading and invoke the procedure **updateDictionary**. Clearly, the pseudocode has no error in this part.

Let us look at the procedure **updateDictionary**. Here we have to update the dictionary fields for the key **Y.CustomerName**. The first step is to check whether **Y.CustomerName** is a key in **D**. If it is not a key, then create **X.CustomerName** as a key. The value for the key is defined as follows:

$$\mathbf{D[Y.CustomerName]} = \{ \text{“Shop”}: \{ \}, \text{“Category”}: \{ \} \}$$

In Line 9, the keys are assigned to empty lists. This is a mistake in the pseudocode. Therefore,

Error in Line 9.

Note that the keys are assigned to empty dictionaries since we encounter the customer for the first time. If the customer name exists in the keys list, then we move to the next step. Next we have to add the shop name to the dictionary. This is done in Line 11. Note that the shop name already exists in the dictionary, then it overwrites the true assignment. It will not harm the objective in any aspect. Finally, we check the item categories. Iterate the item list of the bill using the iterator **A**. Add the item category to the dictionary in the field “Category” as follows:

$$\mathbf{D[Y.CustomerName][“Category”][A.Category]} = \mathbf{True}$$

In Line 13, the item category is added to the key “Shop”. Therefore,

Error in Line 13.

At the end of the procedure, the updated dictionary **D** is returned.

5. The following pseudocode is executed using the “**station-wise**” cards of the “Train” dataset. At the end of the execution, **STN** captures the following information: for a station **X**, and a day of a week **A**, **STN[X][A]** stores the number of trains running through **X** on the day **A**. Choose the correct code fragment to complete the pseudocode. [Note: Assume that for each station, the train list is given in a single card.] [4 Marks]

```

STN = { }
while (Pile 1 has more cards) {
    Read the top card X in Pile 1
    STN[X.StationName] = getInfo(STN, X)
    Move X to Pile 2
}

Procedure getInfo(STN, X) {
    *****
    *      Fill the code      *
    *****
    return (D)
End getInfo

```

a.

```

D = { }
foreach A in X.TrainList {
    foreach B in A.Days {
        D[B] = 1
    }
}

```

b.

```

D = { "M": 0, "Tu": 0, "W": 0, "Th": 0, "F": 0, "Sa": 0, "Su": 0 }
foreach A in X.TrainList {
    foreach B in A.Days {
        D[B] = 1
    }
}

```

c.

```

D = { }
foreach A in X.TrainList {
    foreach B in A.Days {
        D[B] = D[B] + 1
    }
}

```

d.

```
D = { "M": 0, "Tu": 0, "W": 0, "Th": 0, "F": 0, "Sa": 0, "Su": 0}  
foreach A in X.TrainList {  
    foreach B in A.Days {  
        D[B] = D[B] + 1  
    }  
}
```



Solution: The dictionary **STN** captures the following information. For each station **X**, and for each day of a week **A**, **STN[X][A]** is an integer that represents the number of trains are running through **X** on the day **A**. First we have to understand the structure of **D**. First of all, **D** is a dictionary where stations are the keys. For each station **X**, **STN[X]** is another dictionary where days of a week are the keys.

In the pseudocode, the cards are iterated using the iterator **X**. In each step of the iteration, **STN[X.StationName]** is obtained from the procedure **getInfo**.

Now we look at the procedure **getInfo**. As we know that **STN[X.StationName]** is a dictionary, the procedure **getInfo** should return a dictionary. Since it returns **D**, **D** should be a dictionary which can be initialized as an empty dictionary.

$$\mathbf{D} = \{ \}$$

With this initialization of **D**, we need to create/initialize keys when they found for the first time. If the key already exists, then we can update the keys. But, none of our answers are checking the existence of any key in **D**. Therefore, **D** should be initialized with keys and initial values for the keys.

We know that the keys of **D** are days of a week. Then, the keys of **D** is fixed and the values are non-negative integers. Therefore, **D** can be initialized as follows:

$$\mathbf{D} = \{ \text{"M": 0, "Tu": 0, "W": 0, "Th": 0, "F": 0, "Sa": 0, "Su": 0} \}$$

We assign the values are zero since no train is encountered for any day so far. Train list of the station is iterated using the iterator **A**.

foreach **A** in **X.TrainList**

In each step of iteration, the running days of **A** is iterated using **B**.

foreach **B** in **A.Days**

For each day **B**, we have to increment one for **D[B]**.

$$\mathbf{D}[\mathbf{B}] = \mathbf{D}[\mathbf{B}] + 1$$

By combining all the above codes, we get the following:

```

D = { "M": 0, "Tu": 0, "W": 0, "Th": 0, "F": 0, "Sa": 0, "Su": 0}
foreach A in X.TrainList {
    foreach B in A.Days {
        D[B] = D[B] + 1
    }
}

```

6. The following pseudocode is executed using the “**station-wise**” cards of the “Train” dataset. Consider the dictionary **STN** computed in the **previous question**. Choose the correct statement(s) from the options based on the pseudocode. [3 Marks]

```
Z = { }  
foreach A in keys(STN) {  
    C = 0, Y = 0  
    D = STN[A]  
    foreach B in keys(D) {  
        if (Y == D[B]) {  
            C = C + 1  
        }  
        if (Y < D[B]) {  
            C = 1  
            Y = D[B]  
        }  
    }  
    if (not isKey(Z, C)) {  
        Z[C] = 0  
    }  
    Z[C] = Z[C] + 1  
}
```

- a. Keys of the dictionary **Z** is integer
- b. There can be a value **C** such that **Z**[**C**] is zero
- c. All values of the dictionary elements are non-zero
- d. The number of keys in **Z** is 8

Solution: First, we understand the pseudocode. **D** is initialized as an empty dictionary. **keys(STN)** is iterated using the iterator **A**. In each step of iteration, **D** stores **STN[A]**. We know that **STN[A]** is a dictionary. In the subsequent foreach-block, we find the maximum value assigned to any key and number of times the value appeared in **D**. These information are stored in **Y** and **C**, respectively.

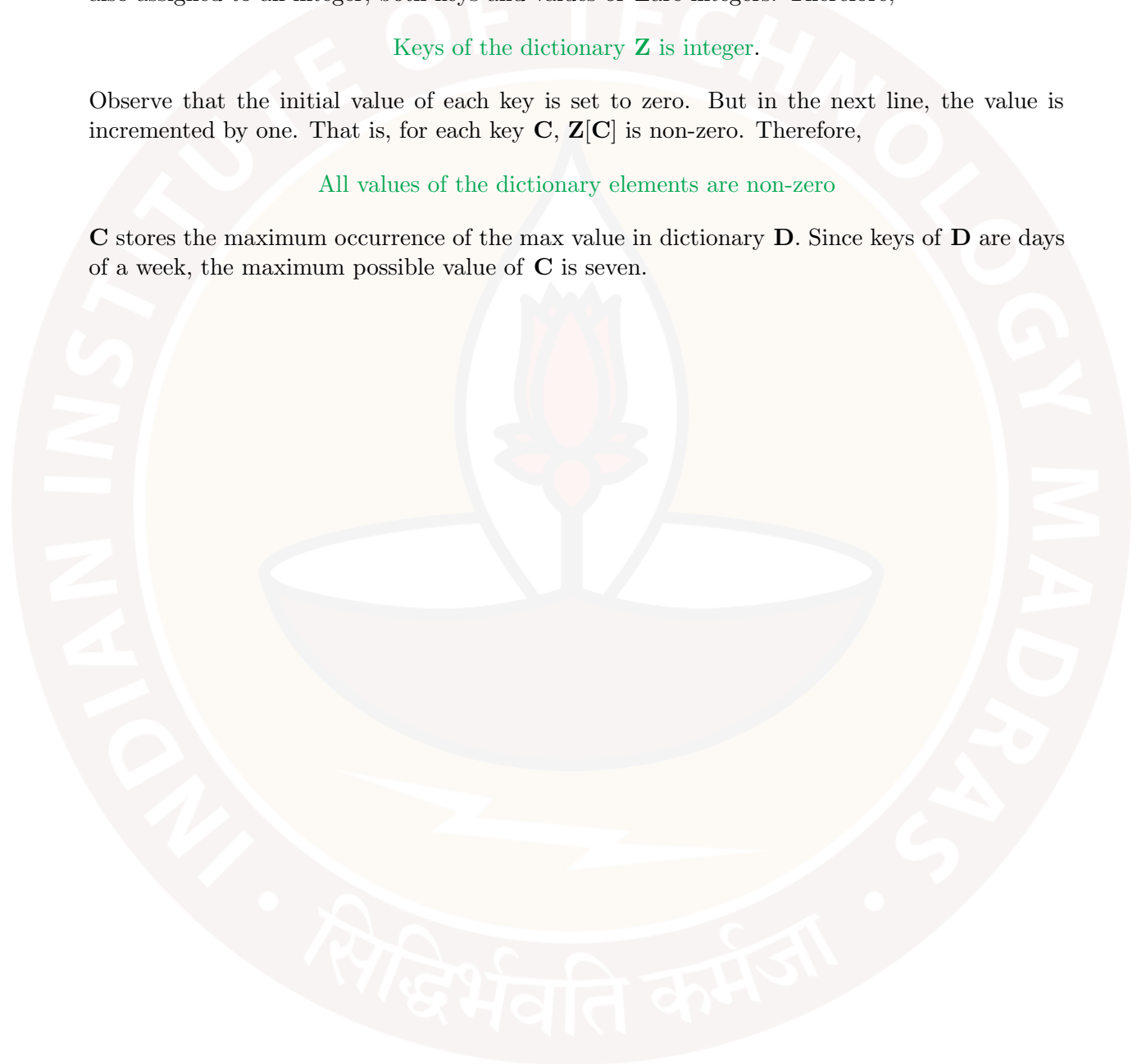
Once we computed **C**, we check whether **C** is a key in **Z**. Since **C** is an integer and **Z[C]** is also assigned to an integer, both keys and values of **Z** are integers. Therefore,

Keys of the dictionary **Z** is integer.

Observe that the initial value of each key is set to zero. But in the next line, the value is incremented by one. That is, for each key **C**, **Z[C]** is non-zero. Therefore,

All values of the dictionary elements are non-zero

C stores the maximum occurrence of the max value in dictionary **D**. Since keys of **D** are days of a week, the maximum possible value of **C** is seven.



7. Consider the dictionary **STN** computed in the **previous question**. Choose the correct pseudocode to compute the number of stations which have trains passing through all days of a week. [4 Marks]

a.

```
Z = 0
foreach A in keys(STN) {
    foreach B in keys(STN[A]) {
        if (STN[A][B] < 1) {
            C = False
        }
    }
    if (C) {
        Z = Z + 1
    }
}
```

b.

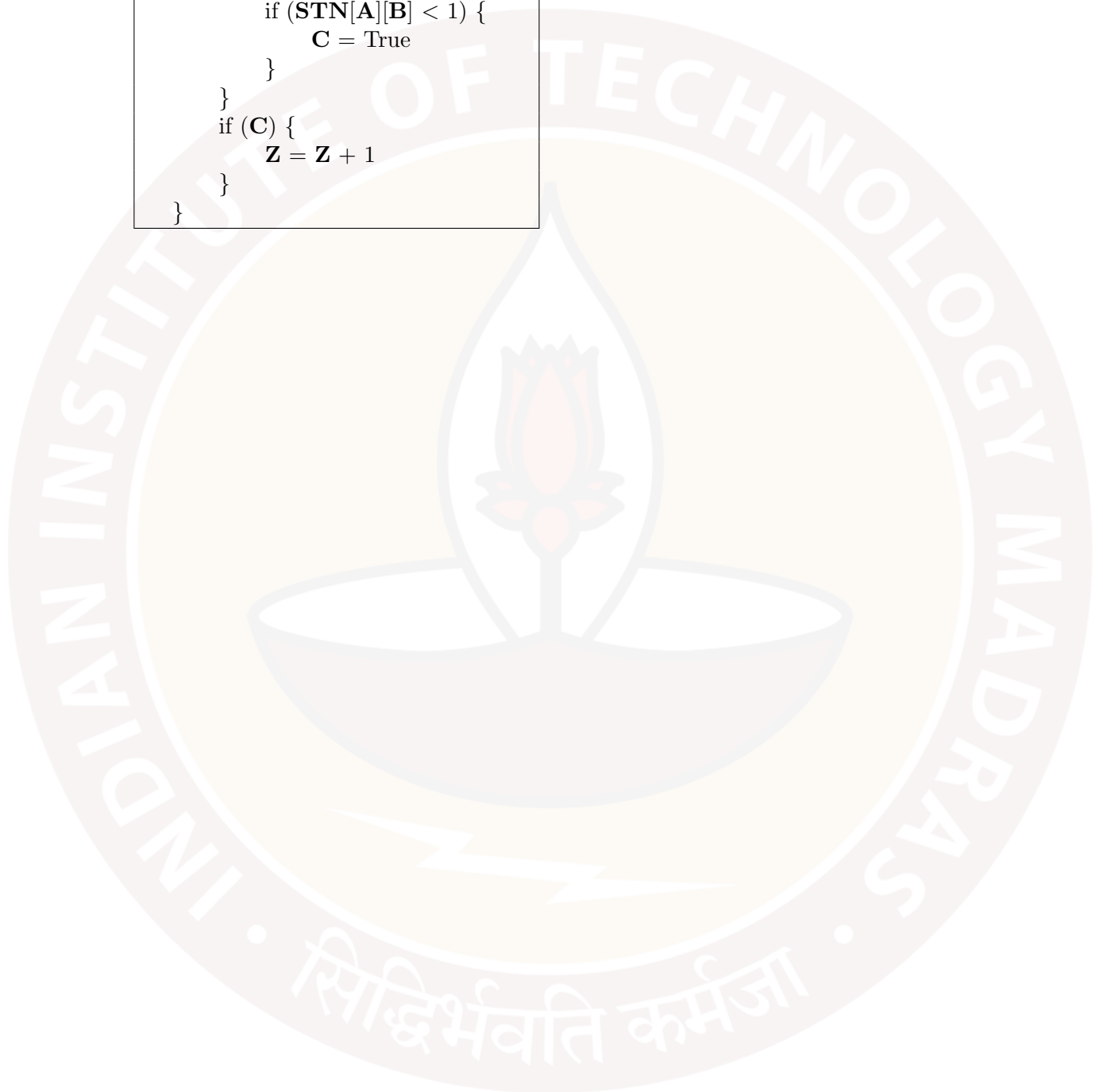
```
Z = 0
foreach A in keys(STN) {
    C = True
    foreach B in keys(STN[A]) {
        if (STN[A][B] < 1) {
            C = False
        }
    }
    if (C) {
        Z = Z + 1
    }
}
```

c.

```
Z = 0
foreach A in keys(STN) {
    C = True
    foreach B in keys(STN[A]) {
        if (STN[A][B] < 1) {
            C = False
        }
        else {
            C = True
        }
    }
    if (C) {
        Z = Z + 1
    }
}
```

d.

```
Z = 0
foreach A in keys(STN) {
    C = False
    foreach B in keys(STN[A]) {
        if (STN[A][B] < 1) {
            C = True
        }
    }
    if (C) {
        Z = Z + 1
    }
}
```



Solution: For a station **A** and a day **B**, **STN[A][B]** denotes the number of trains passing through the station **A** on the day **B**. We iterate the keys of **STN** using **A**, and keys of **STN[A]** is iterated using **B**. The role of **C** is to capture that the station **A** is busy on all days. Then, we should initialize **C** to True.

C = True

If we find a day in which no trains are passing through station **A**, then we have to set **C** to False. That is,

```
if (STN[A][B] < 1) {  
    C = True  
}
```

The variable **Z** should be incremented when **C** remains True at the end of the iteration over the keys of **STN[A]**.

```
if (C) {  
    Z = Z + 1  
}
```

Finally, by combining the pieces of code, we get the following pseudocode.

```
Z = 0  
foreach A in keys(STN) {  
    C = True  
    foreach B in keys(STN[A]) {  
        if (STN[A][B] < 1) {  
            C = False  
        }  
    }  
    if (C) {  
        Z = Z + 1  
    }  
}
```

8. The following pseudocode is executed using the “Words” table. What will the value of **D** represent at the end of the execution? [3 Marks]

```
D = { }
while (Table 1 has more rows) {
    Read the first row X in Table 1
    D = updateDictionary(D, X)
    Move X to Table 2
}

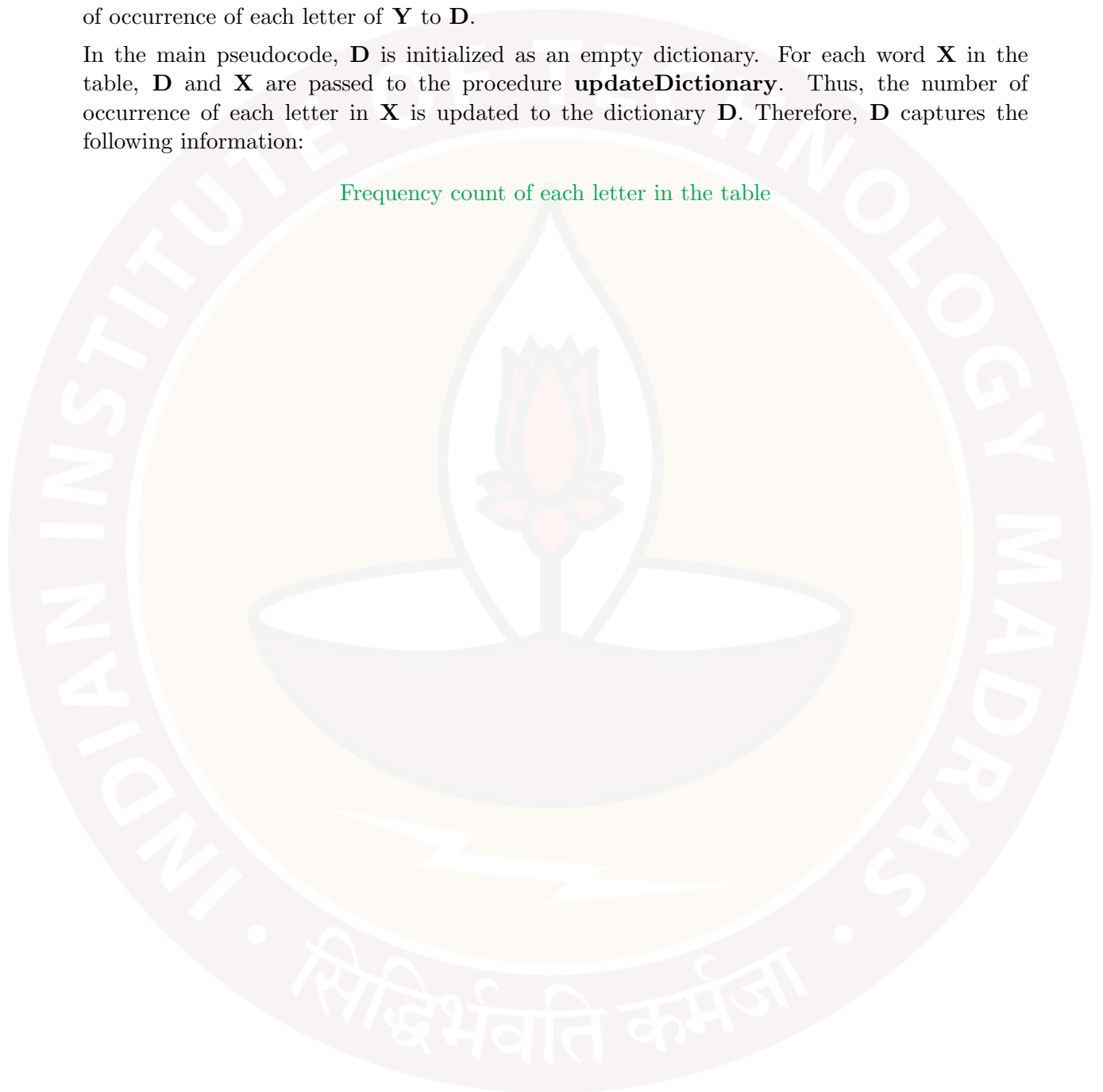
Procedure updateDictionary(D, Y)
    i = 1
    while (i ≤ Y.LetterCount) {
        B = ith letter in Y.Word
        if (isKey(D, B)) {
            D[B] = D[B] + 1
        }
        else{
            D[B] = 1
        }
        i = i + 1
    }
    return (D)
End updateDictionary
```

- a. Frequency count of each letter in the table
- b. Frequency count of each word in the table
- c. Most frequent letter in the table
- d. Frequency count of each letter in each part of speech

Solution: First we consider the procedure **updateDictionary**. It receives a dictionary **D** and a word **Y** as parameters. Based on the usage of **D**, keys of **D** are letters and values are integers. We iterate the letters in the word **Y** using **B**. In the if-condition, we check whether the letter **B** is a key in **D**. If it evaluates to true, then we increment the value of **D[B]** by one. Otherwise, add **B** as a key to **D** and initialize its value to one. Therefore, we add the number of occurrence of each letter of **Y** to **D**.

In the main pseudocode, **D** is initialized as an empty dictionary. For each word **X** in the table, **D** and **X** are passed to the procedure **updateDictionary**. Thus, the number of occurrence of each letter in **X** is updated to the dictionary **D**. Therefore, **D** captures the following information:

Frequency count of each letter in the table



9. Consider the dictionary **D** and the procedure **updateDictionary()** in the **previous question**. Let **POS** be a list that contains all part of speeches. Assume that given a dictionary **D**, there exists a procedure **max** such that **max(D)** returns a list of keys which are mapped to the maximum value. Choose the correct statement(s) from the options based on the following pseudocode. [4 Marks]

```
C = {“Overall” : max(D)}  
foreach A in POS {  
    Move all rows to Table 1  
    B = { }  
    while (Table 1 has more rows) {  
        Read the first row X in Table 1  
        if (X.PartOfSpeech == A) {  
            B = updateDictionary(B, X)  
        }  
        Move X to Table 2  
    }  
    C[A] = max(B)  
}
```

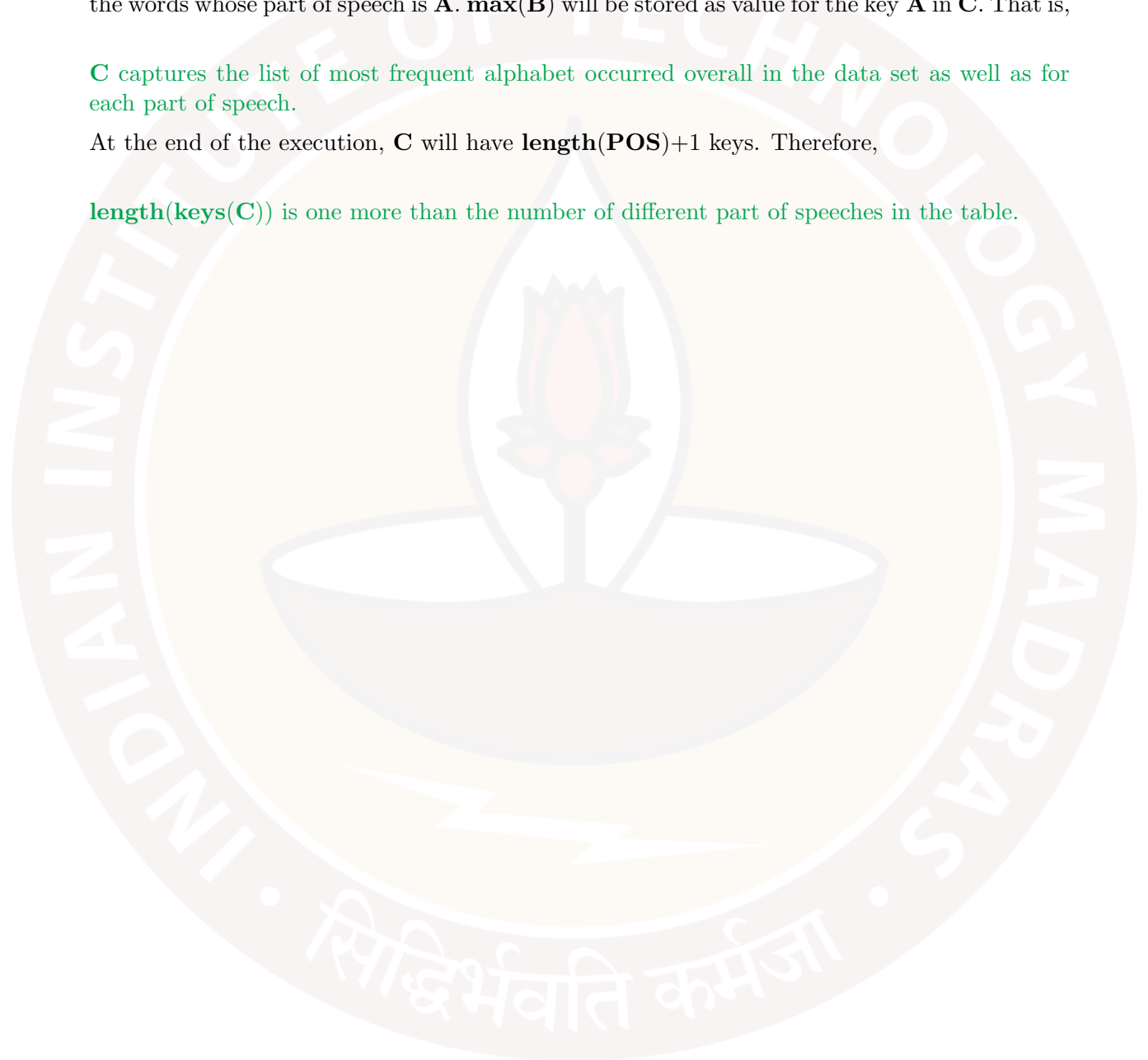
- a. **length(keys(C))** is same as the number of different part of speeches in the input dataset
- b. **C** captures the list of most frequent alphabet occurred in each part of speech
- c. **C** captures the list of most frequent alphabet occurred overall in the data set as well as for each part of speech
- d. **C** captures the most frequent alphabet occurred overall in the data set as well as for each part of speech
- e. **length(keys(C))** is one more than the number of different part of speeches in the table

Solution: The variable **C** is declared as a dictionary and initialized with a key “Overall” with value as **max(D)** where **max(D)** is a list of keys which are assigned to the maximum value. Therefore, **max(D)** is a list of letters with maximum number of occurrence. As it defined, **POS** is a list of part of speeches. **POS** is iterated using **A**. In each iteration, all the rows of “Words” table are moved to Table 1. Rows in the table 1 are iterated using **X**. For each row **X** with part of speech as **A**, the dictionary **D** load the number of occurrence of the letters in the word. That is, at the end of each iteration, **B** stores the frequency count of letter in the words whose part of speech is **A**. **max(B)** will be stored as value for the key **A** in **C**. That is,

C captures the list of most frequent alphabet occurred overall in the data set as well as for each part of speech.

At the end of the execution, **C** will have **length(POS)+1** keys. Therefore,

length(keys(C)) is one more than the number of different part of speeches in the table.



10. The following pseudocode is executed using the “Shopping bills” dataset. What will the value of **D** represent at the end of the execution? [4 Marks]

```

D = { }
while (Pile 1 has more cards) {
    Read the top card X in Pile 1
    D = updateDictionary(D, X)
    Move X to Pile 2
}

Procedure updateDictionary(D, Y)
    foreach A in Y.ItemList {
        C = A.ItemName
        if (isKey(D, C)) {
            if (isKey(D[C], Y.ShopName)) {
                if (D[C][Y.ShopName][“Price”] ≠ A.Price) {
                    D[C][Y.ShopName][“Flag”] = True
                }
            }
            else {
                D[C][Y.ShopName] = { “Price”: A.Price, “Flag” : False }
            }
        }
        else {
            D[C] = { }
            D[C][Y.ShopName] = { “Price”: A.Price, “Flag” : False }
        }
    }
    return (D)
End updateDictionary

```

- For an item **C**, and a shop **S**, **D**[**C**][**S**][“Flag”] is set to True if and only if the item is sold for a constant price
- For an item **C**, and a shop **S**, **D**[**C**][**S**][“Flag”] is set to True if and only if the item is billed in more than one bill
- For an item **C**, and a shop **S**, **D**[**C**][**S**][“Flag”] is set to True if and only if the item is sold for variable price
- For an item **C**, and a shop **S**, **D**[**C**][**S**][“Flag”] is set to True if and only if the item is billed exactly one bill

Solution: **D** is initialized as an empty dictionary. For each card **X** in the dataset, **D** is updated by the procedure **updateDictionary**.

The procedure **updateDictionary** takes a dictionary and a card as parameters. For each item **C** in the item list of **Y**, the if-condition checks whether **C** is a key in **D**. If the condition evaluates to true, then check whether **X.ShopName** is a key in **D[C]**. If the condition evaluates to true, then set the value for the key “Flag” to true when the value of the key “Price” is not equal to **A.Price**. Therefore, in a shop **S**, for an item **C**, if **C** is sold for different prices in the shop **S**, then **D[C][S][“Flag”]** is set to True.

For an item **C**, and a shop **S**, **D[C][S][“Flag”]** is set to True if and only if the item is sold for variable price



BSCCS1001: Week-7, Practice Assignment

1. Consider the following graph generated from the “Scores” table. Each node in this graph corresponds to a student from the table. There is an edge between two nodes if the difference in the physics marks of the corresponding students is at most five.

The graph is represented by the matrix **A**. *SeqNo* is used to label the nodes in the graph. The following pseudocode computes the matrix **A**. Select the correct code fragment to complete it.

```
marks = readPhysicsMarks()
n = length(keys(marks))

*****
*      Fill the code      *
*****

Procedure readPhysicsMarks()
    marks = { }
    while (Table 1 has more rows) {
        Read the first row X in Table 1
        marks[X.SeqNo] = X.Physics
        Move X to Table 2
    }
    return (marks)
End readPhysicsMarks
```

- ✓ **A = createMatrix(n, n)**
 foreach **i** in **keys(marks)** {
 foreach **j** in **keys(marks)** {
 if $(-5 \leq \text{marks}[\mathbf{i}] - \text{marks}[\mathbf{j}] \leq 5)$ {
 A[i][j] = 1
 }
 }
}
- **A = createMatrix(n, n)**
 foreach **i** in **keys(marks)** {
 foreach **j** in **keys(marks)** {
 if $(-5 < \text{marks}[\mathbf{i}] - \text{marks}[\mathbf{j}] < 5)$ {
 A[i][j] = 1
 }
 }
}
- foreach **i** in **keys(marks)** {
 foreach **j** in **keys(marks)** {
 if $(-5 < \text{marks}[\mathbf{i}] - \text{marks}[\mathbf{j}] < 5)$ {
 A[i][j] = 1
 }
 }
}
- **A = createMatrix(n, n)**
 foreach **i** in **keys(marks)** {
 foreach **j** in **keys(marks)** {
 if $(0 \leq \text{marks}[\mathbf{i}] - \text{marks}[\mathbf{j}] \leq 5)$ {
 A[i][j] = 1
 }
 }
}

Solution:

Let us go through the correct answer line by line:

```
A = createMatrix(n, n)
foreach i in keys(marks) {
    foreach j in keys(marks) {
        if (-5 ≤ marks[i] - marks[j] ≤ 5) {
            A[i][j] = 1
        }
    }
}
```

The first line is used to create the matrix **A** with n rows and n columns. We then have a nested iteration to look at the difference in marks of a pair of students. The difference between the marks of students i and j is given by the following expression:

$$| \text{marks}[i] - \text{marks}[j] |$$

There is an edge between i and j if and only if:

$$| \text{marks}[i] - \text{marks}[j] | \leq 5$$

This is the same as:

$$-5 \leq \text{marks}[i] - \text{marks}[j] \leq 5$$

Whenever the above condition evaluates to true, we have an edge between i and j . So, inside the if-block, we have **A**[**i**][**j**] = 1.

2. Consider the following table containing the scores of n students in a class. Assume that all scores are between 0 and 100. The scores in the first and last row are shown as an example. The actual scores may be different. So, refer only to the table's header to answer all questions:

S.No	Score
0	50
$\vdots$	$\vdots$
$n - 1$	89

This information is stored in a dictionary $\mathbf{S}$, with serial numbers as keys and the corresponding scores as values. A graph $\mathbf{G}$ is generated from this table in the following manner:

Given a pair of students (i, j) , there is an edge from i to j if and only if i 's score is strictly greater than that of j . This graph is represented by a matrix $\mathbf{A}$. Assume that the dictionary $\mathbf{S}$ has already been computed. Now, study the following pseudocode.

```

n = length(keys(S))
A = createMatrix(n, n)
foreach r in rows(A) {
    foreach c in columns(A) {
        if (S[r] > S[c]) {
            A[r][c] = 1
        }
    }
}

```

Answer the following questions **after** executing the pseudocode given above.

- (a) A row r of the matrix has three entries that are equal to one and the rest are zero. Which of the following statements about the student r is true?
- ☐ There are exactly three students who have scored more than student r .
 - ☐ There are at least three students who have scored less than student r .
 - ☒ There are exactly three students who have scored less than student r .
 - ☐ There are exactly three students who have scored less than or equal to student r .

Solution:

Let the columns in row r that have three ones in it be j_1, j_2 and j_3 . We have:

$$A[r][j_1] = A[r][j_2] = A[r][j_3] = 1$$

This translates to the fact that there is an edge from r to each of the nodes j_1, j_2 and j_3 . This means that r 's score is strictly greater than that of j_1, j_2 and j_3 . So, there are three students who have scored less than r . So far so good.

But how do we know that there are exactly three students? This is because, had there been another student, say j_4 , who had scored less than r , then $A[r][j_4]$ would have also been one. This can't happen as there are only three ones in the row r of the matrix.

- (b) If r and c are two students with unequal scores, which of the following statements are true? It is a Multiple Select Question (MSQ).

- ☐ If $\mathbf{A}[\mathbf{r}][\mathbf{c}]$ is 1, then $\mathbf{A}[\mathbf{c}][\mathbf{r}]$ is also 1
- ✓ If $\mathbf{A}[\mathbf{r}][\mathbf{c}]$ is 1, then $\mathbf{A}[\mathbf{c}][\mathbf{r}]$ is 0
- ✓ If $\mathbf{A}[\mathbf{r}][\mathbf{c}]$ is 0, then $\mathbf{A}[\mathbf{c}][\mathbf{r}]$ is 1
- ☐ If $\mathbf{A}[\mathbf{r}][\mathbf{c}]$ is 1, then there is insufficient information to determine the value of $\mathbf{A}[\mathbf{c}][\mathbf{r}]$

Solution: Since r and c have unequal scores, we have one of these two possibilities:

$$S[r] > S[c]$$

In this case, there is an edge from r to c . So, $A[r][c] = 1$ and $A[c][r] = 0$.

$$S[c] > S[r]$$

In this case, there is an edge from c to r . So, $A[c][r] = 1$ and $A[r][c] = 0$.

(c) Which of the following statements about the matrix $\mathbf{A}$ are true? It is a Multiple Select Question.

- ☐ There is at least one row that has only ones in it.
- ✓ ☒ There is at least one row that has only zeros in it.
- ☐ There is at least one column that has only ones in it.
- ✓ ☒ There is at least one column that has only zeros in it.

Solution:

Let p be a student who has scored the minimum marks in the class. The row p of the matrix will have only zeros in it. Why? For any student i in the class, the following inequality holds:

$$S[p] \leq S[i]$$

This means that there can never be an edge from p to any other node i . So, $A[p][i] = 0$ for all values of i . Therefore, row p will have only zeros in it.

Let q be a student who has scored the maximum marks in the class. The column q of the matrix will have only zeros in it. Why? For any student i in the class, the following inequality holds:

$$S[q] \geq S[i]$$

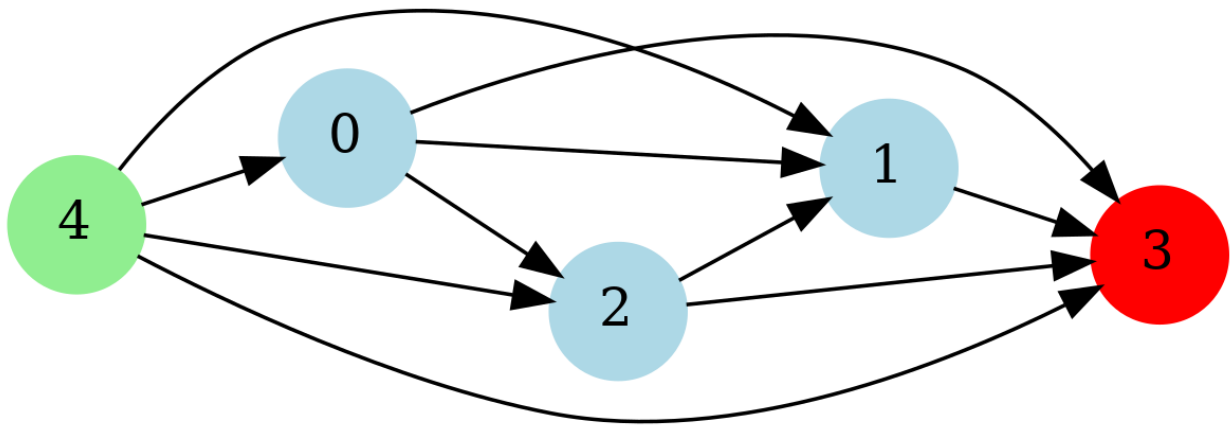
This means that there can never be an edge from any node i to q . So, $A[i][q] = 0$ for all values of i . Therefore, column q will have only zeros in it.

3. Continuing with the previous question, assume that the matrix **A** has been computed. Study the following pseudocode.

```
n = length(keys(S))
k = 0
someScore = S[k]
walk = True
hops = 0
while (walk) {
    walk = False
    foreach c in columns(A) {
        if (A[k][c] == 1) {
            k = c
            someScore = S[k]
            walk = True
            hops = hops + 1
            exitloop
        }
    }
}
```

Let us take up an example to see what is happening here. Consider the following table of five students, the graph G generated from this table, and the matrix representation A of this graph:

S.No	Score
0	70
1	50
2	60
3	40
4	80



	0	1	2	3	4
0	0	1	1	1	0
1	0	0	0	1	0
2	0	1	0	1	0
3	0	0	0	0	0
4	1	1	1	1	0

Now, let us analyze the pseudocode. We start with $k = 0$. The variable **someScore** holds the score of student k . **walk** is initialized to True and **hops** is set to 0 to begin with.

As **walk** is True to begin with, we enter the while loop. Immediately after entering the loop, **walk** is set to False. We shall now take a closer look at the foreach:

```
foreach c in columns(A) {
    if (A[k][c] == 1) {
        k = c
        someScore = S[k]
        walk = True
        hops = hops + 1
        exitloop
    }
}
```

In every step of the foreach, we are checking if there is an edge from k to c . If there is an edge, we update k to c . Subsequently, we update **someScore** to score of the current student k , **walk** is set to True and **hops** is incremented by 1. And finally, we exit the loop.

In summary, this loop performs the function of hopping from the current node k to some neighbor c of k and then it updates k to this new node c . As there is an exitloop command, right after hopping it goes back to the while loop. As **walk** was reset to True, this process will continue. **walk** is a binary switch that tells the computer if it is to continue walking along the edges of the graph.

On the other hand, if there is no edge from k to any other node in the graph, then **walk** will remain False. After exiting the foreach, we will also be exiting the while loop as **walk** is now False, i.e., we are signalling the computer to stop walking.

This is what the pseudocode is trying to achieve:

- Start at a node.
- If this node has a neighbor, then hop to it.
- Keep hopping until you hit a node which has no neighbor.

Answer the following four questions **after** executing the pseudocode given above. The values of all variables mentioned in the following questions represent the state of these variables at the end of the execution of the above pseudocode. All four questions given below are independent of each other.

- (a) Which of the following statements about the variable **someScore** is true, irrespective of the contents of the score-table?
- ☐ It is equal to the score of student 0.
 - ☐ It is equal to the score of student $n - 1$.
 - ✓ ☒ It is the minimum score in the table.
 - ☐ It is the maximum score in the table.

Solution:

We start from the node 0, and then keep hopping one step at a time. At every hop, notice that the score of the student keeps decreasing. That is, if we hop from i to j , then we have $S[i] > S[j]$. This means, when we are no longer able to hop any further, we are at a node which has no outgoing edges. Or, we are at a node k for which $S[k] \leq S[i]$ for all i in 0 to $n - 1$. So, we have reached a student with minimum marks in the table.

This can be seen in the example we have taken up. Starting at node 0, we make the first hop to node 1. From node 1 we hop to node 3. At node 3, there is nowhere else to hop to. So, we exit the while loop. Student 3 (k in general) has scored the minimum marks in the class.

- (b) The value of **hops** is equal to 0. Which of the following statements are true? It is a Multiple Select Question (MSQ).
- ☐ Student k has scored the maximum marks.
 - ✓ ☒ Student k has scored the minimum marks.
 - ✓ ☒ The value of k is equal to 0.
 - ☐ Insufficient information to determine the value of k .

Solution:

If **hops** is zero, it means that it was never updated. The only place where it is updated is inside the if-block. So, the if-block is never executed, i.e., the if-condition never evaluates to True. We can conclude that the node k (in this case 0) has no outgoing edges, and therefore corresponds to a student who has minimum marks.

(c) The variable **hops** satisfies which of the following inequalities?

- ☐ $0 \leq \mathbf{hops} < \frac{n}{2}$
- ☐ $0 \leq \mathbf{hops} \leq \frac{n}{2}$
- ☒ $0 \leq \mathbf{hops} \leq n - 1$
- ☐ $1 \leq \mathbf{hops} \leq n$

Solution:

We have already seen that **hops** can be zero and is never decremented. We only need to worry about the upper bound. Consider the following case:

$$S[0] > S[1] > \cdots > S[n-1]$$

The matrix for this particular case will have ones in all cells above the main diagonal for the following reason:

$$S[i] > S[j] \implies A[i][j] = 1, \text{ for all } j > i$$

If we execute the pseudocode for the graph generated from the above scenario, we will make exactly $n - 1$ hops. The hops will be in this sequence:

- $0 \rightarrow 1$
- $1 \rightarrow 2$
- $\vdots$
- $n - 2 \rightarrow n - 1$

As there are only n nodes, and since we keep hopping to a new node every time, we can make at most $n - 1$ hops for any given graph generated from the table. This gives us the following inequality:

$$0 \leq \mathbf{hops} \leq n - 1$$

- (d) Which of the following statements are true after executing the pseudocode given below? It is a Multiple Select Question (MSQ).

```
sum = 0
foreach c in columns(A) {
    sum = sum + A[k][c]
}
```

- ☒ **sum** is equal to 0.
- ☐ **sum** is equal to $n - 1$.
- ☐ **sum** is the in-degree of node **k** in the graph.
- ☒ **sum** is the out-degree of node **k** in the graph.

Solution:

Please note that the values of all variables mentioned in the question represent the state of these variables at the end of the execution of the main pseudocode. We now know that k corresponds to a student who has scored the minimum marks in the class. In this pseudocode, we are going through each node c in the graph and adding the value $A[k][c]$ to **sum**. This is nothing but the out-degree of the node k in the graph. Since k has no outgoing edges, its out-degree is zero.

4. Consider the graph G that we worked with in the previous two problems. G_r is a graph that is obtained in the following manner:

Given a pair of students (i, j) , there is an edge from i to j if and only if i 's score is strictly less than that of j .

B is a matrix that represents G_r .

- (a) The following pseudocode is used to construct the matrix B from A . Assume that A has already been computed. Select the correct code fragment to complete the pseudocode. It is a Multiple Select Question (MSQ).

```

n = length(keys(A))
B = createMatrix(n, n)
foreach i in rows(A) {
    foreach j in columns(A) {
        *****
        *      Fill the code      *
        *****
    }
}

```

- ☒ $B[i][j] = A[j][i]$
☒ $B[j][i] = A[i][j]$
☐ if ($A[i][j] == 0$) {
 $B[i][j] = 1$
 }
☒ if ($A[j][i] == 1$) {
 $B[i][j] = 1$
 }
☐ $B[i][j] = 1 - A[i][j]$

Solution:

We call G_r the reverse graph of G . If there is an edge from $i \rightarrow j$ in G , then there is an edge from $j \rightarrow i$ in G_r . To see why this is true, notice that if there is an edge from $i \rightarrow j$ in G , then $S[i] > S[j] \implies S[j] < S[i]$, so there is an edge $j \rightarrow i$ in G_r .

This also works the other way round. That is, if there is edge from $i \rightarrow j$ in G_r , then $S[i] < S[j] \implies S[j] > S[i]$, from which it follows that there is an edge from $j \rightarrow i$ in G .

So, there is a bijective mapping between the edges of the two graphs G and G_r . Whenever $A[i][j]$ is equal to one, we have to set $B[j][i]$ equal to one. This is equivalent to saying: whenever $A[j][i]$ is equal to one, we have to set $B[i][j]$ equal to one.

So, we iterate through the rows and columns of A and make sure that whenever $A[j][i]$ is one, we set $B[i][j]$ to one. This can be done as follows:

```
if (A[j][i] == 1) {
    B[i][j] = 1
}
```

The same thing can be achieved without using the if-condition.

```
B[i][j] = A[j][i]
```

or

```
B[j][i] = A[i][j]
```

If you are familiar with matrices and transposes, this is the same as saying that B is the transpose of A . For this reason, the reverse graph is also called the transpose graph. The pseudocode is trying to do just that: compute the transpose of the matrix A and store it in B .

- (b) For a pair of nodes (i, j) , the following expression evaluates to true:

$\mathbf{A[i][j] == B[i][j]}$

Based on the above observation, which of the following statements evaluates to true? $\mathbf{S}$ is the dictionary that represents the table in the previous question.

- ☒ $\mathbf{S[i] == S[j]}$
- ☐ $\mathbf{S[i] > S[j]}$
- ☐ $\mathbf{S[i] < S[j]}$
- ☐ Insufficient information

Solution: If $A[i][j] = B[i][j]$, there could be two possibilities.

$$A[i][j] = B[i][j] = 1$$

This cannot happen since $S[i] > S[j]$ and $S[j] > S[i]$ cannot be true at the same time. This leaves us with:

$$A[i][j] = B[i][j] = 0$$

We have $S[i] \leq S[j]$ and $S[i] \geq S[j]$, which implies $S[i] = S[j]$, i.e., the students i and j have scored equal marks.

COMPUTATIONAL THINKING

Quiz II Mock

Time: 45 Mins

Total Marks: 25 Marks

1. Let **LA** be a non-empty list of integer. A list **LB** is constructed using the following pseudocode.

Select the following statements are True or False.

[5 Marks]

```
LC = LA
LB = [LA]
while (length(LC) > 1) {
    LC = rest(LC)
    LB = LB ++ [LC]
}
```

- i. **length**(**LB**) == **length**(**LA**) - 1
 - a. True
 - b. False
 - ii. At the end of the execution, **LC** == []
 - a. True
 - b. False
 - iii. Each element of **LB** is a copy of **LA**
 - a. True
 - b. False
 - iv. **last**(**LB**) = **LA**
 - a. True
 - b. False
 - v. Each element of **LB** is a non-empty list
 - a. True
 - b. False
-

Solution: The **LA** is already defined as a non-empty list of integer. **LC = LA** assigns the entire value in **LA** into **LC**. A nested list is made with the list **LA** and assigned to the variable **LB**. The while loop is executed based on the condition that **length(LC) > 1**, thus the while loop will execute until the length of the list becomes 2 after that condition will be false, contents inside the loop won't be executed. Until **length(LC) > 1** is True program will execute the contents inside the while loop, there **LC** is replaced by the list **rest(LC)**, after that the a nested list is created using the **LC** and appended to the existing nested list **LB**.

- i. In the assignment **LB = [LA]**, **LB** is initialized as nested list with one list **LA**. After that the while loop is executed n-1 times there the one list is appended into **LB**, where n is the length of the list **LA**. Hence n number of elements present in the list **LB**, thus **length(LB) == length(LA)**. So the answer is False.
- ii. At the end of execution the last element in the list **LC** is still present, hence the answer is False.
- iii. Each element in the **LB** is a list, thus it is not an exact copy of **LA**. Thus the answer is False.
- iv. From the code, the last element of the **LB** is the list of single element, which is last element of **LA**. Hence the answer is False.
- v. The while loop executed up to the **length(LC) > 1**, hence the **LC** will not become empty list and **LB** is appended with the list made up of **LC**. Thus there is no possibility to get an empty list in **LB**. Hence the answer is True.

2. Two words **X** and **Y** are said to be *anagram* pairs if **Y** is phrased by rearranging letters of **X**. The following pseudocode is executed using the “Words” table to count the number of anagram pairs in the table. Choose the correct code fragment to complete the pseudocode. It is a Multiple Select Question (MSQ). [6 Marks]

```

C = 0
while (Table 1 has more rows) {
    Read the first row X in Table 1
    Move X to Table 2
    while (Table 1 has more rows) {
        Read the first row Y in Table 1
        if (anagramPair(X, Y)) {
            C = C + 1
        }
        Move Y to Table 3
    }
    Move all rows from Table 3 to Table 1
}

Procedure anagramPair (X, Y) {
    DX = getDict(X)
    DY = getDict(Y)
    *****
    *      Fill the code      *
    *****

End anagramPair

Procedure getDict (X) {
    i = 1, D = { }
    while (i ≤ X.LetterCount) {
        Z = ith letter of X.Word
        if (not isKey(D, Z)) {
            D[Z] = 0
        }
        D[Z] = D[Z] + 1
        i = i + 1
    }
    return (D)
End getDict

```

a.

```
foreach A in keys(DX) {  
    if (DX[A] ≠ DY[A]) {  
        return (False)  
    }  
}  
foreach A in keys(DY) {  
    if (DX[A] ≠ DY[A]) {  
        return (False)  
    }  
}  
return (True)
```

b.

```
foreach A in keys(DX) {  
    if (not isKey(DY, A)) {  
        return (False)  
    }  
    if (DX[A] ≠ DY[A]) {  
        return (False)  
    }  
}  
return (True)
```

c.

```
foreach A in keys(DX) {  
    if (not isKey(DY, A)) {  
        return (False)  
    }  
    if (DX[A] ≠ DY[A]) {  
        return (False)  
    }  
}  
foreach A in keys(DY) {  
    if (not isKey(DX, A)) {  
        return (False)  
    }  
    if (DX[A] ≠ DY[A]) {  
        return (False)  
    }  
}  
return (True)
```

d.

```
if (length(keys(DX)) == length(keys(DY))) {  
    foreach A in keys(DX) {  
        if (DX[A] ≠ DY[A]) {  
            return (False)  
        }  
    }  
}  
return (True)
```

e.

```
if (length(keys(DX)) ≠ length(keys(DY))) {  
    return (False)  
}  
foreach A in keys(DX) {  
    if (not isKey(DY, A)) {  
        return (False)  
    }  
    if (DX[A] ≠ DY[A]) {  
        return (False)  
    }  
}  
return (True)
```

Solution: The question is to check two words **X** and **Y** is an anagram or not, where the **Y** is the word made up of rearranging the letters in the word **X**. The pseudocode is based on the "Words" Table.

A variable **C** is created and initialized as 0. The while loop will execute until there is a row in the Table 1. **X** is the first row in Table 1 and which is moved to Table 2. The next while loop will be executed until there is a row in Table 1. **Y** is the first row in Table 1. The next if statement is executed only the output from the procedure **anagramPair(X, Y)** is True. If the **anagramPair(X, Y)** is True then the variable **C** is incremented by 1. There after **Y** is moved to Table 3. After the completed execution of nested while all the rows from Table 3 is moved to Table 1.

The procedure **anagramPair** takes two variable **X** and **Y**. Inside which another procedure called **getDict** is called there a word is given as argument that return a dictionary with the letters as key and number of occurrence of the letter as value. **DX** and **DY** is initialized and defined as a dictionary returned from the procedure **getDict**.

The answer is option c and e. In option c the first foreach loop checks whether the letters of **DX** is present in **DY** and its counts are equal. The second foreach loop checks whether the letters of **DX** is present in **DY** and its counts are equal. The **isKey(DY, A)** evaluates whether the letter **A** is present as a key in **DY** and occurrences are evaluated by the next if-statement having the condition **DX[A]≠DY[A]** and similar for the second foreach loop also.

In option e the first if statement checks for the length of keys of dictionaries **DX** and **DY**, which checks the number of letters present in the **DX** and **DY**. The second foreach loop checks whether the letters of **DX** is present in **DY** and its counts are equal. The **isKey(DY, A)** evaluates whether the letter **A** is present as a key in **DY** and occurrences are evaluated by the next if-statement having the condition **DX[A]≠DY[A]**. The value True is returned if none of the if-statement conditions are satisfied.

3. The following pseudocode is executed using the “Words” table. At the end of the execution, **C** captures the number of pairs of words without any common letters. But the pseudocode may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ).[5 marks]

```

1  C = 0
2  while (Table 1 has more rows) {
3      Read the first row X in Table 1
4      Move X to Table 2
5      LX = getList(X)
6      flag = True
7      while (Table 1 has more rows) {
8          Read the first row Y in Table 1
9          LY = getList(Y)
10         foreach A in LY {
11             if (not member(LX, A)) {
12                 flag = False
13             }
14         }
15         if (flag) {
16             C = C + 1
17         }
18         else {
19             flag = True
20         }
21         Move X to Table 3
22     }
23     Move all rows from Table 3 to Table 1
24 }

25 Procedure getList(X)
26     L = [ ]
27     i = 1
28     while (i ≤ X.LetterCount) {
29         A = ith letter in X.Word
30         if (member(L, A)) {
31             L = L ++ [A]
32         }
33     }
34     return (L)
35 End getList

```

- a. Line 11: Incorrect conditional statement
- b. **flag** should be declared/initialized after Line 7
- c. **flag** is not set to True before Line 22

- d. Error in Line 23
- e. **i is not incremented in the while-loop in Procedure getList**
- f. **Line 30: Incorrect conditional statement**

Solution: The pseudocode is written to calculate number of pair of words without any common letters using "Words" table and which is stored in **C**. In the pseudocode, **C** is initialized with the value 0 and while loop will execute until Table 1. The first row **X** of Table 1 is read and moved to Table 2. **LX** is initialized with the value returned from the **getList(X)** and another variable **flag** is created and initialized as True. The nested while loop in line 7 will be executed when there is more rows in Table 1 and the first row in Table 1 **Y** is read. Another variable is defined and initialized with the value returned from the procedure **getList(Y)**, where **Y** is the word given as argument to the procedure. The **getList(Y)** returns the list of letters used in the word **Y**.

The foreach loop in line 10 is executed for each letter **A** in the list **LY**, checked for the letter in word **Y** is present in the word **X**, **LX** have the letters used in the word **X**.

The if-statement in Line 11 should be if (**member(LX, A)**) to make the **flag** = False.

If the flag is still True only when there no common letters between two words **X** and **Y** then program goes through the if-statement in line 15 and **C** will be incremented by 1 or the else-statement is executed to make **flag** to True. Then the word **X** is moved to Table 3. Once the while loop in line 7 is completed every rows from Table 3 is again moved back to Table 1.

In the procedure **getList**, it takes an argument **X** and returns list **L**. **L** is defined and initialized as an empty list and **i** is another variable initialized with value 1 to track the position of letter in a word. The while loop in the line 28 executed when the value of **i** is less than number of the letters present in the word **X**. In line 29, a new variable **A** is defined and initialized with the letter of **ith** position.

The if-statement in the line 30 should be if(**not member(L,A)**) to append the letter **A** into the list **L** in line 31.

After the completion of if-statement and before closing braces of while loop, variable **i** has to be incremented by 1 otherwise while-loop in line 28 execute indefinitely and to keep track of next position of the letter.

4. Let **LA** and **LB** be two sorted lists of integers in ascending order. The procedure **merge()** constructs a list **LC** by merging **LA** and **LB** into a single sorted list. But the procedure may have mistakes. Identify all such mistakes (if any). It is a Multiple Select Question (MSQ). [5 Marks]

```
1  Procedure merge(LA, LB)
2      LC = { }
3      while (length(LA) > 0 or length(LB) > 0) {
4          X = first(LA)
5          Y = first(LB)
6          if (X <= Y) {
7              LC = LC ++ [X]
8              LA = rest(LA)
9          }
10         else{
11             LC = LC ++ [Y]
12             LB = rest(LB)
13         }
14     }
15     while (length(LA) > 0) {
16         LC = LC ++ [first(LA)]
17         LA = rest(LA)
18     }
19     while (length(LB) > 0) {
20         LC = LC ++ [first(LB)]
21         LB = first(LB)
22     }
23     return (LC)
24 End merge
```

- a. Line 1: **LC** should be initialized as an empty list
- b. Line 3: Condition expression should use “and” operator
- c. Lists **LA** and **LB** does not updated correctly
- d. Line 7: **Y** should be appended to **LC**
- e. Error in Line 17
- f. Error in Line 21

Solution: **LA** and **LB** are predefined list sorted in ascending order, the procedure **merge** takes two list **LA** and **LB** to return a list which is a merged list of **LA** and **LB**.

In the line 2, a new variable **LC** has to be initialized as a list instead of dictionary **LC = []**

In the line 3, while-loop should take the condition **length(LA) > 0** and **length(LB) > 0** otherwise one of the list can be entered in to while-loop which can be empty and **X** can not take any value from the empty list.

In line 4 and 5, the **X** and **Y** is initialized with first element of the list **LA** and **LB** respectively. If the **X** is less than **Y** it will be appended in to the list **LC** with the if-statement in line 6. **LA** is replaced with the rest of the elements other than the first element of **LA**, if the **X** is greater than **Y** then else-statement will be executed, where the **Y** is appended in to the list **LC** and **LB** is replaced with the rest of elements in the list of **LB** other than the first element.

After the completion of while-loop in line 3, another while-loop in line 15 will be executed if still any elements present in the **LA** using the condition **length(LA) > 0**, **LC** is appended with the first element of **LA** and **LA** is replaced with **rest(LA)**. Similarly, elements in the **LB** is appended in to **LC** with the while-loop in line 19 with the **LB = rest(LB)** instead of **LB = first(LB)**. Finally, **LC** is returned with merged list of sorted integers.

5. The details of persons tested for Covid 19 for the year 2020 is recorded in the dictionary **Covid** as follows: for each city **X**, for each patient **Y** in **X**, and for a day **Z** on which the test is taken for **Y**, **Covid[X][Y][Z]** = *Positive/Negative*. Date of test is recorded from 0 to 365 (0 : Jan 1, . . . , 365 : Dec 31).

The following pseudocode is executed using the Covid-19 dataset. Choose the correct statement based on the pseudocode. [4 Marks]

```

Total = { }
foreach X in keys(Covid) {
    Patients = 0
    foreach Y in keys(Covid[X]) {
        A = 0
        foreach Z in keys(Covid[X][Y]) {
            if(A < Z) {
                A = Z
            }
        }
        if (Covid[X][Y][A] == "Positive") {
            Patients = Patients + 1
        }
    }
    Total[X] = Patients
}
Max = 0, City = [ ]
foreach X in keys(Total) {
    if (Total[X] == Max) {
        City = City ++ [X]
    }
    if (Total[X] > Max) {
        Max = Total[X]
        City = [X]
    }
}

```

- Variable **City** stores exactly one city with the maximum number of current Covid patients
- Variable **City** stores list of cities with the maximum number of current Covid patients
- Variable **City** stores all cities with current Covid patients
- Variable **City** stores list of cities with the minimum number of current Covid patients

Solution: The question used a dictionary to store the data using the Covid 19 for the year 2020. The dictionary is predefined and initialized in a way that $\text{Covid}[\text{City}][\text{Patient}][\text{Date}] = \text{Positive/Negative}$. The test date is stored in a ways that 0 being January 1th and 365 being December 31th that 2020 is a leap year.

A new variable **Total** is defined and initialized as empty dictionary and the first foreach-loop **X** takes the keys of **Covid** which are the cities and introduces a new variable **Patients** and initialized to 0. The nested foreach-loop **Y** takes keys of **Covid[X]** which are the patients in the city **X** and introduces another variable **A** and initialized to 0. The next nested foreach-loop **Z** takes the keys of **Covid[X][Y]** the date in which the test is taken for the patient **Y** in the city **X**.

Inside the innermost for-loop an if-statement is introduced to check whether **A** is lesser than **Z**, if the condition is true then **A** stores the value of **Z**, which is the test date.

After completion of inner most for-loop another if-statement is introduced to check whether the Patient is tested *Positive* on the date, if they are tested positive, the **Patients** is incremented by 1 using **Patients = Patients + 1**.

After completion of second for-loop, the dictionary **Total** with the key **X** is assigned a value of **Patients** using **Total[X] = Patients**.

After completion of outermost for-loop, two new variables **Max** and **City** are defined and initialized with 0 and empty list.

The foreach-loop **X** iterates over the keys of **Total**, and checks for the condition that **Total[X] == Max** then the list **City** is appended with the city **X** using the if-statement. After that another if-statement is introduced to check whether the **Total[X] > Max** then **Max** is replaced with the value of **Total[X]** and the list **City** is replaced with the list of one element **X**.

At the end of the foreach-loop the variable **City** stores the list of cities where the maximum number of Covid patients exists.